Feedback-Coupled Cryptographic Observation Protocol

A Deterministic Integrity Framework for Feedback-Coupled Agentic Systems

Inventors:

Mohit Ranjan (Student Researcher)
Dr. Ram Chandra Barik

Application Type:

International Patent Application (PCT)

Technical Field:

Cryptographic Telemetry Systems, Distributed Systems Integrity, Machine Learning Infrastructure, Zero-Trust Architectures

Document Classification:

Technical Specification & Patent Disclosure - 2026
Version 1.0 (Submission Draft)

Confidentiality Notice:
This document contains proprietary and confidential technical information related to the Feedback-Coupled Cryptographic Observation (RCO) Protocol. Unauthorized reproduction, distribution, or disclosure is strictly prohibited except as required for patent examination and legal review.

Abstract

The present disclosure describes a system and method for ensuring deterministic integrity and verifiable lineage in feedback-coupled computational environments, particularly within agentic and machine learning systems where telemetry influences internal model evolution. Traditional telemetry pipelines lack guarantees of immutability, cross-platform determinism, and resistance to adversarial manipulation, resulting in inconsistent or unverifiable system states.

The disclosed Feedback-Coupled Cryptographic Observation (RCO) protocol introduces a cryptographic framework that binds telemetry data to its originating computational state through deterministic serialization, chained hashing, and verifiable proof mechanisms. In one embodiment, telemetry batches are encoded using a canonical serialization scheme with fixed-precision normalization to eliminate numerical drift across heterogeneous architectures. Each batch is linked to a prior state using a cryptographic hash chain, thereby establishing an immutable lineage.

The system further incorporates a multi-party validation mechanism requiring a threshold of independent signatures to authorize ingestion, preventing unilateral data injection and mitigating single-point compromise. Additional embodiments include scalable ingestion pipelines, zero-trust communication models, and optional analytical layers for anomaly detection.

The disclosed approach enables reproducible, tamper-evident, and audit-grade telemetry across distributed systems, improving the reliability of feedback-driven computational processes and supporting high-assurance environments such as large-scale simulations, reinforcement learning systems, and autonomous decision frameworks.

Acknowledgements

The development of the Feedback-Coupled Cryptographic Observation (RCO) Protocol has been the result of sustained research, engineering rigor, and critical evaluation of existing limitations in distributed computational systems.

The primary conceptualization, system design, and technical development were led by Mohit Ranjan. The author acknowledges the academic guidance, technical discussions, and research support provided by Dr. Ram Chandra Barik, whose insights contributed to the formalization and refinement of the underlying framework.

The work also reflects cumulative understanding derived from prior research in cryptography, distributed systems, and machine learning infrastructure. The author recognizes the broader scientific and engineering community whose foundational contributions in these domains enabled the development of the present disclosure.

No external funding, institutional sponsorship, or third-party proprietary frameworks were used in the development of this protocol, unless otherwise explicitly stated in the relevant sections.

The responsibility for the content, claims, and technical assertions made in this document rests solely with the inventors.

Definitions

For the purposes of this disclosure, the following terms shall have the meanings set forth below. These definitions are intended to provide clarity and shall not be construed to limit the scope of the invention unless explicitly stated.

Definition .1 (Feedback-Coupled Agentic System (FCAS))**.** A computational system in which telemetry outputs are not passive records but actively influence subsequent internal state transitions, including but not limited to model weights, policy updates, or decision functions.

Definition .2 (Telemetry)**.** Structured data generated during execution of a computational process, including observations, actions, rewards, or state transitions, which may be used for monitoring, training, or feedback.

Definition .3 (Deterministic State)**.** A system condition in which identical inputs, including telemetry disclosures, result in identical internal computational outcomes across executions and across heterogeneous hardware environments.

Definition .4 (Canonical Serialization)**.** A deterministic encoding process in which structured data is transformed into a byte sequence using a fixed and unambiguous ordering, representation, and precision, such that identical logical inputs always yield identical binary outputs.

Definition .5 (Fixed-Precision Scaling)**.** A numerical normalization process in which floating-point values are transformed into integer representations using a predefined scaling factor, thereby eliminating inconsistencies arising from floating-point arithmetic across architectures.

Definition .6 (Merkle-Chain (Lineage Chain))**.** A cryptographic structure in which each data block is hashed and linked to the hash of a preceding block, forming a sequential chain that enables detection of any modification, omission, or reordering of historical data.

Definition .7 (Hash Function)**.** A deterministic algorithm that maps input data of arbitrary size to a fixed-size output, such that it is computationally infeasible to reverse or to find collisions under standard cryptographic assumptions.

Definition .8 (Proof of Reflexion (PoR))**.** A cryptographic proof mechanism that binds a telemetry disclosure to the originating computational state or agent, ensuring that the data was generated by an authorized and consistent internal process.

Definition .9 (Threshold Signature Scheme)**.** A cryptographic protocol in which a valid signature is produced only when a predefined minimum number of independent participants (threshold t out of n) contribute partial signatures.

Definition .10 (Zero-Trust Model)**.** A security model in which no component, node, or process is inherently trusted, and every interaction must be explicitly verified through cryptographic or formal validation mechanisms.

Definition .11 (Telemetry Batch)**.** A discrete unit of telemetry data grouped for processing, serialization, hashing, and verification within the system.

Definition .12 (Lineage Integrity)**.** The property that the complete sequence of telemetry batches is preserved without alteration, deletion, or reordering, and can be cryptographically verified end-to-end.

Definition .13 (Adversary)**.** A probabilistic computational entity capable of attempting to manipulate, forge, replay, or disrupt telemetry data, system state, or communication channels within defined computational bounds.

Definition .14 (Audit-Grade Verification)**.** A level of assurance in which system outputs and historical data can be independently verified for correctness, authenticity, and completeness using cryptographic and procedural methods.

Definition .15 (Ingestion)**.** The process by which telemetry data is received, validated, cryptographically anchored, and stored within a persistent system.

Definition .16 (Replay Attack)**.** An adversarial action in which previously valid data is retransmitted or reused to produce unintended or fraudulent system outcomes.

Definition .17 (Semantic Validity)**.** The property that telemetry data not only satisfies structural and cryptographic checks but also conforms to expected behavioral or domain-specific constraints of the system.

Definition .18 (Heterogeneous Architecture)**.** A computing environment composed of multiple hardware or software platforms with differing numerical, computational, or execution characteristics.

Definition .19 (Immutable Ledger)**.** A data structure in which recorded entries cannot be altered retroactively without detection through cryptographic verification.

Definition .20 (Sidecar Component)**.** An auxiliary process or service that operates alongside a primary application to provide additional functionality such as monitoring, logging, or cryptographic processing without modifying the core application logic.

Glossary

Term / Acronym	Description
RCO	Feedback-Coupled Cryptographic Observation Protocol
FCAS	Feedback-Coupled Agentic System
PoR	Proof of Reflexion; cryptographic mechanism binding telemetry to originating computational state
MVR	Minimal Viable RCO; baseline configuration ensuring core integrity guarantees
SRE	Site Reliability Engineering; operational discipline for maintaining system reliability and performance
TSS	Threshold Signature Scheme; cryptographic method requiring multiple parties to generate a valid signature
MPC	Multi-Party Computation; distributed cryptographic technique for joint computation without revealing private inputs
SHA3-256	Cryptographic hash function based on the Keccak algorithm, producing a 256-bit output
Merkle-Chain	Sequential cryptographic structure linking data blocks via hash dependencies
Bencode	Deterministic serialization format with lexicographical ordering and length-prefix encoding
Canonical Serialization	Encoding process ensuring identical binary output for identical logical data
Fixed-Precision Scaling	Transformation of floating-point values into integer representations using a predefined scaling factor
Zero-Trust	Security model where no component is inherently trusted and all actions require verification
Telemetry	Structured runtime data generated during system execution
Telemetry Batch	Grouped unit of telemetry processed and verified as a single entity
Ingestion	Process of receiving, validating, and storing telemetry data
Lineage	Ordered sequence of telemetry batches linked cryptographically

Term / Acronym	Description
Integrity Proof	Cryptographic evidence verifying authenticity and correctness of telemetry
Auditor Node	Independent verification entity participating in validation and signature generation
Agent Node	Computational entity generating telemetry data
Sidecar	Auxiliary component operating alongside a primary system to provide additional functionality
gRPC	High-performance remote procedure call framework used for communication between distributed components
io_uring	Linux kernel interface for asynchronous I/O operations with reduced syscall overhead
AVX-512	CPU instruction set enabling parallel computation on 512-bit data registers
TPM	Trusted Platform Module; hardware-based security component for key storage and attestation
PCR	Platform Configuration Register; TPM register used for system state attestation
TDA	Topological Data Analysis; method for analyzing structural properties of data
Lyapunov Stability	Mathematical concept used to determine stability of dynamic systems
Deterministic State	System condition yielding identical outputs for identical inputs across executions
Immutable Ledger	Storage structure where data cannot be altered without detection
Replay Attack	Reuse of previously valid data to produce unintended system effects

Contents

List of Figures

List of Tables

A. Executive Engineering Mandate

A.1 Technical Context and Problem Statement

The evolution of modern computational systems has led to the emergence of architectures in which the distinction between observation and computation is no longer strictly separable. In classical computational paradigms, a system is modeled as a mapping from inputs to outputs, wherein observational data is treated as an external artifact that does not influence the internal state transition dynamics. However, contemporary systems, particularly those arising in machine learning, reinforcement learning, adaptive control, and autonomous decision-making domains, exhibit a fundamentally different structure characterized by feedback-coupled interactions between system state and observational disclosures.

Let the computational system be defined over a measurable state space $\mathcal{S} \subseteq \mathbb{R}^n$, an action space $\mathcal{A} \subseteq \mathbb{R}^m$, and a telemetry or disclosure space $\mathcal{D} \subseteq \mathbb{R}^k$. The temporal evolution of the system may be described by a transition operator $\mathcal{F}$ such that:

$$S_{t+1} = \mathcal{F}(S_t, A_t, \Theta_t) \tag{A.1}$$

where Θ_t denotes latent parameters that may include model weights, environmental parameters, or control coefficients. In traditional formulations, telemetry is defined as a projection:

$$D_t = \mathcal{G}(S_t, A_t) \tag{A.2}$$

where $\mathcal{G}$ is an observation operator that does not feed back into the system. This assumption implies that the trajectory $\{S_t\}$ is independent of the representation or transmission of D_t.

This assumption fails in feedback-coupled systems. In such systems, telemetry is reintegrated into the computational loop, yielding a generalized transition operator:

$$S_{t+1} = \mathcal{F}(S_t, A_t, D_t, \Theta_t) \tag{A.3}$$

This formulation implies that the system evolves not only as a function of its internal state and actions but also as a function of its disclosed observational history. The telemetry

stream $\{D_t\}$ thus becomes an endogenous variable within the system dynamics.

To formalize this interaction, define the trajectory operator:

$$\Phi : \mathcal{S} \times \mathcal{A}^T \times \mathcal{D}^T \to \mathcal{S}^T \tag{A.4}$$

such that:

$$\Phi(S_0, \{A_t\}, \{D_t\}) = \{S_t\}_{t=0}^T \tag{A.5}$$

A necessary condition for reproducibility in such systems is that Φ be invariant under equivalent telemetry representations. That is, for any two telemetry sequences D and $\tilde{D}$ satisfying semantic equivalence $D \equiv \tilde{D}$, the following condition must hold:

$$\Phi(S_0, A, D) = \Phi(S_0, A, \tilde{D}) \tag{A.6}$$

However, in practical implementations, telemetry is not represented as an abstract element of $\mathcal{D}$ but as a binary encoding produced by an encoding function:

$$\mathcal{E} : \mathcal{D} \to \{0, 1\}^* \tag{A.7}$$

Thus, the actual system dynamics depend on $\mathcal{E}(D_t)$ rather than D_t itself, yielding:

$$S_{t+1} = \mathcal{F}(S_t, A_t, \mathcal{E}(D_t), \Theta_t) \tag{A.8}$$

This introduces a critical dependency on the properties of $\mathcal{E}$. In particular, for reproducibility, $\mathcal{E}$ must satisfy injective consistency:

$$D_1 \equiv D_2 \implies \mathcal{E}(D_1) = \mathcal{E}(D_2) \tag{A.9}$$

In conventional systems, this condition is violated. For example, consider floating-point representations. Let $x \in \mathbb{R}$ be a real-valued telemetry component. Due to IEEE-754 representation differences across architectures, there exist encodings $\mathcal{E}_1(x)$ and $\mathcal{E}_2(x)$ such that:

$$x = x \quad \text{but} \quad \mathcal{E}_1(x) \neq \mathcal{E}_2(x) \tag{A.10}$$

More generally, for structured data D composed of multiple fields, the absence of canonical ordering implies:

$$\exists \pi \in S_n \text{ such that } \mathcal{E}(D) \neq \mathcal{E}(\pi(D)) \tag{A.11}$$

where π is a permutation of key ordering. This non-uniqueness of representation introduces ambiguity in downstream operations, particularly cryptographic hashing.

Let $\mathcal{H} : \{0,1\}^* \to \{0,1\}^k$ be a cryptographic hash function. Then:

$$\mathcal{H}(\mathcal{E}(D_1)) \neq \mathcal{H}(\mathcal{E}(D_2)) \quad \text{even if} \quad D_1 \equiv D_2 \tag{A.12}$$

This violates the fundamental requirement for deterministic verification.

Furthermore, consider a distributed system consisting of nodes indexed by $i \in \mathcal{I}$, each with hardware configuration $\mathcal{H}_i$ and runtime environment $\mathcal{R}_i$. The encoding function becomes node-dependent:

$$\mathcal{E}_i = \mathcal{E}_{\mathcal{H}_i, \mathcal{R}_i} \tag{A.13}$$

Thus, for a fixed logical telemetry element D, the system produces a family of encodings:

$$\{\mathcal{E}_i(D)\}_{i \in \mathcal{I}} \tag{A.14}$$

which are not guaranteed to be identical. This lack of invariance across nodes leads to divergence in hash values, lineage construction, and ultimately system state evolution.

To quantify the impact of such discrepancies, define the encoding error:

$$\epsilon_i(D) = \|\mathcal{E}_i(D) - \mathcal{E}^*(D)\| \tag{A.15}$$

where $\mathcal{E}^*$ is the ideal canonical encoding. The propagation of this error into system dynamics can be expressed as:

$$S_{t+1} = \mathcal{F}(S_t, A_t, \mathcal{E}^*(D_t) + \epsilon_t, \Theta_t) \tag{A.16}$$

Under Lipschitz continuity assumptions:

$$\|S_{t+1} - S^*_{t+1}\| \leq L_d \|\epsilon_t\| \tag{A.17}$$

and recursively:

$$\|S_t - S_t^*\| \leq \sum_{k=0}^{t-1} L_d L_s^{t-1-k} \|\epsilon_k\| \tag{A.18}$$

where L_s and L_d are Lipschitz constants. This demonstrates that encoding inconsistencies, even if small, accumulate over time and can significantly alter system trajectories.

The problem is further exacerbated by the absence of lineage constraints. Let D_t denote telemetry at time t. In conventional systems, there is no enforced relation between D_t and D_{t-1}. Thus, the sequence $\{D_t\}$ may be arbitrarily modified without detection. Define a lineage function:

$$L_t = \mathcal{H}(D_t \parallel L_{t-1}) \tag{A.19}$$

In existing systems, no such function is enforced, implying that:

$$\exists D_t', \quad L_t' \neq L_t \quad \text{but system does not detect inconsistency} \tag{A.20}$$

This lack of continuity guarantees allows for replay attacks, omission of intermediate states, and insertion of fabricated data.

Additionally, consider the absence of provenance binding. Let Σ_t denote a signature associated with telemetry D_t. In conventional systems:

$$\Sigma_t = \text{Sign}(D_t, K) \tag{A.21}$$

This signature verifies the origin of D_t but does not guarantee that D_t is consistent with the internal state S_t. Thus, authenticity does not imply correctness.

The combination of non-deterministic encoding, absence of lineage, and lack of state binding leads to a fundamental breakdown of system integrity. The telemetry stream ceases to be a reliable representation of system behavior and instead becomes a source of uncertainty and potential manipulation.

Therefore, there exists a critical need for a framework that enforces deterministic representation, cryptographic continuity, and verifiable provenance of telemetry data. Such a framework must operate across heterogeneous environments, maintain scalability, and provide strong guarantees against both accidental inconsistencies and adversarial manipulation.

A.2 Causative Telemetry Distortion

The integration of telemetry into the state transition operator fundamentally alters the mathematical structure of the computational system by introducing a feedback-dependent perturbation channel. In classical systems, observational data is decoupled from the evolution operator, allowing the system to be analyzed independently of its representation. However, in feedback-coupled systems, telemetry becomes an endogenous variable that participates directly in the evolution of the system state, thereby introducing a new class of instability mechanisms that arise from inconsistencies in representation, transmission, or validation.

Let the system dynamics be defined over a state space $\mathcal{S}$ with transition operator $\mathcal{F} : \mathcal{S} \times \mathcal{A} \times \mathcal{D} \to \mathcal{S}$. The evolution of the system is given by:

$$S_{t+1} = \mathcal{F}(S_t, A_t, D_t) \tag{A.22}$$

Consider an ideal telemetry sequence $\{D_t\}$ and a perturbed sequence $\{\tilde{D}_t\}$ such that:

$$\tilde{D}_t = D_t + \delta_t \tag{A.23}$$

where δ_t represents a perturbation induced by encoding inconsistency, numerical drift, or adversarial manipulation. Let the corresponding state trajectories be $\{S_t\}$ and $\{\tilde{S}_t\}$, respectively, defined by:

$$S_{t+1} = \mathcal{F}(S_t, A_t, D_t), \quad \tilde{S}_{t+1} = \mathcal{F}(\tilde{S}_t, A_t, \tilde{D}_t) \tag{A.24}$$

Define the trajectory divergence:

$$\Delta_t = \|S_t - \tilde{S}_t\| \tag{A.25}$$

Assuming that $\mathcal{F}$ is Lipschitz continuous in both state and telemetry inputs, there exist constants $L_s, L_d > 0$ such that:

$$\|\mathcal{F}(S_1, A, D_1) - \mathcal{F}(S_2, A, D_2)\| \leq L_s\|S_1 - S_2\| + L_d\|D_1 - D_2\| \tag{A.26}$$

Applying this to the trajectories yields:

$$\Delta_{t+1} \leq L_s\Delta_t + L_d\|\delta_t\| \tag{A.27}$$

This recursive inequality defines a non-homogeneous linear difference equation whose solution can be expressed as:

$$\Delta_t \leq L_s^t \Delta_0 + \sum_{k=0}^{t-1} L_s^{t-1-k} L_d \|\delta_k\| \tag{A.28}$$

This expression reveals two critical regimes. In the case where $L_s < 1$, the system exhibits contractive behavior and perturbations decay over time. However, in many practical systems, particularly those involving iterative optimization or reinforcement learning, the effective Lipschitz constant may satisfy $L_s \geq 1$, leading to amplification of perturbations. In such regimes, even small perturbations δ_t can result in exponential divergence of trajectories.

To further analyze stability, consider a Lyapunov candidate function $V : \mathcal{S} \to \mathbb{R}_{\geq 0}$ defined as:

$$V(S_t, \tilde{S}_t) = \|S_t - \tilde{S}_t\|^2 \tag{A.29}$$

Then:

$$V(S_{t+1}, \tilde{S}_{t+1}) \leq (L_s \Delta_t + L_d \|\delta_t\|)^2 \tag{A.30}$$

Expanding:

$$V_{t+1} \leq L_s^2 V_t + 2 L_s L_d \Delta_t \|\delta_t\| + L_d^2 \|\delta_t\|^2 \tag{A.31}$$

This inequality demonstrates that the stability of the system is directly influenced by the magnitude and persistence of telemetry perturbations. In the absence of strict control over δ_t, the Lyapunov function may increase unbounded, indicating instability.

In adversarial contexts, an attacker may intentionally construct perturbations δ_t that are small in magnitude but strategically aligned with the system dynamics to maximize divergence. Consider an adversary that selects δ_t such that:

$$\delta_t = \epsilon \cdot \frac{\nabla_{D_t} \mathcal{F}(S_t, A_t, D_t)}{\|\nabla_{D_t} \mathcal{F}(S_t, A_t, D_t)\|} \tag{A.32}$$

where ϵ is a small scalar. This perturbation aligns with the direction of maximum sensitivity of the transition operator, thereby maximizing its impact on S_{t+1}. Over multiple iterations, such perturbations can induce significant divergence while remaining within acceptable noise thresholds.

Furthermore, the problem is compounded by temporal accumulation. Define the cumulative perturbation energy:

$$E_T = \sum_{t=0}^{T} \|\delta_t\|^2 \tag{A.33}$$

Then the trajectory divergence satisfies:

$$\Delta_T \leq L_s^T \Delta_0 + L_d \sum_{k=0}^{T-1} L_s^{T-1-k} \|\delta_k\| \tag{A.34}$$

Using Cauchy-Schwarz inequality:

$$\Delta_T \leq L_s^T \Delta_0 + L_d \left(\sum_{k=0}^{T-1} L_s^{2(T-1-k)} \right)^{1/2} \cdot \sqrt{E_T} \tag{A.35}$$

This establishes that the divergence is bounded by the cumulative energy of perturbations weighted by system sensitivity. In systems with large L_s, this bound becomes loose, indicating high susceptibility to perturbation amplification.

Another critical aspect of Causative Telemetry Distortion is its interaction with stochasticity. In systems where δ_t is modeled as a stochastic process with variance σ^2, the expected divergence satisfies:

$$\mathbb{E}[\Delta_t^2] \leq L_s^2 \mathbb{E}[\Delta_{t-1}^2] + L_d^2 \sigma^2 \tag{A.36}$$

Solving this recursion yields:

$$\mathbb{E}[\Delta_t^2] \leq L_s^{2t} \Delta_0^2 + L_d^2 \sigma^2 \sum_{k=0}^{t-1} L_s^{2k} \tag{A.37}$$

If $L_s > 1$, the expected divergence grows exponentially, indicating that even random noise in telemetry can destabilize the system.

The implications of these results are profound. They demonstrate that telemetry integrity is not merely a matter of data correctness but a fundamental requirement for system stability. Any inconsistency in telemetry, regardless of its origin, acts as a perturbation that propagates through the system dynamics and may be amplified over time.

To mitigate this phenomenon, it is necessary to enforce constraints on the telemetry channel such that $\delta_t = 0$ for all t in the ideal case. This requires that telemetry be represented deterministically, transmitted without alteration, and verified upon ingestion. Additionally, the system must enforce continuity constraints to ensure that the sequence

$\{D_t\}$ is complete and unaltered.

Define a deterministic telemetry operator $\mathcal{E}$ and a verification function $\mathcal{V}$ such that:

$$\mathcal{V}(\mathcal{E}(D_t)) = \text{true} \iff \delta_t = 0 \tag{A.38}$$

Then the system achieves distortion-free evolution:

$$S_{t+1} = \mathcal{F}(S_t, A_t, \mathcal{E}(D_t)) \tag{A.39}$$

with guaranteed consistency across executions.

In summary, Causative Telemetry Distortion represents a fundamental instability mechanism in feedback-coupled systems arising from inconsistencies in telemetry representation and transmission. The mathematical analysis demonstrates that even small perturbations can be amplified over time, leading to significant divergence in system behavior. This establishes the necessity for a rigorous framework that enforces deterministic telemetry representation, cryptographic integrity, and continuity constraints to eliminate distortion and ensure reproducibility.

A.3 Limitations of Existing Systems

The deficiencies of existing telemetry and experiment tracking systems may be rigorously characterized as violations of fundamental mathematical invariants required for deterministic, verifiable, and stable feedback-coupled computation. While such systems are often designed with scalability and usability in mind, their underlying abstractions fail to enforce properties necessary for preserving the integrity of telemetry as a causal component of system evolution.

To formalize these limitations, consider the telemetry pipeline as a composite operator:

$$\mathcal{T} = \mathcal{P} \circ \mathcal{H} \circ \mathcal{E} \tag{A.40}$$

where $\mathcal{E}$ is the encoding function, $\mathcal{H}$ is a cryptographic hashing function, and $\mathcal{P}$ is a persistence or storage operator. For the system to be deterministic and verifiable, $\mathcal{T}$ must satisfy invariance under semantic equivalence:

$$D_1 \equiv D_2 \implies \mathcal{T}(D_1) = \mathcal{T}(D_2) \tag{A.41}$$

In existing systems, this condition is violated at multiple levels.

A.3.1 Non-Deterministic Serialization as a Violation of Encoding Invariance

Let $\mathcal{E}$ denote the encoding function used by a telemetry system. In an ideal system, $\mathcal{E}$ must be canonical, meaning that it defines a unique mapping from logical data structures to binary representations. Formally, for any $D_1, D_2 \in \mathcal{D}$:

$$D_1 \equiv D_2 \implies \mathcal{E}(D_1) = \mathcal{E}(D_2) \tag{A.42}$$

However, in conventional systems utilizing formats such as JSON or protocol buffers, the encoding function is not injectively consistent. Consider a dictionary-like structure:

$$D = \{(k_1, v_1), (k_2, v_2)\} \tag{A.43}$$

Then multiple encodings exist:

$$\mathcal{E}_1(D) = \texttt{\{k}_1\texttt{:v}_1\texttt{, k}_2\texttt{:v}_2\texttt{\}}, \quad \mathcal{E}_2(D) = \texttt{\{k}_2\texttt{:v}_2\texttt{, k}_1\texttt{:v}_1\texttt{\}} \tag{A.44}$$

Despite D being identical, $\mathcal{E}_1(D) \neq \mathcal{E}_2(D)$, violating determinism. This non-uniqueness propagates into hashing:

$$\mathcal{H}(\mathcal{E}_1(D)) \neq \mathcal{H}(\mathcal{E}_2(D)) \tag{A.45}$$

Thus, identical logical telemetry produces divergent cryptographic identities.

Furthermore, floating-point representations introduce additional non-determinism. Let $x \in \mathbb{R}$ be represented in finite precision. Due to rounding and architecture-dependent implementations:

$$\mathcal{E}_{\mathcal{H}_1}(x) \neq \mathcal{E}_{\mathcal{H}_2}(x) \tag{A.46}$$

even though x is semantically identical. This leads to a violation of cross-platform consistency:

$$\exists D, \quad \mathcal{E}_{\mathcal{H}_1}(D) \neq \mathcal{E}_{\mathcal{H}_2}(D) \tag{A.47}$$

which directly undermines reproducibility.

A.3.2 Absence of Cryptographic Lineage as a Failure of Temporal Integrity

In a system with temporal evolution, telemetry must form a sequence $\{D_t\}$ that preserves ordering and continuity. Define a lineage function:

$$L_t = \mathcal{H}(\mathcal{E}(D_t) \parallel L_{t-1}) \tag{A.48}$$

This recursive construction ensures that any modification to D_k for $k < t$ propagates forward and alters L_t.

In existing systems, no such constraint is enforced. Telemetry entries are stored independently, meaning:

$$\exists \{D_t\}, \{D'_t\} \text{ such that } D'_k \neq D_k \text{ for some } k < t \quad \text{but system does not detect} \tag{A.49}$$

This implies that the system lacks temporal integrity. The absence of lineage enforcement allows for arbitrary modification, deletion, or insertion of telemetry entries without invalidating subsequent records.

Formally, the system fails to enforce the invariant:

$$\forall t, \quad L_t = f(L_{t-1}, D_t) \tag{A.50}$$

for some deterministic function f. Without this invariant, the telemetry sequence does not define a valid chain and cannot be audited.

A.3.3 Single-Actor Trust Models as a Failure of Byzantine Resilience

Existing systems frequently rely on a single signing authority to validate telemetry. Let K denote a private key and σ a signature function:

$$\sigma_t = \text{Sign}(D_t, K) \tag{A.51}$$

Verification is performed as:

$$\text{Verify}(D_t, \sigma_t, PK) = \text{true} \tag{A.52}$$

While this ensures authenticity under the assumption that K is secure, it introduces a critical vulnerability. If K is compromised, an adversary can generate valid signatures for arbitrary data:

$$\exists D'_t, \quad \text{Verify}(D'_t, \sigma'_t, PK) = \text{true} \tag{A.53}$$

where $\sigma'_t = \text{Sign}(D'_t, K)$. This implies that the system cannot distinguish between legitimate and malicious telemetry once the key is compromised.

From a distributed systems perspective, this represents a failure to tolerate Byzantine faults. Let the system consist of n nodes, and let one node be compromised. In a single-actor model, the adversary gains full control over the validation process. The system lacks redundancy and cannot enforce consensus.

A.3.4 Absence of State Binding as a Failure of Semantic Consistency

Even in systems where signatures are employed, the verification process typically ensures only that the data originated from a particular entity, not that it is consistent with the internal state of that entity. Let S_t denote the internal state and D_t the telemetry. In existing systems:

$$\Sigma_t = \text{Sign}(D_t, K) \tag{A.54}$$

but there is no constraint that:

$$D_t = \mathcal{G}(S_t) \tag{A.55}$$

for a valid observation function $\mathcal{G}$. Thus, an adversary controlling the node can produce arbitrary D'_t and sign it, resulting in:

$$\text{Verify}(D'_t, \Sigma'_t, PK) = \text{true} \tag{A.56}$$

while D'_t is not consistent with S_t. This represents a failure of semantic integrity, wherein structurally valid data does not correspond to the actual system behavior.

A.3.5 Inadequate Replay and Omission Protection as a Failure of Sequence Constraints

In a temporally ordered system, telemetry must satisfy strict sequencing constraints. Let t denote the index of a telemetry entry. A valid sequence must satisfy:

$$t = t_{\text{prev}} + 1 \tag{A.57}$$

In existing systems, this constraint is not enforced. As a result, adversaries may perform replay attacks by reusing previous telemetry entries:

$$D'_t = D_k \quad \text{for some } k < t \tag{A.58}$$

or omission attacks by removing entries from the sequence. Without lineage enforcement, these manipulations are not detectable.

Formally, the system fails to enforce injectivity of the sequence mapping:

$$t \mapsto D_t \tag{A.59}$$

and does not guarantee surjectivity over the intended time domain, allowing gaps in the sequence.

A.3.6 Composite Failure of Integrity Invariants

The combined effect of these deficiencies can be expressed as the failure of a composite integrity condition. Define a telemetry system as valid if:

$$\text{Valid}(D_t) \iff \begin{cases} \mathcal{E}(D_t) \text{ is deterministic} \\ L_t = \mathcal{H}(\mathcal{E}(D_t) \parallel L_{t-1}) \\ \text{Verify}(L_t, \Sigma_t) = \text{true} \\ D_t = \mathcal{G}(S_t) \end{cases} \tag{A.60}$$

Existing systems violate one or more of these conditions, implying:

$$\exists t, \quad \text{Valid}(D_t) = \text{false} \tag{A.61}$$

even when the system reports successful ingestion or verification.

A.3.7 Implications for System Behavior

The violation of these invariants leads to a breakdown in system reliability. From the analysis in the previous section, any inconsistency in telemetry acts as a perturbation δ_t that propagates through the system dynamics. The absence of deterministic encoding, lineage enforcement, and state binding implies that δ_t may be non-zero even in the absence of adversarial action.

Consequently, the system exhibits non-reproducibility, as identical logical executions produce divergent trajectories. In adversarial scenarios, these vulnerabilities may be exploited to manipulate system behavior, inject false data, or conceal undesirable outcomes.

Therefore, existing telemetry systems are fundamentally inadequate for feedback-coupled computational environments, as they fail to enforce the mathematical invariants necessary for deterministic and verifiable system evolution.

A.4 Engineering Requirements

The deficiencies identified in preceding sections establish that any telemetry system operating within a feedback-coupled computational environment must satisfy a set of strict mathematical invariants in order to ensure deterministic, stable, and verifiable system evolution. These invariants are not optional enhancements but necessary conditions derived from the structural properties of the system dynamics. The purpose of this section is to formalize these requirements as constraints on the telemetry processing pipeline, thereby defining the conditions under which Causative Telemetry Distortion is eliminated and reproducibility is guaranteed.

Let the telemetry processing pipeline be defined as a composition of operators:

$$\mathcal{P} = \mathcal{V} \circ \mathcal{S} \circ \mathcal{C} \circ \mathcal{E} \tag{A.62}$$

where $\mathcal{E}$ is the encoding operator, $\mathcal{C}$ is the cryptographic chaining operator, $\mathcal{S}$ is a proof generation operator, and $\mathcal{V}$ is a verification operator. The system is said to be integrity-preserving if and only if, for all telemetry inputs D_t, the pipeline satisfies:

$$\mathcal{V}(\mathcal{S}(\mathcal{C}(\mathcal{E}(D_t)))) = \text{true} \tag{A.63}$$

and the resulting state evolution is invariant under semantically equivalent inputs.

A.4.1 Deterministic Representation Constraint

The first and most fundamental requirement is that the encoding operator $\mathcal{E}$ must be canonical. Formally, $\mathcal{E}$ must define a deterministic mapping:

$$\mathcal{E} : \mathcal{D} \to \{0, 1\}^* \tag{A.64}$$

such that:

$$\forall D_1, D_2 \in \mathcal{D}, \quad D_1 \equiv D_2 \implies \mathcal{E}(D_1) = \mathcal{E}(D_2) \tag{A.65}$$

This property ensures that logical equivalence implies binary equivalence. In order to enforce this constraint, the encoding must eliminate all sources of non-determinism, including key ordering ambiguity and floating-point representation variability.

Let $x \in \mathbb{R}$ be a numerical component of telemetry. Define a fixed-precision projection operator:

$$\Pi_k(x) = \lfloor x \cdot 10^k \rfloor \tag{A.66}$$

for some precision parameter $k \in \mathbb{N}$. The encoding operator is then defined over the transformed space:

$$\mathcal{E}(D) = \text{Encode}(\Pi_k(D)) \tag{A.67}$$

where $\Pi_k(D)$ applies the projection component-wise. This ensures that:

$$\forall i, j, \quad \Pi_k^{(i)}(x) = \Pi_k^{(j)}(x) \tag{A.68}$$

for all architectures i, j, thereby eliminating cross-platform discrepancies.

A.4.2 Immutable Lineage Constraint

The second requirement is that telemetry must be embedded within a cryptographically verifiable lineage structure. Define the lineage function:

$$L_t = \mathcal{H}(\mathcal{E}(D_t) \parallel L_{t-1}) \tag{A.69}$$

with initial condition L_0 defined by a secure genesis value. This recursive construction

ensures that the lineage forms a hash chain over the sequence $\{D_t\}$.

The requirement of immutability can be expressed as the following invariant:

$$\forall k < t, \quad D'_k \neq D_k \implies L'_t \neq L_t \tag{A.70}$$

This property follows from the collision resistance of $\mathcal{H}$ and guarantees that any modification to historical telemetry is detectable at the current state.

Additionally, the lineage must enforce continuity:

$$t = t_{\text{prev}} + 1 \tag{A.71}$$

ensuring that the sequence is strictly ordered and complete.

A.4.3 State Provenance Binding Constraint

The third requirement is that telemetry must be cryptographically bound to the internal computational state from which it was generated. Let $\mathcal{G}$ denote the observation function such that:

$$D_t = \mathcal{G}(S_t) \tag{A.72}$$

To enforce this relationship, the system must construct a proof Σ_t that binds L_t to a representation of S_t. Let K denote a private key associated with the computational entity. Then:

$$\Sigma_t = \text{Sign}(L_t, K) \tag{A.73}$$

The verification condition is:

$$\text{Verify}(L_t, \Sigma_t, PK) = \text{true} \tag{A.74}$$

This ensures that the telemetry sequence is not only structurally valid but also originates from an authenticated source. To strengthen this constraint, the proof may incorporate a commitment to the internal state:

$$\Sigma_t = \text{Sign}(\mathcal{H}(S_t) \parallel L_t, K) \tag{A.75}$$

thereby binding telemetry to the actual system state.

A.4.4 Multi-Party Verification Constraint

To eliminate single points of failure, the system must employ a distributed verification mechanism. Let there be n validators, each with a private key share K_i. Define partial signatures:

$$\sigma_i = \text{Sign}(L_t, K_i) \tag{A.76}$$

An aggregate signature is constructed as:

$$\Sigma_t = \text{Aggregate}(\sigma_{i_1}, \ldots, \sigma_{i_m}) \tag{A.77}$$

for some subset of validators $\{i_1, \ldots, i_m\}$ with $m \geq t$, where t is the threshold. The verification condition becomes:

$$\text{Verify}(L_t, \Sigma_t, PK) = \text{true} \tag{A.78}$$

This ensures that no single compromised node can unilaterally validate telemetry.

From a Byzantine fault tolerance perspective, the system tolerates up to $f < t$ faulty nodes, as an adversary must compromise at least t validators to produce a valid signature.

A.4.5 Replay and Omission Resistance Constraint

The system must enforce strict sequence integrity. Let the telemetry sequence be indexed by $t \in \mathbb{N}$. Define a sequence mapping:

$$\psi : t \mapsto D_t \tag{A.79}$$

The system must enforce injectivity and continuity:

$$t_1 \neq t_2 \implies D_{t_1} \neq D_{t_2} \tag{A.80}$$

and:

$$\forall t > 0, \quad \exists D_{t-1} \tag{A.81}$$

ensuring that no entries are replayed or omitted. This is enforced through the lineage function, as any discontinuity results in a mismatch in L_t.

A.4.6 Cross-Platform Consistency Constraint

Given a set of heterogeneous environments $\{(\mathcal{H}_i, \mathcal{R}_i)\}$, the encoding function must satisfy:

$$\forall i, j, \quad \mathcal{E}_{\mathcal{H}_i,\mathcal{R}_i}(D) = \mathcal{E}_{\mathcal{H}_j,\mathcal{R}_j}(D) \tag{A.82}$$

This ensures that the system produces identical outputs regardless of execution environment.

A.4.7 Scalability Constraint

Finally, the system must maintain these invariants under high-throughput conditions. Let λ denote the rate of telemetry generation. The system must satisfy:

$$\lim_{\lambda \to \infty} \text{Latency}(\mathcal{P}) < \infty \tag{A.83}$$

while preserving correctness:

$$\forall D_t, \quad \text{Valid}(D_t) = \text{true} \tag{A.84}$$

A.4.8 Unified Integrity Condition

The above requirements may be consolidated into a single validity condition. A telemetry entry D_t is valid if and only if:

$$\text{Valid}(D_t) \iff \begin{cases} \mathcal{E}(D_t) \text{ is canonical} \\ L_t = \mathcal{H}(\mathcal{E}(D_t) \parallel L_{t-1}) \\ \text{Verify}(L_t, \Sigma_t) = \text{true} \\ D_t = \mathcal{G}(S_t) \end{cases} \tag{A.85}$$

This condition defines the necessary and sufficient criteria for deterministic and verifiable telemetry in feedback-coupled systems.

A.4.9 Implications for System Design

The enforcement of these constraints transforms the telemetry pipeline from a passive logging mechanism into a formally verified computational subsystem. By ensuring deterministic representation, immutable lineage, and distributed verification, the system eliminates the perturbation channel δ_t identified in previous sections, thereby stabilizing

the system dynamics and ensuring reproducibility.

These requirements form the foundation for the architecture described in subsequent sections and define the core invariants that the disclosed RCO protocol is designed to satisfy.

A.5 Overview of the Disclosed Solution

The limitations of existing telemetry systems, when formalized as violations of deterministic encoding, lineage continuity, state binding, and distributed verification, necessitate the introduction of a unified framework that enforces these invariants as intrinsic properties of the system. The Feedback-Coupled Cryptographic Observation (RCO) Protocol constitutes such a framework, providing a mathematically grounded mechanism for ensuring that telemetry data functions as a deterministic, immutable, and verifiable component of system evolution.

The RCO protocol may be interpreted as a transformation of the telemetry pipeline into a formally constrained computational operator. Let the pipeline be defined as a composition:

$$\mathcal{P}_{\text{RCO}} = \mathcal{V} \circ \mathcal{S} \circ \mathcal{C} \circ \mathcal{E} \tag{A.86}$$

where $\mathcal{E}$ denotes canonical encoding, $\mathcal{C}$ denotes cryptographic chaining, $\mathcal{S}$ denotes proof generation, and $\mathcal{V}$ denotes verification. The defining property of the RCO protocol is that $\mathcal{P}_{\text{RCO}}$ enforces the validity condition:

$$\forall D_t \in \mathcal{D}, \quad \mathcal{P}_{\text{RCO}}(D_t) = \text{true} \iff \text{Valid}(D_t) \tag{A.87}$$

where $\text{Valid}(D_t)$ is defined by the composite integrity condition introduced previously.

The encoding operator $\mathcal{E}$ is constructed as a canonical mapping that eliminates all non-determinism in representation. Let Π_k denote a fixed-precision projection operator. Then the encoding is defined as:

$$\mathcal{E}(D_t) = \text{Encode}(\Pi_k(D_t)) \tag{A.88}$$

where the encoding function enforces lexicographic ordering and fixed structural representation. This ensures that for any semantically equivalent telemetry elements D_1 and D_2:

$$D_1 \equiv D_2 \implies \mathcal{E}(D_1) = \mathcal{E}(D_2) \tag{A.89}$$

thereby eliminating the perturbation term δ_t arising from representation inconsistencies.

The chaining operator $\mathcal{C}$ constructs a lineage over telemetry entries by recursively applying a cryptographic hash function $\mathcal{H}$. Define:

$$L_t = \mathcal{H}(\mathcal{E}(D_t) \parallel L_{t-1}) \tag{A.90}$$

This construction induces a directed acyclic structure over the sequence $\{D_t\}$, wherein each node depends on all prior nodes. The resulting lineage sequence $\{L_t\}$ satisfies the property:

$$\forall k < t, \quad D'_k \neq D_k \implies L'_t \neq L_t \tag{A.91}$$

with overwhelming probability under the collision resistance assumption of $\mathcal{H}$. This ensures immutability and provides a mechanism for detecting any historical modification.

The proof generation operator $\mathcal{S}$ constructs a cryptographic binding between the lineage and the originating computational state. Let K denote a private key associated with the computational entity, and let $\mathcal{H}_S$ denote a state hashing function. The proof is defined as:

$$\Sigma_t = \text{Sign}(\mathcal{H}_S(S_t) \parallel L_t, K) \tag{A.92}$$

This formulation ensures that the proof incorporates both the lineage and a representation of the internal state, thereby enforcing state provenance. The verification operator $\mathcal{V}$ evaluates:

$$\mathcal{V}(L_t, \Sigma_t) = \text{Verify}(\mathcal{H}_S(S_t) \parallel L_t, \Sigma_t, PK) \tag{A.93}$$

ensuring that the telemetry is both authentic and consistent with the originating state.

To eliminate single points of failure, the RCO protocol generalizes the proof generation mechanism to a distributed setting. Let there be a set of validators indexed by $i \in \{1, \ldots, n\}$, each holding a key share K_i. Each validator produces a partial signature:

$$\sigma_i = \text{Sign}(L_t, K_i) \tag{A.94}$$

An aggregate proof is constructed as:

$$\Sigma_t = \text{Aggregate}(\sigma_{i_1}, \ldots, \sigma_{i_m}) \tag{A.95}$$

for a subset of validators satisfying $m \geq t$, where t is a predefined threshold. The verification condition becomes:

$$\text{Verify}(L_t, \Sigma_t, PK) = \text{true} \tag{A.96}$$

This construction ensures that the validation process is resilient to Byzantine faults, as an adversary must compromise at least t independent validators to forge a valid proof.

The RCO protocol further enforces sequence integrity through the implicit constraints of the lineage function. Given the recursive definition of L_t, any omission or reordering of telemetry entries results in a mismatch in the lineage sequence. Thus, the protocol enforces:

$$t = t_{\text{prev}} + 1 \tag{A.97}$$

as a necessary condition for acceptance, ensuring that the telemetry sequence is complete and strictly ordered.

From a dynamical systems perspective, the RCO protocol eliminates the perturbation channel δ_t by enforcing:

$$\delta_t = 0 \quad \forall t \tag{A.98}$$

under ideal conditions. Substituting into the divergence relation derived earlier:

$$\Delta_{t+1} \leq L_s \Delta_t \tag{A.99}$$

which implies that if $L_s < 1$, the system converges, and if $L_s = 1$, the system remains stable. In either case, the absence of external perturbations ensures that the system evolution is determined solely by its initial conditions and actions.

The protocol may also be interpreted as enforcing a constraint on the trajectory operator Φ such that:

$$\Phi(S_0, A, D) = \Phi(S_0, A, \mathcal{E}^{-1}(\mathcal{E}(D))) \tag{A.100}$$

ensuring invariance under encoding. This property guarantees that the system behaves identically across executions, provided that the same logical telemetry is supplied.

In summary, the RCO protocol transforms the telemetry pipeline into a formally verified subsystem that enforces deterministic representation, cryptographic continuity, and distributed validation. By integrating these components into a unified framework, the protocol eliminates the sources of distortion identified in previous sections and establishes a foundation for reproducible, tamper-evident, and verifiable computation in feedback-coupled systems.

A.6 System Architecture Overview

The Feedback-Coupled Cryptographic Observation (RCO) protocol may be rigorously characterized as a compositional system of operators acting over telemetry data and system state, wherein each operator enforces a distinct invariant required for deterministic and verifiable computation. Rather than treating the architecture as a collection of loosely coupled modules, the RCO protocol defines a mathematically structured pipeline in which each transformation is explicitly constrained and contributes to a globally consistent integrity condition.

Let the telemetry processing architecture be defined as a sequence of transformations over an input telemetry element D_t and system state S_t. The architecture may be expressed as a functional composition:

$$\mathcal{A}_{\text{RCO}} = \mathcal{V} \circ \mathcal{S} \circ \mathcal{C} \circ \mathcal{E} \tag{A.101}$$

where each operator acts on the output of the preceding transformation. The domain and codomain of each operator are defined such that the composition is well-formed and deterministic.

The encoding operator $\mathcal{E} : \mathcal{D} \to \{0, 1\}^*$ maps telemetry data from its abstract representation into a canonical binary form. This operator is defined as a composition of a projection operator Π_k and a structural encoding function:

$$\mathcal{E} = \text{Encode} \circ \Pi_k \tag{A.102}$$

The projection operator Π_k ensures numerical determinism by mapping real-valued inputs into a discrete lattice, while the encoding function enforces a unique ordering and representation of structured data. The output of $\mathcal{E}$ is a byte sequence that serves as the canonical representation of the telemetry element.

The chaining operator $\mathcal{C}$ operates over the encoded telemetry and the previous lineage state. Let $L_{t-1} \in \{0,1\}^k$ denote the lineage at time $t-1$. Then:

$$\mathcal{C}(\mathcal{E}(D_t), L_{t-1}) = L_t = \mathcal{H}(\mathcal{E}(D_t) \parallel L_{t-1}) \tag{A.103}$$

This operator defines a recursive dependency structure over the sequence $\{D_t\}$, effectively embedding the entire history of telemetry into the current lineage state. The chaining operator may be interpreted as constructing a path in a hash-induced state space, where each transition is uniquely determined by the current input and previous state.

The proof generation operator $\mathcal{S}$ maps the lineage state into a cryptographic proof that binds the data to an originating computational entity. Let $\mathcal{K}$ denote the key space and $\mathcal{PK}$ the corresponding public key space. Then:

$$\mathcal{S} : \{0,1\}^k \times \mathcal{K} \to \Sigma \tag{A.104}$$

where Σ denotes the space of cryptographic proofs. The operator is defined as:

$$\Sigma_t = \mathcal{S}(L_t, K) \tag{A.105}$$

In an extended formulation incorporating state binding, the operator may take the form:

$$\Sigma_t = \mathcal{S}(\mathcal{H}_S(S_t) \parallel L_t, K) \tag{A.106}$$

thereby coupling the lineage with a representation of the system state.

The verification operator $\mathcal{V}$ evaluates the validity of the proof with respect to the lineage and public key. Formally:

$$\mathcal{V} : \{0,1\}^k \times \Sigma \times \mathcal{PK} \to \{\text{true}, \text{false}\} \tag{A.107}$$

such that:

$$\mathcal{V}(L_t, \Sigma_t, PK) = \text{Verify}(L_t, \Sigma_t, PK) \tag{A.108}$$

The composition of these operators yields a deterministic mapping from telemetry input to a binary acceptance decision.

To analyze the architecture at a higher level, consider the system as a state machine operating over an augmented state space $\mathcal{X} = \mathcal{S} \times \mathcal{L}$, where $\mathcal{L}$ denotes the lineage space.

The system evolves according to:

$$(S_{t+1}, L_t) = \Psi(S_t, L_{t-1}, A_t, D_t) \tag{A.109}$$

where Ψ incorporates both the computational transition $\mathcal{F}$ and the lineage update $\mathcal{C}$. This formulation reveals that the RCO protocol effectively augments the system state with a cryptographic memory that encodes the entire history of telemetry.

The architecture may also be interpreted through the lens of category theory, wherein each operator defines a morphism between objects in a structured category. The composition $\mathcal{A}_{\text{RCO}}$ then defines a functor from the category of telemetry data to the category of verified states. The commutativity of the diagram:

$$\mathcal{V} \circ \mathcal{S} \circ \mathcal{C} \circ \mathcal{E} = \text{Valid} \tag{A.110}$$

ensures that the outcome is invariant under equivalent transformations of the input.

From an information-theoretic perspective, the lineage function L_t may be viewed as a compression of the telemetry history into a fixed-length representation that preserves integrity information. Let $H(\cdot)$ denote entropy. Then:

$$H(L_t) \leq k \tag{A.111}$$

while the effective information content of the history grows with t. The cryptographic properties of $\mathcal{H}$ ensure that this compression is collision-resistant, thereby preserving distinguishability between different histories.

The architecture further supports parallelism and distribution. Let the system be partitioned into multiple nodes, each processing a subset of telemetry. The encoding and chaining operators are inherently local, while the proof generation and verification operators may be distributed across validator nodes. Define a set of validators $\mathcal{V}_i$ with local verification functions $\mathcal{V}_i$. The global verification condition is:

$$\bigwedge_{i=1}^{m} \mathcal{V}_i(L_t, \sigma_i, PK_i) = \text{true} \tag{A.112}$$

for a sufficient subset of validators. This distributed structure enables scalability while preserving the integrity invariants.

From a computational complexity standpoint, the cost of the pipeline is dominated by the hashing and signature operations. Let $T_{\mathcal{H}}$, T_{Sign}, and T_{Verify} denote the time complexities of hashing, signing, and verification, respectively. Then the total cost per telemetry entry

is:

$$T_{\text{total}} = T_{\mathcal{E}} + T_{\mathcal{H}} + m \cdot T_{\text{Sign}} + T_{\text{Verify}} \tag{A.113}$$

where m is the number of validators participating in the threshold scheme. The architecture is designed such that these operations can be parallelized, ensuring that throughput scales with available computational resources.

Finally, the architecture enforces a global invariant over the system state:

$$\forall t, \quad \mathcal{V}(L_t, \Sigma_t) = \text{true} \implies (S_t, L_t) \text{ is valid} \tag{A.114}$$

This invariant ensures that the system state and telemetry lineage remain synchronized and verifiable at all times.

In summary, the RCO architecture defines a mathematically structured pipeline in which each operator enforces a specific integrity constraint, and the composition of these operators yields a system that is deterministic, immutable, and verifiable. By expressing the architecture in terms of functional composition and state augmentation, the protocol provides a rigorous foundation for the implementation of high-assurance telemetry systems in feedback-coupled computational environments.

A.7 Minimal Viable Integrity Model

While the full formulation of the RCO protocol enforces a comprehensive set of invariants including deterministic encoding, cryptographic lineage, state-bound proofs, and multi-party verification, it is neither necessary nor always practical for all deployment environments to implement the entirety of these mechanisms simultaneously. In order to facilitate incremental adoption and to accommodate systems with constrained computational or infrastructural resources, the RCO framework admits a reduced configuration, herein referred to as the Minimal Viable Integrity Model, which preserves the core guarantees required to eliminate undetected telemetry distortion while relaxing certain auxiliary constraints.

The Minimal Viable Integrity Model may be derived by identifying the minimal subset of invariants necessary to ensure that the perturbation term δ_t introduced in previous sections is eliminated at the level of representation and temporal continuity. Formally, the goal is to enforce conditions under which:

$$\delta_t = 0 \quad \forall t \tag{A.115}$$

with respect to the encoded telemetry stream, while tolerating reduced guarantees in provenance and distributed trust.

Let the reduced telemetry pipeline be defined as:

$$\mathcal{P}_{\text{MVI}} = \mathcal{C} \circ \mathcal{E} \tag{A.116}$$

where $\mathcal{E}$ is the canonical encoding operator and $\mathcal{C}$ is the cryptographic chaining operator. In this configuration, the proof generation and multi-party verification operators are omitted, resulting in a simplified pipeline that nevertheless enforces deterministic representation and immutable lineage.

The encoding operator $\mathcal{E}$ retains its canonical properties as defined previously. That is:

$$\forall D_1, D_2 \in \mathcal{D}, \quad D_1 \equiv D_2 \implies \mathcal{E}(D_1) = \mathcal{E}(D_2) \tag{A.117}$$

ensuring that the encoded telemetry is invariant under semantic equivalence. This property alone eliminates a significant class of perturbations arising from non-deterministic serialization and cross-platform discrepancies.

The chaining operator $\mathcal{C}$ constructs the lineage sequence:

$$L_t = \mathcal{H}(\mathcal{E}(D_t) \parallel L_{t-1}) \tag{A.118}$$

which enforces temporal integrity by embedding each telemetry element within a cryptographic dependency chain. The lineage sequence $\{L_t\}$ satisfies the property:

$$\forall k < t, \quad D'_k \neq D_k \implies L'_t \neq L_t \tag{A.119}$$

with overwhelming probability. This ensures that any modification to historical telemetry is detectable at the current state, even in the absence of explicit proof verification.

The Minimal Viable Integrity Model may be characterized by the following validity condition:

$$\text{Valid}_{\text{MVI}}(D_t) \iff L_t = \mathcal{H}(\mathcal{E}(D_t) \parallel L_{t-1}) \tag{A.120}$$

This condition ensures that the telemetry sequence is both deterministically encoded and cryptographically chained, thereby preserving structural integrity and continuity.

To analyze the effectiveness of this reduced model, consider the divergence relation derived earlier:

$$\Delta_{t+1} \leq L_s \Delta_t + L_d \|\delta_t\| \tag{A.121}$$

Under the Minimal Viable Integrity Model, the encoding constraint ensures that $\delta_t = 0$ with respect to representation-induced perturbations. Therefore:

$$\Delta_{t+1} \leq L_s \Delta_t \tag{A.122}$$

which implies that the system evolves deterministically with respect to telemetry inputs. Any divergence that remains is attributable solely to differences in initial conditions or actions, rather than inconsistencies in telemetry encoding.

However, the absence of proof generation and verification introduces a residual vulnerability in the form of unverified provenance. In particular, while the lineage structure ensures that modifications are detectable, it does not guarantee that the data was generated by an authorized or legitimate computational entity. Let Σ_t denote a hypothetical proof. In the Minimal Viable Integrity Model, no such proof is required, and therefore the condition:

$$\text{Verify}(L_t, \Sigma_t, PK) = \text{true} \tag{A.123}$$

is not enforced. As a result, the system is susceptible to adversarial injection of telemetry that is structurally valid but semantically incorrect.

To quantify this limitation, consider an adversary that generates a sequence $\{D'_t\}$ satisfying:

$$L'_t = \mathcal{H}(\mathcal{E}(D'_t) \parallel L_{t-1}) \tag{A.124}$$

Such a sequence will be accepted by the Minimal Viable Integrity Model, as it satisfies the chaining condition. However, there is no guarantee that D'_t corresponds to any legitimate system state S_t. This highlights the distinction between structural integrity and semantic integrity, with the former being preserved in the minimal model and the latter requiring additional mechanisms.

Despite this limitation, the Minimal Viable Integrity Model provides a significant improvement over conventional telemetry systems. By enforcing deterministic encoding and cryptographic chaining, it eliminates undetected modifications and ensures that the telemetry sequence is both consistent and auditable. In many practical scenarios, particularly those involving non-adversarial environments or internal system monitoring, these guarantees may be sufficient to achieve reliable operation.

The Minimal Viable Integrity Model may also serve as a foundational layer upon which additional mechanisms are incrementally introduced. For example, the proof generation operator $\mathcal{S}$ may be incorporated to provide state binding, and the verification operator $\mathcal{V}$ may be extended to support multi-party validation. This layered approach allows systems to progressively enhance their security posture without requiring a complete redesign of the telemetry pipeline.

From an implementation perspective, the computational cost of the minimal model is significantly lower than that of the full RCO protocol. Let $T_{\mathcal{E}}$ and $T_{\mathcal{H}}$ denote the time complexities of encoding and hashing, respectively. Then the total cost per telemetry entry is:

$$T_{\text{MVI}} = T_{\mathcal{E}} + T_{\mathcal{H}} \tag{A.125}$$

which is substantially less than the cost of signature generation and verification in the full model. This makes the minimal model suitable for high-throughput environments or resource-constrained systems.

Finally, the Minimal Viable Integrity Model satisfies a reduced invariant over the augmented state space:

$$\forall t, \quad L_t = \mathcal{H}(\mathcal{E}(D_t) \parallel L_{t-1}) \tag{A.126}$$

which ensures that the lineage remains consistent and verifiable. While this invariant does not guarantee full security, it establishes a baseline level of integrity that is absent in conventional systems.

In summary, the Minimal Viable Integrity Model represents a reduced configuration of the RCO protocol that preserves deterministic representation and immutable lineage while omitting advanced verification mechanisms. This model provides a practical entry point for adoption and serves as a foundation for progressively enhancing system integrity through the integration of additional components.

A.8 Scalability and Extensibility

The RCO protocol is designed not merely as a theoretically consistent framework for telemetry integrity, but as a system capable of operating under the practical constraints of high-throughput, distributed computational environments. In such environments, telemetry generation rates may be substantial, system components may be geographically distributed, and computational resources may be heterogeneous. The architecture must

therefore satisfy scalability properties while preserving the integrity invariants defined in preceding sections.

Let λ denote the rate of telemetry generation, measured in units of telemetry elements per unit time. Let $\mathcal{P}_{\text{RCO}}$ denote the full processing pipeline, and let $T_{\mathcal{P}}$ denote the expected processing time per telemetry element. A necessary condition for scalability is that the system satisfies:

$$\lim_{\lambda \to \infty} \frac{T_{\mathcal{P}}}{1/\lambda} < \infty \tag{A.127}$$

which ensures that the processing pipeline does not become a bottleneck as input rates increase. In practical terms, this condition requires that the pipeline be parallelizable and that its constituent operations admit efficient implementations.

The encoding operator $\mathcal{E}$ is inherently parallelizable, as it operates independently on each telemetry element. Let $\mathcal{E}_i$ denote the encoding operation applied to the i-th telemetry element. Then:

$$\mathcal{E}(\{D_i\}_{i=1}^N) = \{\mathcal{E}_i(D_i)\}_{i=1}^N \tag{A.128}$$

This independence allows encoding to be distributed across multiple processing units without introducing synchronization constraints.

The chaining operator $\mathcal{C}$ introduces a sequential dependency due to its recursive definition:

$$L_t = \mathcal{H}(\mathcal{E}(D_t) \parallel L_{t-1}) \tag{A.129}$$

However, this dependency is limited to a single scalar state L_{t-1}, allowing the computation to be pipelined. Let $T_{\mathcal{H}}$ denote the time required to compute the hash function. The throughput of the chaining operation is bounded by:

$$\text{Throughput}_{\mathcal{C}} \leq \frac{1}{T_{\mathcal{H}}} \tag{A.130}$$

In practice, modern cryptographic hash functions admit highly optimized implementations, including hardware acceleration, which allows this bound to be sufficiently high for most applications.

The proof generation and verification operators introduce additional computational overhead. Let T_{Sign} and T_{Verify} denote the time required for signature generation and verification, respectively. In a threshold setting with m participating validators, the total cost per telemetry element is:

$$T_{\mathcal{S},\mathcal{V}} = m \cdot T_{\text{Sign}} + T_{\text{Verify}} \tag{A.131}$$

To ensure scalability, these operations must be distributed across validator nodes. Let the set of validators be partitioned across p processing units. Then the effective cost per unit is:

$$T_{\text{per-unit}} = \frac{m}{p} \cdot T_{\text{Sign}} + \frac{1}{p} T_{\text{Verify}} \tag{A.132}$$

which decreases as p increases, assuming balanced load distribution.

The overall pipeline throughput is therefore determined by the maximum of the individual operator costs:

$$\text{Throughput}_{\mathcal{P}} = \min\left(\frac{1}{T_{\mathcal{E}}}, \frac{1}{T_{\mathcal{H}}}, \frac{p}{m \cdot T_{\text{Sign}} + T_{\text{Verify}}}\right) \tag{A.133}$$

This expression highlights the importance of parallelization and distributed processing in achieving scalability.

From a distributed systems perspective, the RCO protocol may be modeled as a set of interacting processes $\{\mathcal{N}_i\}$, each responsible for a subset of telemetry processing tasks. Let $\mathcal{N}_i$ denote node i with local state $(S_t^{(i)}, L_t^{(i)})$. The global system state is defined as the aggregation:

$$\mathcal{X}_t = \bigcup_i (S_t^{(i)}, L_t^{(i)}) \tag{A.134}$$

Consistency across nodes is maintained through synchronization of lineage states and validation of proofs. Let $\mathcal{C}_{ij}$ denote the communication channel between nodes i and j. The system must satisfy:

$$L_t^{(i)} = L_t^{(j)} \quad \forall i, j \tag{A.135}$$

for all accepted telemetry entries. This condition ensures that all nodes maintain a consistent view of the telemetry lineage.

The communication overhead associated with maintaining this consistency may be expressed as:

$$C_{\text{comm}} = O(n \cdot k) \tag{A.136}$$

where n is the number of nodes and k is the size of the lineage hash. This overhead is minimal relative to the size of the telemetry data itself, as only the lineage and proof information need to be transmitted.

Extensibility of the RCO protocol is achieved through the modularity of its operator composition. Each operator $\mathcal{E}$, $\mathcal{C}$, $\mathcal{S}$, and $\mathcal{V}$ may be replaced or extended provided that the core invariants are preserved. Formally, let $\mathcal{E}'$, $\mathcal{C}'$, $\mathcal{S}'$, and $\mathcal{V}'$ denote alternative operators. The system remains valid if:

$$\mathcal{V}' \circ \mathcal{S}' \circ \mathcal{C}' \circ \mathcal{E}' = \text{Valid} \tag{A.137}$$

This condition defines a class of equivalent implementations that satisfy the same integrity requirements.

For example, the hash function $\mathcal{H}$ may be replaced with any function $\mathcal{H}'$ satisfying collision resistance:

$$\Pr[\mathcal{H}'(x) = \mathcal{H}'(y)] \leq \epsilon \quad \text{for } x \neq y \tag{A.138}$$

Similarly, the signature scheme may be replaced with any scheme satisfying existential unforgeability under chosen-message attacks (EUF-CMA). Let Σ' denote such a scheme. Then:

$$\Pr[\text{Forge}(\Sigma')] \leq \epsilon \tag{A.139}$$

ensures that the security properties of the system are preserved.

The protocol also supports hierarchical composition, wherein multiple RCO pipelines are composed to form a higher-level system. Let $\mathcal{P}_1$ and $\mathcal{P}_2$ denote two pipelines. Their composition:

$$\mathcal{P}_{\text{hier}} = \mathcal{P}_2 \circ \mathcal{P}_1 \tag{A.140}$$

remains valid provided that the output of $\mathcal{P}_1$ satisfies the input requirements of $\mathcal{P}_2$. This enables the construction of multi-layered systems with nested integrity guarantees.

From an asymptotic perspective, the storage complexity of the lineage grows linearly with the number of telemetry entries:

$$\text{Storage}(t) = O(t \cdot k) \tag{A.141}$$

However, the verification of integrity at any point requires only the current lineage hash and proof, resulting in constant-time verification:

$$T_{\text{verify}} = O(1) \tag{A.142}$$

This property ensures that the system remains efficient even as the telemetry history grows.

Finally, the scalability and extensibility of the RCO protocol may be summarized by the following invariant:

$$\forall \lambda, \quad \text{Valid}(D_t) \text{ is preserved under parallel, distributed, and extended implementations} \tag{A.143}$$

This invariant ensures that the integrity guarantees of the protocol are maintained regardless of the scale or configuration of the system.

In summary, the RCO protocol achieves scalability through parallelizable encoding, pipelined chaining, and distributed verification, while extensibility is ensured through the modularity of its operator composition. These properties enable the protocol to operate effectively in large-scale, heterogeneous environments without compromising its core integrity guarantees.

A.9 Industrial Applicability

The applicability of the Feedback-Coupled Cryptographic Observation (RCO) protocol extends to any computational domain in which telemetry data participates directly or indirectly in influencing system evolution, decision-making processes, or downstream analytical outcomes. The mathematical framework established in preceding sections demonstrates that the integrity of telemetry is not merely a matter of data correctness but a fundamental requirement for ensuring stability, reproducibility, and trustworthiness of system behavior. Consequently, any domain in which telemetry forms a feedback signal within a dynamical system is inherently susceptible to the distortions previously characterized and therefore stands to benefit from the enforcement of the invariants defined by the RCO protocol.

To formalize this applicability, consider a general class of systems defined by a transition operator:

$$S_{t+1} = \mathcal{F}(S_t, A_t, D_t) \tag{A.144}$$

where D_t represents telemetry that influences subsequent computation. Let $\mathcal{C}_{\text{RCO}}$ denote the constraint set imposed by the RCO protocol. A system is said to be RCO-compliant if:

$$D_t \in \mathcal{C}_{\text{RCO}} \quad \forall t \tag{A.145}$$

which implies that the telemetry satisfies deterministic encoding, cryptographic lineage, and verifiable provenance conditions. The impact of enforcing $\mathcal{C}_{\text{RCO}}$ may be analyzed by comparing system trajectories under constrained and unconstrained telemetry.

Let $\{S_t\}$ denote the trajectory under unconstrained telemetry and $\{S_t^*\}$ the trajectory under RCO-compliant telemetry. From the perturbation model:

$$\Delta_t = \|S_t - S_t^*\| \tag{A.146}$$

and:

$$\Delta_{t+1} \leq L_s \Delta_t + L_d \|\delta_t\| \tag{A.147}$$

Under RCO enforcement, $\delta_t = 0$, yielding:

$$\Delta_{t+1} \leq L_s \Delta_t \tag{A.148}$$

which implies that system behavior is governed solely by initial conditions and control inputs, thereby ensuring reproducibility and stability. This result is domain-agnostic and applies to any system in which telemetry acts as a feedback variable.

In the context of reinforcement learning systems, the state S_t may represent the parameters of a policy or value function, while D_t corresponds to observed rewards, transitions, or gradients. The update rule may be expressed as:

$$\theta_{t+1} = \theta_t + \alpha \cdot \nabla J(\theta_t, D_t) \tag{A.149}$$

where θ_t denotes model parameters and J is an objective function. Any inconsistency in D_t directly affects the gradient computation, resulting in divergence of the training trajectory. Under RCO constraints, the deterministic encoding and lineage verification of D_t ensure that the gradient updates are reproducible, enabling consistent training outcomes across distributed environments.

In financial systems, telemetry may represent market data, transaction logs, or risk indi-

cators that influence trading strategies or risk models. Let S_t denote the state of a trading system and D_t the incoming market data. The decision function may be expressed as:

$$A_t = \pi(S_t, D_t) \tag{A.150}$$

where π is a policy function. In such systems, even minor discrepancies in D_t can lead to divergent trading decisions with significant financial implications. The enforcement of deterministic and verifiable telemetry ensures that decisions are based on consistent data, thereby reducing the risk of unintended behavior and enabling auditability.

In scientific computing and simulation environments, telemetry often represents intermediate results, boundary conditions, or measurement data that influence subsequent computations. Let the system be defined by a numerical solver:

$$S_{t+1} = \mathcal{F}(S_t, D_t) \tag{A.151}$$

where D_t may include discretized inputs or observational constraints. Non-deterministic representation of D_t leads to divergence in simulation outcomes, undermining reproducibility. By enforcing canonical encoding and lineage tracking, the RCO protocol ensures that simulation results can be independently verified and reproduced.

In autonomous systems and robotics, telemetry includes sensor data, environmental observations, and internal diagnostics that influence control decisions. Let S_t represent the state of the system and D_t the sensor input. The control law may be expressed as:

$$A_t = \mathcal{K}(S_t, D_t) \tag{A.152}$$

where $\mathcal{K}$ is a control function. In such systems, inconsistencies in telemetry may lead to unsafe or unpredictable behavior. The RCO protocol ensures that sensor data is processed deterministically and that its integrity is verifiable, thereby enhancing safety and reliability.

In distributed control systems and cyber-physical infrastructures, multiple nodes interact through shared telemetry streams. Let $\mathcal{N}_i$ denote node i with local state $S_t^{(i)}$ and telemetry $D_t^{(i)}$. The global system behavior depends on the aggregation:

$$S_{t+1}^{(i)} = \mathcal{F}_i(S_t^{(i)}, D_t^{(i)}, \{D_t^{(j)}\}_{j \neq i}) \tag{A.153}$$

In such systems, inconsistencies in telemetry across nodes lead to divergence in local states and potential system instability. The RCO protocol enforces a consistent lineage

across nodes:

$$L_t^{(i)} = L_t^{(j)} \quad \forall i, j \tag{A.154}$$

ensuring that all nodes operate on a shared and verifiable history.

From a regulatory and compliance perspective, the ability to provide audit-grade verification of system behavior is increasingly critical. Let $\mathcal{A}$ denote an audit function that reconstructs system history from telemetry. Under conventional systems, $\mathcal{A}$ may produce ambiguous or unverifiable results due to inconsistencies in data. Under RCO enforcement:

$$\mathcal{A}(\{D_t\}, \{L_t\}, \{\Sigma_t\}) = \text{Verified History} \tag{A.155}$$

where the lineage and proofs provide cryptographic guarantees of integrity. This enables independent verification of system behavior and supports compliance with regulatory requirements.

The applicability of the RCO protocol may therefore be characterized by the condition:

$$\exists \mathcal{F} \text{ such that } \frac{\partial S_{t+1}}{\partial D_t} \neq 0 \tag{A.156}$$

which indicates that telemetry influences system evolution. For any such system, the enforcement of RCO constraints eliminates the perturbation channel and ensures deterministic behavior.

In summary, the RCO protocol is applicable to a broad class of computational systems in which telemetry participates in feedback-driven processes. By enforcing deterministic representation, cryptographic lineage, and verifiable provenance, the protocol ensures stability, reproducibility, and auditability across domains including machine learning, finance, scientific computing, autonomous systems, and distributed control infrastructures. The mathematical framework presented herein demonstrates that these benefits arise directly from the elimination of telemetry-induced perturbations and the enforcement of integrity invariants, establishing the RCO protocol as a foundational component for high-assurance computational systems.

A.10 Summary of Contribution

The present disclosure introduces a unified mathematical and architectural framework for ensuring deterministic, verifiable, and tamper-evident telemetry in feedback-coupled computational systems. The necessity of such a framework arises from the structural de-

pendence of system evolution on telemetry data, as formalized by the transition relation:

$$S_{t+1} = \mathcal{F}(S_t, A_t, D_t) \tag{A.157}$$

where the sensitivity condition:

$$\frac{\partial S_{t+1}}{\partial D_t} \neq 0 \tag{A.158}$$

implies that any perturbation in telemetry propagates directly into system behavior. Existing systems fail to control this perturbation channel, resulting in the emergence of Causative Telemetry Distortion, characterized by the divergence relation:

$$\Delta_{t+1} \leq L_s \Delta_t + L_d \|\delta_t\| \tag{A.159}$$

where δ_t represents inconsistencies in telemetry representation or transmission. The absence of deterministic encoding, lineage enforcement, and verifiable provenance in conventional systems ensures that $\delta_t \neq 0$ in general, leading to instability and non-reproducibility.

The disclosed RCO protocol eliminates this perturbation channel by enforcing a set of mathematical invariants over the telemetry pipeline. These invariants are realized through a compositional architecture defined by the operator:

$$\mathcal{A}_{\text{RCO}} = \mathcal{V} \circ \mathcal{S} \circ \mathcal{C} \circ \mathcal{E} \tag{A.160}$$

where each operator enforces a specific constraint necessary for integrity preservation. The encoding operator $\mathcal{E}$ ensures canonical representation, satisfying:

$$D_1 \equiv D_2 \implies \mathcal{E}(D_1) = \mathcal{E}(D_2) \tag{A.161}$$

thereby eliminating representation-induced perturbations. The chaining operator $\mathcal{C}$ constructs an immutable lineage:

$$L_t = \mathcal{H}(\mathcal{E}(D_t) \parallel L_{t-1}) \tag{A.162}$$

which ensures that any modification to historical telemetry is detectable. The proof generation operator $\mathcal{S}$ binds the lineage to the originating computational state:

$$\Sigma_t = \mathrm{Sign}(\mathcal{H}_S(S_t) \parallel L_t, K) \tag{A.163}$$

while the verification operator $\mathcal{V}$ enforces authenticity and correctness through cryptographic validation.

The combined effect of these operators is to enforce the condition:

$$\delta_t = 0 \quad \forall t \tag{A.164}$$

with respect to telemetry-induced perturbations, yielding a reduced divergence relation:

$$\Delta_{t+1} \leq L_s \Delta_t \tag{A.165}$$

which ensures deterministic and stable system evolution.

A key contribution of the present disclosure lies in the formalization of telemetry as a causative variable within system dynamics and the identification of deterministic representation and cryptographic lineage as necessary conditions for stability. This contrasts with existing approaches that treat telemetry as an auxiliary artifact and fail to enforce invariants at the level of system evolution.

Another contribution is the introduction of a unified validity condition that integrates encoding, lineage, and verification into a single criterion:

$$\mathrm{Valid}(D_t) \iff \begin{cases} \mathcal{E}(D_t) \text{ is canonical} \\ L_t = \mathcal{H}(\mathcal{E}(D_t) \parallel L_{t-1}) \\ \mathrm{Verify}(L_t, \Sigma_t) = \text{true} \\ D_t = \mathcal{G}(S_t) \end{cases} \tag{A.166}$$

This condition defines the necessary and sufficient requirements for telemetry integrity in feedback-coupled systems and provides a basis for formal verification.

The disclosure further introduces a Minimal Viable Integrity Model defined by the reduced pipeline:

$$\mathcal{P}_{\mathrm{MVI}} = \mathcal{C} \circ \mathcal{E} \tag{A.167}$$

which preserves deterministic representation and immutable lineage while relaxing provenance constraints. This model enables incremental adoption and broadens the applicability of the protocol across diverse system environments.

From an architectural perspective, the RCO protocol defines a modular and extensible system in which each operator may be independently implemented or replaced, provided that the core invariants are preserved. This modularity is captured by the equivalence condition:

$$\mathcal{V}' \circ \mathcal{S}' \circ \mathcal{C}' \circ \mathcal{E}' = \text{Valid} \tag{A.168}$$

which defines a class of implementations satisfying the same integrity guarantees.

The disclosure also establishes the scalability of the protocol through parallelizable encoding, pipelined chaining, and distributed verification, ensuring that the system can operate under high-throughput conditions without compromising correctness. The distributed validation mechanism provides resilience against Byzantine faults, requiring a threshold of independent validators to authorize telemetry.

In addition to its theoretical contributions, the RCO protocol provides practical benefits across a wide range of domains, including machine learning, financial systems, scientific computing, and autonomous control. In each of these domains, the enforcement of deterministic and verifiable telemetry eliminates sources of instability and enables reproducible and auditable system behavior.

The novelty of the present disclosure resides in the integration of deterministic serialization, cryptographic lineage construction, and feedback-aware validation into a single unified framework that directly addresses the causal role of telemetry in system dynamics. Unlike prior systems that address these aspects in isolation or not at all, the RCO protocol provides a comprehensive solution that enforces integrity at the level of system evolution.

In summary, the present disclosure establishes a new paradigm for telemetry integrity in feedback-coupled computational systems by formalizing the role of telemetry as a causative variable, identifying the invariants required for stability, and providing a scalable and extensible architecture for enforcing these invariants. This paradigm enables the construction of systems that are deterministic, verifiable, and resistant to adversarial manipulation, thereby addressing a fundamental limitation of existing computational infrastructures.

B. Cryptographic Core

B.1 Overview

The cryptographic core of the Feedback-Coupled Cryptographic Observation (RCO) protocol defines the formal mechanisms by which telemetry data is transformed into a deterministic, immutable, and verifiable representation suitable for use in feedback-coupled computational systems. Unlike conventional cryptographic applications that operate on static data, the present framework operates on temporally evolving telemetry sequences that directly influence system dynamics. Consequently, the cryptographic core must enforce not only classical security properties such as collision resistance and unforgeability, but also structural invariants that guarantee deterministic behavior across heterogeneous computational environments.

Let $\mathcal{D}$ denote the space of telemetry data and $\{D_t\}_{t=0}^{T}$ a sequence of telemetry elements. The cryptographic core defines a transformation:

$$\mathcal{K} : \mathcal{D}^T \to \mathcal{L}^T \tag{B.1}$$

where $\mathcal{L}$ denotes the lineage space. The transformation $\mathcal{K}$ must satisfy the following global invariant:

$$\forall D, \tilde{D} \in \mathcal{D}^T, \quad D \equiv \tilde{D} \implies \mathcal{K}(D) = \mathcal{K}(\tilde{D}) \tag{B.2}$$

ensuring that semantically equivalent telemetry sequences produce identical cryptographic representations.

This requirement imposes a stricter condition than standard cryptographic hashing, as it necessitates determinism at the level of input representation prior to hashing. The cryptographic core is therefore decomposed into three primary operators:

$$\mathcal{K} = \mathcal{C} \circ \mathcal{H} \circ \mathcal{S} \tag{B.3}$$

where $\mathcal{S}$ is a canonical serialization operator, $\mathcal{H}$ is a cryptographic hashing operator, and $\mathcal{C}$ is a chaining operator that constructs lineage.

B.2 Deterministic Serialization Layer

The deterministic serialization layer constitutes the foundational component of the cryptographic core, as it ensures that all subsequent cryptographic operations operate on a uniquely defined binary representation. Without this property, the guarantees provided by cryptographic hashing and chaining are invalidated, as identical logical inputs may produce divergent outputs.

B.2.1 Formal Definition of Serialization

Let $\mathcal{D}$ denote the space of structured telemetry data, which may be represented as a finite set of key-value pairs:

$$D = \{(k_i, v_i)\}_{i=1}^{n} \tag{B.4}$$

where $k_i \in \mathcal{K}$ and $v_i \in \mathcal{V}$. A serialization function is defined as:

$$\mathcal{S} : \mathcal{D} \to \{0, 1\}^* \tag{B.5}$$

mapping structured data into a binary sequence.

The serialization function is said to be canonical if it satisfies:

$$D_1 \equiv D_2 \implies \mathcal{S}(D_1) = \mathcal{S}(D_2) \tag{B.6}$$

This property defines injective consistency with respect to semantic equivalence.

B.2.2 Non-Determinism in Conventional Serialization

In conventional systems, serialization functions fail to satisfy the canonical property. Consider two equivalent representations of a dictionary:

$$D_1 = \{(k_1, v_1), (k_2, v_2)\}, \quad D_2 = \{(k_2, v_2), (k_1, v_1)\} \tag{B.7}$$

Then:

$$\mathcal{S}(D_1) \neq \mathcal{S}(D_2) \tag{B.8}$$

in the absence of enforced ordering. This violation introduces ambiguity in downstream

cryptographic operations.

Furthermore, for numerical values $x \in \mathbb{R}$, floating-point representations introduce additional non-determinism. Let $\mathcal{F}_i$ denote the floating-point encoding under architecture i. Then:

$$\mathcal{F}_i(x) \neq \mathcal{F}_j(x) \tag{B.9}$$

for certain values of x, particularly when rounding behavior differs.

B.2.3 Canonical Serialization Construction

To eliminate these sources of non-determinism, the serialization function $\mathcal{S}$ is constructed as a composition:

$$\mathcal{S} = \mathcal{E}_{\text{struct}} \circ \Pi_k \tag{B.10}$$

where Π_k is a fixed-precision projection operator and $\mathcal{E}_{\text{struct}}$ is a structural encoding function.

The projection operator is defined as:

$$\Pi_k : \mathbb{R} \to \mathbb{Z}, \quad \Pi_k(x) = \lfloor x \cdot 10^k \rfloor \tag{B.11}$$

and extended component-wise to structured data:

$$\Pi_k(D) = \{(k_i, \Pi_k(v_i))\}_{i=1}^n \tag{B.12}$$

This transformation maps real-valued inputs into a discrete lattice, eliminating floating-point inconsistencies.

The structural encoding function $\mathcal{E}_{\text{struct}}$ enforces a total ordering over keys. Let $\prec$ denote a lexicographic ordering over $\mathcal{K}$. Then:

$$\mathcal{E}_{\text{struct}}(D) = \text{Encode}(\{(k_{\pi(i)}, v_{\pi(i)})\}_{i=1}^n) \tag{B.13}$$

where π is a permutation such that:

$$k_{\pi(1)} \prec k_{\pi(2)} \prec \cdots \prec k_{\pi(n)} \tag{B.14}$$

This ensures that the ordering of elements is uniquely defined.

B.2.4 Determinism Theorem

Theorem B.1 (Canonical Determinism)**.** *The serialization function* $\mathcal{S} = \mathcal{E}_{struct} \circ \Pi_k$ *is canonical, i.e.,*

$$D_1 \equiv D_2 \implies \mathcal{S}(D_1) = \mathcal{S}(D_2)$$

Proof. Assume $D_1 \equiv D_2$. Then for each key k, the corresponding values satisfy $v_1 = v_2$. Applying Π_k yields identical integer representations:

$$\Pi_k(v_1) = \Pi_k(v_2)$$

Since both datasets contain identical key-value pairs, sorting under lexicographic order produces identical sequences. Therefore, the structural encoding produces identical outputs:

$$\mathcal{E}_{\text{struct}}(\Pi_k(D_1)) = \mathcal{E}_{\text{struct}}(\Pi_k(D_2))$$

Thus:

$$\mathcal{S}(D_1) = \mathcal{S}(D_2)$$

which completes the proof. □

B.2.5 Cross-Platform Consistency

Let $\mathcal{S}_i$ denote the serialization function executed on platform i. The use of fixed-precision projection ensures:

$$\forall i, j, \quad \mathcal{S}_i(D) = \mathcal{S}_j(D) \tag{B.15}$$

as Π_k eliminates floating-point discrepancies and $\mathcal{E}_{\text{struct}}$ enforces deterministic ordering independent of platform.

B.2.6 Stability Under Perturbation

Consider a perturbation δ applied to a numerical value x such that $|\delta| < 10^{-k}$. Then:

$$\Pi_k(x + \delta) = \Pi_k(x) \tag{B.16}$$

provided that the perturbation does not cross a quantization boundary. This implies that the serialization function is robust to small perturbations within quantization limits, preventing minor numerical noise from affecting the encoded representation.

B.2.7 Entropy Preservation

Let $H(D)$ denote the entropy of the telemetry data. The projection operator Π_k introduces quantization, reducing entropy:

$$H(\Pi_k(D)) \leq H(D) \tag{B.17}$$

However, this reduction is controlled by the choice of k, allowing a trade-off between precision and determinism. The structural encoding preserves entropy with respect to ordering, as it introduces no additional ambiguity.

B.2.8 Implications for Cryptographic Processing

The canonical serialization function ensures that all subsequent cryptographic operations operate on a stable and unique representation. In particular, for any hash function $\mathcal{H}$:

$$\mathcal{H}(\mathcal{S}(D_1)) = \mathcal{H}(\mathcal{S}(D_2)) \iff D_1 \equiv D_2 \tag{B.18}$$

under the assumption that $\mathcal{H}$ is collision-resistant. This establishes the necessary precondition for reliable hashing and lineage construction.

In summary, the deterministic serialization layer eliminates all sources of representation-induced ambiguity by enforcing canonical ordering and fixed-precision normalization. This layer provides the foundation upon which the remaining components of the cryptographic core are constructed, ensuring that the integrity guarantees of the system are preserved across heterogeneous environments and repeated executions.

B.3 Cryptographic Hashing Layer

The cryptographic hashing layer transforms the canonical binary representation of telemetry data into a fixed-length digest that serves as the fundamental unit of integrity within the RCO protocol. While the deterministic serialization layer ensures that semantically equivalent inputs produce identical binary encodings, the hashing layer ensures that any

deviation in this encoding is reflected in a corresponding change in the hash output with overwhelming probability. The hashing function thus acts as a sensitivity amplifier, converting even minimal differences in input into statistically independent outputs.

Let $\mathcal{H} : \{0,1\}^* \to \{0,1\}^k$ denote a cryptographic hash function mapping arbitrary-length inputs to fixed-length outputs of size k. The choice of $\mathcal{H}$ must satisfy standard cryptographic security properties, which are formally defined below.

B.3.1 Preimage Resistance

The hash function $\mathcal{H}$ is said to be preimage-resistant if, given a hash value $y \in \{0,1\}^k$, it is computationally infeasible to find an input $x \in \{0,1\}^*$ such that:

$$\mathcal{H}(x) = y \tag{B.19}$$

Formally, for any probabilistic polynomial-time adversary $\mathcal{A}$:

$$\Pr[\mathcal{A}(y) = x \text{ such that } \mathcal{H}(x) = y] \leq \epsilon \tag{B.20}$$

where ϵ is negligible in k. This property ensures that given a lineage hash L_t, an adversary cannot reconstruct the underlying telemetry data.

B.3.2 Second-Preimage Resistance

The hash function is second-preimage resistant if, given an input x, it is computationally infeasible to find a distinct input $x' \neq x$ such that:

$$\mathcal{H}(x') = \mathcal{H}(x) \tag{B.21}$$

Formally:

$$\Pr[\mathcal{A}(x) = x' \neq x \text{ such that } \mathcal{H}(x') = \mathcal{H}(x)] \leq \epsilon \tag{B.22}$$

This property is essential for ensuring that an adversary cannot replace a telemetry entry with a different one that yields the same hash.

B.3.3 Collision Resistance

The hash function is collision-resistant if it is computationally infeasible to find any pair of distinct inputs (x, x') such that:

$$\mathcal{H}(x) = \mathcal{H}(x') \tag{B.23}$$

Formally:

$$\Pr[\mathcal{A}() = (x, x') \text{ with } x \neq x' \text{ and } \mathcal{H}(x) = \mathcal{H}(x')] \leq \epsilon \tag{B.24}$$

Collision resistance ensures that the lineage structure uniquely represents the telemetry sequence.

B.3.4 Content Hash Definition

Given a telemetry element D_t, its canonical encoding is $\mathcal{S}(D_t)$. The content hash is defined as:

$$H_t = \mathcal{H}(\mathcal{S}(D_t)) \tag{B.25}$$

This hash serves as a compact representation of the telemetry element.

B.3.5 Avalanche Property and Sensitivity Amplification

A desirable property of $\mathcal{H}$ is the avalanche effect, whereby a single-bit change in the input results in a statistically independent output. Let x and x' differ by a single bit. Then:

$$\Pr[\mathcal{H}(x)_i \neq \mathcal{H}(x')_i] \approx \frac{1}{2} \tag{B.26}$$

for each output bit i. This ensures that even minimal perturbations in the serialized telemetry result in large changes in the hash output.

From the perspective of Causative Telemetry Distortion, this property ensures that any non-zero perturbation δ_t in the serialized representation leads to:

$$H_t \neq H'_t \tag{B.27}$$

with overwhelming probability, thereby making inconsistencies detectable.

B.3.6 Adversarial Model

Consider an adversary $\mathcal{A}$ attempting to manipulate telemetry data. The adversary may attempt to construct an alternative telemetry element D'_t such that:

$$\mathcal{H}(\mathcal{S}(D'_t)) = \mathcal{H}(\mathcal{S}(D_t)) \tag{B.28}$$

This constitutes a second-preimage attack. Under the assumption that $\mathcal{H}$ is second-preimage resistant, the probability of success is bounded by:

$$\Pr[\text{Success}] \leq 2^{-k} \tag{B.29}$$

which is negligible for sufficiently large k.

Alternatively, the adversary may attempt to find any two distinct telemetry elements D_t and D'_t such that:

$$\mathcal{H}(\mathcal{S}(D_t)) = \mathcal{H}(\mathcal{S}(D'_t)) \tag{B.30}$$

This constitutes a collision attack. The expected complexity of such an attack is:

$$O(2^{k/2}) \tag{B.31}$$

due to the birthday bound. For $k = 256$, this corresponds to approximately 2^{128} operations, which is computationally infeasible.

B.3.7 Hash Function as a Random Oracle Approximation

In the security analysis of the RCO protocol, the hash function $\mathcal{H}$ may be modeled as a random oracle. That is, $\mathcal{H}$ is treated as a function that maps inputs to uniformly random outputs, subject to consistency constraints. Under this model, for any distinct inputs $x \neq x'$:

$$\Pr[\mathcal{H}(x) = \mathcal{H}(x')] = 2^{-k} \tag{B.32}$$

This abstraction simplifies the analysis of lineage integrity and proof unforgeability.

B.3.8 Entropy Compression and Information Preservation

The hash function compresses input data of arbitrary length into a fixed-length output. Let $H(\cdot)$ denote entropy. Then:

$$H(H_t) \leq k \tag{B.33}$$

while the input entropy $H(\mathcal{S}(D_t))$ may be significantly larger. Despite this compression, collision resistance ensures that distinct inputs remain distinguishable with high probability.

B.3.9 Compositional Integrity with Serialization

The combination of canonical serialization and cryptographic hashing yields a composite function:

$$\mathcal{H} \circ \mathcal{S} : \mathcal{D} \to \{0,1\}^k \tag{B.34}$$

which satisfies:

$$D_1 \equiv D_2 \implies \mathcal{H}(\mathcal{S}(D_1)) = \mathcal{H}(\mathcal{S}(D_2)) \tag{B.35}$$

and:

$$D_1 \not\equiv D_2 \implies \Pr[\mathcal{H}(\mathcal{S}(D_1)) = \mathcal{H}(\mathcal{S}(D_2))] \leq 2^{-k} \tag{B.36}$$

This establishes that the composite function is both deterministic and collision-resistant with respect to semantic equivalence.

B.3.10 Robustness Against Structural Attacks

An adversary may attempt to exploit structural properties of the serialization to produce collisions. However, the canonical nature of $\mathcal{S}$ ensures that the input space is well-defined and free of ambiguity. Combined with the cryptographic properties of $\mathcal{H}$, this prevents structural manipulation from yielding identical hash outputs.

B.3.11 Implications for Lineage Construction

The content hash H_t serves as the input to the chaining operator. The security of the lineage structure depends critically on the properties of $\mathcal{H}$. In particular, the collision resistance of $\mathcal{H}$ ensures that the lineage uniquely represents the sequence of telemetry elements.

In summary, the cryptographic hashing layer provides a mechanism for transforming deterministic telemetry representations into fixed-length digests that preserve integrity and detect any deviation with overwhelming probability. By satisfying preimage resistance, second-preimage resistance, and collision resistance, the hash function ensures that the

telemetry data cannot be manipulated without detection, forming a critical component of the RCO protocol.

B.4 Merkle-Lineage Construction

The Merkle-Lineage construction constitutes the temporal integrity backbone of the RCO protocol. While the hashing layer ensures that individual telemetry elements are uniquely represented, it does not by itself enforce any relationship between successive elements. The lineage construction introduces a recursive dependency structure over the sequence of telemetry data, ensuring that the integrity of the entire history is cryptographically bound to the current state.

Let $\{D_t\}_{t=0}^{T}$ denote a sequence of telemetry elements and $\{H_t\}_{t=0}^{T}$ their corresponding content hashes, where:

$$H_t = \mathcal{H}(\mathcal{S}(D_t)) \tag{B.37}$$

The lineage sequence $\{L_t\}_{t=0}^{T}$ is defined recursively as:

$$L_t = \mathcal{H}(H_t \parallel L_{t-1}) \tag{B.38}$$

with an initial condition:

$$L_0 = \mathcal{H}(H_0 \parallel \mathsf{IV}) \tag{B.39}$$

where IV is a fixed initialization vector or genesis constant.

This recursive definition induces a chain structure in which each lineage value L_t depends on all preceding telemetry elements.

B.4.1 Lineage as a Cryptographic State Machine

The lineage sequence may be interpreted as the evolution of a cryptographic state machine over the state space $\mathcal{L} = \{0,1\}^k$. The transition function is defined as:

$$L_t = \mathcal{F}_L(L_{t-1}, D_t) = \mathcal{H}(\mathcal{H}(\mathcal{S}(D_t)) \parallel L_{t-1}) \tag{B.40}$$

This formulation reveals that the lineage state encapsulates the entire history of telemetry through iterative compression.

Let $\mathcal{L}_t$ denote the set of all possible lineage states at time t. Then:

$$|\mathcal{L}_t| \leq 2^k \tag{B.41}$$

while the number of possible telemetry histories grows exponentially with t. The cryptographic properties of $\mathcal{H}$ ensure that distinct histories map to distinct lineage states with overwhelming probability.

B.4.2 Forward Integrity Property

A fundamental property of the lineage construction is forward integrity, which ensures that any modification to historical data propagates to all future lineage states.

Theorem B.2 (Forward Integrity)**.** *Let $\{D_t\}$ and $\{D'_t\}$ be two telemetry sequences such that $D'_k \neq D_k$ for some $k < t$. Then:*

$$\Pr[L_t = L'_t] \leq 2^{-k}$$

Proof. Assume $D'_k \neq D_k$. Then:

$$H'_k = \mathcal{H}(\mathcal{S}(D'_k)) \neq \mathcal{H}(\mathcal{S}(D_k)) = H_k$$

with overwhelming probability due to collision resistance. Therefore:

$$L'_k = \mathcal{H}(H'_k \parallel L_{k-1}) \neq \mathcal{H}(H_k \parallel L_{k-1}) = L_k$$

Again, with overwhelming probability. By induction:

$$L'_{k+1} = \mathcal{H}(H_{k+1} \parallel L'_k) \neq \mathcal{H}(H_{k+1} \parallel L_k) = L_{k+1}$$

and this inequality propagates forward to L_t. The only way for $L_t = L'_t$ is if a collision occurs in $\mathcal{H}$, which has probability at most 2^{-k}. □

B.4.3 Tamper Propagation and Detection

The forward integrity property implies that any tampering with historical telemetry results in a mismatch in the current lineage state. Let Δ_t^L denote the lineage divergence:

$$\Delta_t^L = \|L_t - L'_t\| \tag{B.42}$$

Then:

$$\Delta_t^L \neq 0 \quad \text{if any } D'_k \neq D_k \text{ for } k < t \tag{B.43}$$

with overwhelming probability. This provides a mechanism for detecting tampering by comparing lineage values.

B.4.4 Resistance to Truncation Attacks

An adversary may attempt to truncate the telemetry sequence by removing entries beyond a certain index k. Let the original sequence be $\{D_0, \ldots, D_t\}$ and the truncated sequence be $\{D_0, \ldots, D_k\}$. The corresponding lineage states are L_t and L_k.

The adversary may attempt to construct a new sequence $\{D'_{k+1}, \ldots, D'_t\}$ such that:

$$L'_t = L_t \tag{B.44}$$

This requires finding inputs such that:

$$\mathcal{H}(H'_t \parallel \ldots \mathcal{H}(H'_{k+1} \parallel L_k)) = L_t \tag{B.45}$$

This is equivalent to a preimage attack on $\mathcal{H}$, which is computationally infeasible. Therefore, truncation attacks are prevented.

B.4.5 Resistance to Reordering Attacks

Consider a permutation π applied to the telemetry sequence such that:

$$D'_t = D_{\pi(t)} \tag{B.46}$$

Unless π is the identity permutation, the sequence of content hashes $\{H'_t\}$ differs from $\{H_t\}$. Due to the recursive structure of the lineage, this results in:

$$L'_t \neq L_t \tag{B.47}$$

with overwhelming probability. Thus, the lineage construction enforces strict ordering.

B.4.6 Uniqueness of Lineage Representation

The lineage function defines a mapping:

$$\mathcal{L} : \mathcal{D}^t \to \{0,1\}^k \tag{B.48}$$

which is injective with high probability. That is:

$$\Pr[\mathcal{L}(D) = \mathcal{L}(D')] \leq 2^{-k} \quad \text{for } D \neq D' \tag{B.49}$$

This ensures that the lineage uniquely identifies the telemetry history.

B.4.7 Relation to Merkle Structures

The lineage construction may be viewed as a degenerate Merkle tree in which each node has a single parent. Unlike traditional Merkle trees, which provide logarithmic verification paths for static datasets, the lineage construction provides constant-time verification for dynamic sequences.

Let $\mathcal{T}$ denote a Merkle tree and $\mathcal{L}$ the lineage. Then:

$$\mathcal{L}(D_0, \ldots, D_t) = \mathcal{T}_{\text{linear}}(D_0, \ldots, D_t) \tag{B.50}$$

where $\mathcal{T}_{\text{linear}}$ is a tree with depth t and branching factor 1.

B.4.8 Composability of Lineage

The lineage construction is composable across segments. Let $\{D_0, \ldots, D_k\}$ and $\{D_{k+1}, \ldots, D_t\}$ be two segments. Then:

$$L_t = \mathcal{C}(L_k, \{D_{k+1}, \ldots, D_t\}) \tag{B.51}$$

This property enables distributed processing and later recombination of telemetry segments.

B.4.9 Entropy Accumulation

Although each lineage state has fixed size k, it encodes information about the entire history. Let $H(D_0, \ldots, D_t)$ denote the entropy of the telemetry sequence. Then:

$$H(L_t) \leq k \tag{B.52}$$

but collision resistance ensures that distinct histories remain distinguishable.

B.4.10 Implications for System Integrity

The lineage construction enforces a global invariant:

$$L_t = \mathcal{H}(H_t \parallel L_{t-1}) \quad \forall t \tag{B.53}$$

which ensures that the telemetry sequence is immutable, ordered, and verifiable. Any deviation from this invariant results in a detectable inconsistency.

In summary, the Merkle-Lineage construction provides a cryptographic mechanism for enforcing temporal integrity over telemetry sequences. By embedding each element within a recursively defined hash chain, the protocol ensures that any modification, omission, or reordering of telemetry data is detectable with overwhelming probability. This construction forms the core of the RCO protocol's ability to guarantee tamper-evident and reproducible system behavior.

B.5 Proof of Reflexion (PoR)

While deterministic serialization and Merkle-lineage construction ensure structural integrity and temporal consistency of telemetry data, they do not, by themselves, guarantee that the telemetry originates from a legitimate computational process or that it accurately reflects the internal state of the system. The Proof of Reflexion (PoR) addresses this limitation by introducing a cryptographic mechanism that binds telemetry lineage to the originating computational state, thereby enforcing semantic integrity in addition to structural integrity.

The central concept underlying PoR is that telemetry disclosures must not only be consistent and immutable but must also be reflexively derived from the internal state of the system. This requirement is formalized by establishing a cryptographic dependency between the lineage state L_t and a representation of the system state S_t.

B.5.1 Formal Definition of Reflexive Binding

Let $\mathcal{H}_S : \mathcal{S} \to \{0,1\}^k$ denote a state hashing function that maps the system state to a fixed-length digest. The reflexive binding condition requires that for each telemetry element D_t:

$$D_t = \mathcal{G}(S_t) \tag{B.54}$$

for some observation function $\mathcal{G}$. The Proof of Reflexion enforces this relationship indi-

rectly by constructing a cryptographic proof:

$$\Sigma_t = \mathrm{Sign}(\mathcal{H}_S(S_t) \parallel L_t, K) \tag{B.55}$$

where K is a private key associated with the computational entity.

This construction ensures that the proof Σ_t is a function of both the lineage and the internal state, thereby binding the two.

B.5.2 Verification Condition

Given a public key PK, the verification operator evaluates:

$$\mathcal{V}_{\mathrm{PoR}}(S_t, L_t, \Sigma_t) = \mathrm{Verify}(\mathcal{H}_S(S_t) \parallel L_t, \Sigma_t, PK) \tag{B.56}$$

The proof is considered valid if and only if:

$$\mathcal{V}_{\mathrm{PoR}}(S_t, L_t, \Sigma_t) = \mathrm{true} \tag{B.57}$$

This condition ensures that the lineage L_t is cryptographically bound to a valid state representation.

B.5.3 State Consistency Guarantee

The PoR mechanism enforces that any valid telemetry entry must correspond to a state S_t satisfying:

$$\Sigma_t = \mathrm{Sign}(\mathcal{H}_S(S_t) \parallel L_t, K) \tag{B.58}$$

An adversary attempting to produce a valid proof for a falsified telemetry element D'_t must either:

$$\text{(i)} \quad \mathcal{H}_S(S'_t) = \mathcal{H}_S(S_t) \tag{B.59}$$

or:

$$\text{(ii)} \quad \text{forge } \Sigma_t \tag{B.60}$$

The first condition corresponds to a collision in the state hash function, while the second

corresponds to a signature forgery.

B.5.4 Unforgeability Theorem

Theorem B.3 (PoR Unforgeability)**.** *Assuming that the signature scheme is existentially unforgeable under chosen-message attacks (EUF-CMA) and that $\mathcal{H}_S$ is collision-resistant, an adversary cannot produce a valid proof Σ'_t for a falsified pair (S'_t, L_t) without access to the private key K.*

Proof. Suppose an adversary $\mathcal{A}$ produces a valid proof Σ'_t for (S'_t, L_t) such that:

$$\text{Verify}(\mathcal{H}_S(S'_t) \parallel L_t, \Sigma'_t, PK) = \text{true}$$

There are two possibilities:

1. $\mathcal{H}_S(S'_t) = \mathcal{H}_S(S_t)$, implying a collision in $\mathcal{H}_S$.
2. Σ'_t is a valid signature on a new message $\mathcal{H}_S(S'_t) \parallel L_t$.

The first case contradicts the collision resistance of $\mathcal{H}_S$. The second case contradicts the EUF-CMA security of the signature scheme. Therefore, the adversary cannot succeed except with negligible probability. □

B.5.5 Resistance to State Substitution Attacks

An adversary may attempt to substitute the system state while preserving the lineage. Let (S_t, L_t) be a valid pair and (S'_t, L_t) a substituted pair. The adversary must produce:

$$\Sigma'_t = \text{Sign}(\mathcal{H}_S(S'_t) \parallel L_t, K) \tag{B.61}$$

Without access to K, this is infeasible. Thus, the PoR mechanism prevents substitution of system state without detection.

B.5.6 Binding Strength Analysis

The strength of the binding between S_t and L_t depends on the entropy of $\mathcal{H}_S(S_t)$. Let:

$$H(\mathcal{H}_S(S_t)) = k \tag{B.62}$$

Then the probability of a collision is bounded by:

$$\Pr[\mathcal{H}_S(S_t) = \mathcal{H}_S(S'_t)] \leq 2^{-k} \tag{B.63}$$

This ensures that the binding is strong and resistant to collision-based attacks.

B.5.7 Extension to Threshold Proofs

In a distributed setting, the PoR mechanism may be extended to a threshold signature scheme. Let K_i denote key shares held by validators. Each validator produces:

$$\sigma_i = \text{Sign}(\mathcal{H}_S(S_t) \parallel L_t, K_i) \tag{B.64}$$

An aggregate proof is constructed as:

$$\Sigma_t = \text{Aggregate}(\sigma_{i_1}, \ldots, \sigma_{i_m}) \tag{B.65}$$

for $m \geq t$, where t is the threshold. The verification condition remains:

$$\text{Verify}(\mathcal{H}_S(S_t) \parallel L_t, \Sigma_t, PK) = \text{true} \tag{B.66}$$

This extension provides resilience against Byzantine faults.

B.5.8 Separation from Classical Signatures

It is important to distinguish PoR from classical digital signatures. In standard systems, signatures are applied directly to data:

$$\Sigma_t = \text{Sign}(D_t, K) \tag{B.67}$$

This provides authenticity but does not guarantee that D_t is consistent with the system state. In contrast, PoR signs a composite message:

$$\mathcal{H}_S(S_t) \parallel L_t \tag{B.68}$$

which encodes both the lineage and the state, thereby enforcing a stronger integrity condition.

B.5.9 Reflexive Integrity Condition

The PoR mechanism enforces the following invariant:

$$\text{Valid}(D_t) \implies \exists S_t \text{ such that } D_t = \mathcal{G}(S_t) \wedge \text{Verify}(\mathcal{H}_S(S_t) \parallel L_t, \Sigma_t) \tag{B.69}$$

This condition ensures that valid telemetry is both structurally correct and semantically consistent.

B.5.10 Implications for System Security

By binding telemetry lineage to system state, the PoR mechanism eliminates a critical class of attacks in which adversaries generate structurally valid but semantically incorrect telemetry. Combined with the lineage construction, this ensures that telemetry data is both tamper-evident and state-consistent.

In summary, the Proof of Reflexion introduces a novel cryptographic mechanism that extends traditional integrity guarantees to include state provenance. By binding telemetry to the originating computational state, the RCO protocol ensures that data is not only consistent and immutable but also semantically valid, thereby addressing a fundamental limitation of existing systems.

B.6 Unified Integrity Model and Security Composition

The preceding sections have introduced the individual components of the cryptographic core, namely deterministic serialization, cryptographic hashing, Merkle-lineage construction, and Proof of Reflexion. Each component enforces a specific invariant necessary for ensuring the integrity of telemetry data in feedback-coupled computational systems. However, the full strength of the RCO protocol emerges only when these components are composed into a unified framework that enforces a global integrity condition.

Let the telemetry processing pipeline be defined as the composition:

$$\mathcal{P}_{\text{RCO}} = \mathcal{V}_{\text{PoR}} \circ \mathcal{S}_{\text{PoR}} \circ \mathcal{C} \circ \mathcal{H} \circ \mathcal{S} \tag{B.70}$$

where $\mathcal{S}$ is the canonical serialization operator, $\mathcal{H}$ is the cryptographic hashing operator, $\mathcal{C}$ is the lineage construction operator, $\mathcal{S}_{\text{PoR}}$ is the proof generation operator, and $\mathcal{V}_{\text{PoR}}$ is the verification operator.

The pipeline $\mathcal{P}_{\text{RCO}}$ defines a mapping:

$$\mathcal{P}_{\text{RCO}} : \mathcal{D} \times \mathcal{S} \to \{\text{true}, \text{false}\} \tag{B.71}$$

which determines whether a telemetry element is valid.

B.6.1 Composite Validity Condition

A telemetry element D_t is considered valid if and only if the following composite condition is satisfied:

$$\text{Valid}(D_t) \iff \begin{cases} \mathcal{S}(D_t) \text{ is canonical} \\ H_t = \mathcal{H}(\mathcal{S}(D_t)) \\ L_t = \mathcal{H}(H_t \parallel L_{t-1}) \\ \Sigma_t = \text{Sign}(\mathcal{H}_S(S_t) \parallel L_t, K) \\ \text{Verify}(\mathcal{H}_S(S_t) \parallel L_t, \Sigma_t, PK) = \text{true} \end{cases} \tag{B.72}$$

This condition integrates representation determinism, structural integrity, temporal continuity, and state provenance into a single verification criterion.

B.6.2 Global Integrity Theorem

Theorem B.4 (Global Integrity of RCO)**.** *Assuming that the serialization function $\mathcal{S}$ is canonical, the hash function $\mathcal{H}$ is collision-resistant, the state hash function $\mathcal{H}_S$ is collision-resistant, and the signature scheme is EUF-CMA secure, the RCO pipeline satisfies the following property:*

$$\forall D, \tilde{D}, \quad \mathcal{P}_{RCO}(D) = \mathcal{P}_{RCO}(\tilde{D}) = \textit{true} \implies D \equiv \tilde{D}$$

with overwhelming probability.

Proof. Assume $\mathcal{P}_{\text{RCO}}(D) = \mathcal{P}_{\text{RCO}}(\tilde{D}) = \text{true}$. Then both inputs satisfy the composite validity condition.

From canonical serialization:

$$\mathcal{S}(D) = \mathcal{S}(\tilde{D}) \implies D \equiv \tilde{D}$$

Otherwise, if $D \not\equiv \tilde{D}$, then:

$$\mathcal{S}(D) \neq \mathcal{S}(\tilde{D})$$

which implies:

$$\mathcal{H}(\mathcal{S}(D)) \neq \mathcal{H}(\mathcal{S}(\tilde{D}))$$

with overwhelming probability due to collision resistance. This leads to:

$$H_t \neq \tilde{H}_t \implies L_t \neq \tilde{L}_t$$

by the forward integrity of the lineage construction.

Finally, any attempt to produce valid proofs Σ_t and $\tilde{\Sigma}_t$ for inconsistent lineage states or system states would require either forging a signature or finding a collision in $\mathcal{H}_S$, both of which are infeasible under the given assumptions.

Therefore, $D \equiv \tilde{D}$ must hold, completing the proof. □

B.6.3 Elimination of the Perturbation Channel

From the system dynamics established in Chapter 01, telemetry-induced perturbations are represented by δ_t. Under the RCO protocol, the validity condition enforces:

$$\delta_t = 0 \quad \forall t \tag{B.73}$$

with respect to representation and structural integrity. Substituting into the divergence relation:

$$\Delta_{t+1} \leq L_s \Delta_t + L_d \|\delta_t\| \tag{B.74}$$

yields:

$$\Delta_{t+1} \leq L_s \Delta_t \tag{B.75}$$

which ensures that system evolution is deterministic and free from telemetry-induced distortion.

B.6.4 Security Reduction

The security of the RCO protocol may be reduced to the security of its underlying primitives. Let $\mathcal{A}$ be an adversary attempting to violate the integrity of the system. Then $\mathcal{A}$ must succeed in at least one of the following:

$$\text{(i)} \quad \text{Break canonical determinism of } \mathcal{S} \tag{B.76}$$

$$\text{(ii)} \quad \text{Find a collision in } \mathcal{H} \tag{B.77}$$

$$\text{(iii)} \quad \text{Find a collision in } \mathcal{H}_S \tag{B.78}$$

$$\text{(iv)} \quad \text{Forge a signature under EUF-CMA} \tag{B.79}$$

Thus, the overall security is bounded by:

$$\Pr[\text{Break RCO}] \leq \epsilon_{\mathcal{S}} + \epsilon_{\mathcal{H}} + \epsilon_{\mathcal{H}_S} + \epsilon_{\text{sig}} \tag{B.80}$$

where each term is negligible.

B.6.5 Minimal Integrity Composition

In the Minimal Viable Integrity Model, the pipeline reduces to:

$$\mathcal{P}_{\text{MVI}} = \mathcal{C} \circ \mathcal{H} \circ \mathcal{S} \tag{B.81}$$

The corresponding validity condition is:

$$\text{Valid}_{\text{MVI}}(D_t) \iff L_t = \mathcal{H}(\mathcal{H}(\mathcal{S}(D_t)) \parallel L_{t-1}) \tag{B.82}$$

This ensures structural and temporal integrity but does not enforce state provenance.

B.6.6 Compositional Closure

The RCO pipeline is closed under composition in the sense that multiple pipelines may be combined while preserving integrity. Let $\mathcal{P}_1$ and $\mathcal{P}_2$ be two valid pipelines. Then:

$$\mathcal{P}_{\text{combined}} = \mathcal{P}_2 \circ \mathcal{P}_1 \tag{B.83}$$

satisfies the same integrity condition, provided that the output of $\mathcal{P}_1$ conforms to the input requirements of $\mathcal{P}_2$.

B.6.7 Final Integrity Invariant

The RCO protocol enforces the following global invariant over the telemetry sequence:

$$\forall t, \quad \text{Valid}(D_t) = \text{true} \implies \begin{cases} D_t \text{ is deterministically represented} \\ \text{The sequence } \{D_0, \ldots, D_t\} \text{ is immutable} \\ D_t \text{ is consistent with } S_t \\ \text{The lineage } L_t \text{ uniquely identifies the history} \end{cases} \tag{B.84}$$

This invariant encapsulates the complete integrity guarantees of the protocol.

B.6.8 Conclusion

The unified cryptographic core of the RCO protocol establishes a formally verified framework for ensuring the integrity of telemetry data in feedback-coupled systems. By composing deterministic serialization, cryptographic hashing, lineage construction, and Proof of Reflexion, the protocol enforces a comprehensive set of invariants that eliminate ambiguity, prevent tampering, and ensure consistency between telemetry and system state.

This unified model not only addresses the deficiencies of existing systems but also provides a foundation for the development of high-assurance computational infrastructures in which telemetry integrity is treated as a first-class requirement.

C. Zero-Trust Ingestion

C.1 Overview

The ingestion phase of the Feedback-Coupled Cryptographic Observation (RCO) protocol constitutes the boundary at which externally generated telemetry is admitted into the cryptographically secured lineage. Unlike traditional ingestion mechanisms that rely on implicit trust in data producers or centralized validation authorities, the RCO protocol adopts a zero-trust model in which every telemetry element is treated as potentially adversarial until it satisfies a formally defined set of verification conditions.

Let $\mathcal{D}$ denote the space of telemetry inputs and $\mathcal{I}$ the ingestion operator. The ingestion process defines a mapping:

$$\mathcal{I} : \mathcal{D} \times \mathcal{L} \to \{\text{accept}, \text{reject}\} \tag{C.1}$$

where $\mathcal{L}$ denotes the lineage state. The operator $\mathcal{I}$ must enforce that only telemetry satisfying the global integrity condition defined in the cryptographic core is admitted into the system.

Formally, the ingestion operator must satisfy:

$$\mathcal{I}(D_t, L_{t-1}) = \text{accept} \iff \text{Valid}(D_t) \tag{C.2}$$

where $\text{Valid}(D_t)$ is the composite condition defined in Section 02.

C.2 Adversarial Model

The zero-trust ingestion framework is designed under a strong adversarial model in which multiple components of the system may be compromised. Let $\mathcal{A}$ denote an adversary with the following capabilities:

$$\mathcal{A} = (\mathcal{A}_{\text{inject}}, \mathcal{A}_{\text{modify}}, \mathcal{A}_{\text{replay}}, \mathcal{A}_{\text{collude}}) \tag{C.3}$$

where each component represents a class of attacks.

The injection capability allows the adversary to generate arbitrary telemetry D'_t:

$$D'_t \in \mathcal{D} \tag{C.4}$$

The modification capability allows alteration of in-transit data:

$$D'_t = D_t + \delta_t \tag{C.5}$$

The replay capability allows reuse of previously valid telemetry:

$$D'_t = D_k \quad \text{for } k < t \tag{C.6}$$

The collusion capability allows multiple compromised nodes to coordinate their actions.

The goal of the adversary is to produce a telemetry element D'_t such that:

$$\mathcal{I}(D'_t, L_{t-1}) = \text{accept} \quad \text{and} \quad D'_t \not\equiv D_t \tag{C.7}$$

That is, to inject invalid telemetry that passes validation.

C.3 Zero-Trust Principle

The ingestion framework enforces the principle that no entity is inherently trusted. Let $\mathcal{N}$ denote the set of nodes in the system. In a zero-trust model:

$$\forall i \in \mathcal{N}, \quad \text{Trust}(i) = 0 \tag{C.8}$$

Trust is replaced by verification, such that acceptance is determined solely by the satisfaction of cryptographic and structural conditions.

This principle may be formalized as:

$$\mathcal{I}(D_t, L_{t-1}) = \text{accept} \iff \mathcal{V}_{\text{global}}(D_t, L_{t-1}) = \text{true} \tag{C.9}$$

independent of the origin of D_t.

C.4 Multi-Party Verification Model

To eliminate single points of failure, the ingestion process employs a distributed verification model involving a set of validator nodes. Let $\mathcal{V} = \{V_1, V_2, \ldots, V_n\}$ denote the set of

validators.

Each validator V_i implements a local verification function:

$$\mathcal{V}_i : \mathcal{D} \times \mathcal{L} \to \{\text{true}, \text{false}\} \tag{C.10}$$

The global verification condition is defined as a threshold function:

$$\mathcal{V}_{\text{global}}(D_t, L_{t-1}) = \begin{cases} \text{true}, & \sum_{i=1}^{n} \mathbb{I}[\mathcal{V}_i(D_t, L_{t-1}) = \text{true}] \geq t \\ \text{false}, & \text{otherwise} \end{cases} \tag{C.11}$$

where t is the acceptance threshold and $\mathbb{I}$ is the indicator function.

C.5 Correctness of Threshold Verification

The threshold verification mechanism ensures that telemetry is accepted only if a sufficient number of independent validators agree on its validity.

Theorem C.1 (Threshold Correctness)**.** *If at least t validators are honest and correctly implement the verification function, then any invalid telemetry D'_t is rejected with overwhelming probability.*

Proof. Let $\mathcal{H}$ denote the set of honest validators and assume $|\mathcal{H}| \geq t$. For invalid telemetry D'_t, each honest validator evaluates:

$$\mathcal{V}_i(D'_t, L_{t-1}) = \text{false}$$

since D'_t does not satisfy the validity condition. Therefore:

$$\sum_{i=1}^{n} \mathbb{I}[\mathcal{V}_i(D'_t, L_{t-1}) = \text{true}] \leq n - |\mathcal{H}| < t$$

Thus, $\mathcal{V}_{\text{global}}(D'_t, L_{t-1}) = \text{false}$, and the telemetry is rejected. □

C.6 Byzantine Fault Model

The system operates under a Byzantine fault model in which up to f validators may be malicious. The threshold t must satisfy:

$$t > f \tag{C.12}$$

to ensure that malicious validators cannot unilaterally accept invalid telemetry.

In the presence of collusion, the adversary controls a subset $\mathcal{C} \subseteq \mathcal{V}$ with $|\mathcal{C}| = f$. The adversary succeeds only if:

$$f \geq t \tag{C.13}$$

which is prevented by the threshold condition.

C.7 Ingestion State Transition

The ingestion process updates the lineage state upon acceptance of telemetry. Let the system state be (S_t, L_{t-1}). Upon receiving D_t, the system performs:

$$\text{if } \mathcal{I}(D_t, L_{t-1}) = \text{accept then } L_t = \mathcal{H}(\mathcal{H}(\mathcal{S}(D_t)) \parallel L_{t-1}) \tag{C.14}$$

Otherwise, the state remains unchanged.

This defines a state transition function:

$$(S_t, L_{t-1}) \xrightarrow{D_t} (S_{t+1}, L_t) \tag{C.15}$$

only if validation succeeds.

C.8 Rejection as Default Behavior

The ingestion operator enforces a default rejection policy:

$$\mathcal{I}(D_t, L_{t-1}) = \text{reject} \quad \text{unless explicitly validated} \tag{C.16}$$

This ensures that any failure in verification results in rejection, eliminating ambiguity in decision-making.

C.9 Implications for Security

The zero-trust ingestion framework ensures that the acceptance of telemetry is determined solely by cryptographic and structural validity. By distributing verification across multiple independent validators and enforcing a threshold condition, the system eliminates reliance on any single entity and provides resilience against adversarial manipulation.

In summary, the ingestion process transforms the telemetry pipeline into a formally verified boundary that enforces the integrity invariants defined in the cryptographic core. This establishes a robust defense against injection, modification, replay, and collusion attacks, ensuring that only valid telemetry is admitted into the system.

C.10 Threshold Cryptography and Distributed Proof Aggregation

The zero-trust ingestion framework relies fundamentally on the ability to distribute trust across multiple independent validators while preserving a unified verification outcome. This is achieved through threshold cryptography, which enables a set of validators to collectively generate a single cryptographic proof without any individual validator possessing sufficient authority to do so independently. The resulting construction ensures that the acceptance of telemetry is contingent upon a quorum of validators, thereby providing resilience against Byzantine faults and adversarial compromise.

Let $\mathcal{V} = \{V_1, V_2, \ldots, V_n\}$ denote the set of validator nodes. Each validator V_i is associated with a private key share K_i derived from a master secret K through a distributed key generation process. The corresponding public key is denoted by PK.

C.10.1 Threshold Signature Model

A threshold signature scheme is defined by a tuple of algorithms:

$$(\text{KeyGen}, \text{Sign}_i, \text{Aggregate}, \text{Verify}) \tag{C.17}$$

where KeyGen generates key shares $(K_1, \ldots, K_n)$ and a public key PK, Sign_i produces a partial signature using K_i, Aggregate combines partial signatures into a single signature, and Verify validates the aggregated signature.

Given a message M_t, each validator computes a partial signature:

$$\sigma_i = \text{Sign}_i(M_t, K_i) \tag{C.18}$$

An aggregated signature is constructed as:

$$\Sigma_t = \text{Aggregate}(\sigma_{i_1}, \ldots, \sigma_{i_m}) \tag{C.19}$$

for a subset of validators $\{i_1, \ldots, i_m\}$ such that $m \geq t$, where t is the threshold.

The aggregated signature satisfies:

$$\text{Verify}(M_t, \Sigma_t, PK) = \text{true} \tag{C.20}$$

C.10.2 Application to Proof of Reflexion

In the context of the RCO protocol, the message M_t is defined as:

$$M_t = \mathcal{H}_S(S_t) \parallel L_t \tag{C.21}$$

Each validator independently verifies the validity of (S_t, L_t) and produces a partial signature:

$$\sigma_i = \text{Sign}_i(\mathcal{H}_S(S_t) \parallel L_t, K_i) \tag{C.22}$$

The aggregated proof is:

$$\Sigma_t = \text{Aggregate}(\sigma_{i_1}, \ldots, \sigma_{i_m}) \tag{C.23}$$

This construction ensures that the proof Σ_t is valid only if at least t validators agree on the validity of the telemetry.

C.10.3 Correctness Property

The threshold signature scheme satisfies correctness if:

$$\forall M_t, \quad \text{Verify}(M_t, \Sigma_t, PK) = \text{true} \tag{C.24}$$

whenever Σ_t is constructed from at least t valid partial signatures.

C.10.4 Unforgeability Under Threshold

Theorem C.2 (Threshold Unforgeability)**.** *Assuming that the underlying signature scheme is EUF-CMA secure, an adversary controlling fewer than t key shares cannot produce a valid aggregated signature for any message M_t not previously signed by at least t honest validators.*

Proof. Let $\mathcal{A}$ be an adversary controlling a subset $\mathcal{C} \subset \mathcal{V}$ with $|\mathcal{C}| = f < t$. The adversary can produce partial signatures $\{\sigma_i\}_{i \in \mathcal{C}}$ for any message M_t.

To produce a valid aggregated signature Σ_t, the adversary must obtain at least t valid partial signatures. Since $f < t$, the adversary must either:

$$\text{(i)} \quad \text{Forge at least one partial signature from an honest validator} \tag{C.25}$$

or:

$$\text{(ii)} \quad \text{Forge the aggregated signature directly} \tag{C.26}$$

The first case contradicts the EUF-CMA security of the signature scheme, as it requires forging a signature under an honest validator's key share. The second case also contradicts EUF-CMA security, as the aggregated signature is a valid signature under PK.

Therefore, the adversary cannot produce a valid Σ_t without controlling at least t key shares. □

C.10.5 Robustness Against Collusion

Let $\mathcal{C} \subseteq \mathcal{V}$ be a colluding subset of validators. The adversary succeeds only if:

$$|\mathcal{C}| \geq t \tag{C.27}$$

Otherwise, the adversary cannot produce a valid aggregated signature. This establishes that the system tolerates up to $f = t - 1$ Byzantine validators.

C.10.6 Soundness of Verification

The verification function ensures that only correctly aggregated signatures are accepted. Let Σ_t be an aggregated signature. Then:

$$\text{Verify}(M_t, \Sigma_t, PK) = \text{true} \tag{C.28}$$

if and only if Σ_t is constructed from at least t valid partial signatures.

Any deviation in the aggregation process results in:

$$\text{Verify}(M_t, \Sigma_t, PK) = \text{false} \tag{C.29}$$

with overwhelming probability.

C.10.7 Aggregation Linearity and Efficiency

In many threshold schemes, such as those based on pairing-based cryptography, the aggregation function is linear:

$$\Sigma_t = \sum_{i=1}^{m} \sigma_i \tag{C.30}$$

This property enables efficient aggregation and verification, as the cost of verification is independent of m.

C.10.8 Resistance to Replay Attacks

Each message M_t includes the lineage state L_t, which is unique for each time step. Therefore, even if an adversary reuses a previously valid signature Σ_k, it will not be valid for a different lineage state:

$$M_k \neq M_t \implies \text{Verify}(M_t, \Sigma_k, PK) = \text{false} \tag{C.31}$$

This prevents replay attacks.

C.10.9 Consistency Across Validators

Each validator independently computes:

$$M_t = \mathcal{H}_S(S_t) \parallel L_t \tag{C.32}$$

Deterministic serialization and lineage construction ensure that all honest validators compute identical M_t. Therefore:

$$\sigma_i = \text{Sign}_i(M_t, K_i) \tag{C.33}$$

are consistent across validators, enabling correct aggregation.

C.10.10 Integration with Ingestion Pipeline

The ingestion operator incorporates threshold verification as follows:

$$\mathcal{I}(D_t, L_{t-1}) = \begin{cases} \text{accept}, & \text{Verify}(M_t, \Sigma_t, PK) = \text{true} \\ \text{reject}, & \text{otherwise} \end{cases} \tag{C.34}$$

where $M_t = \mathcal{H}_S(S_t) \parallel L_t$.

C.10.11 Latency and Parallelism

Partial signatures may be generated in parallel across validators. Let T_{sign} denote the time to generate a partial signature and T_{agg} the time to aggregate. Then the total latency is:

$$T_{\text{total}} = \max_i T_{\text{sign}}^{(i)} + T_{\text{agg}} + T_{\text{verify}} \tag{C.35}$$

This enables scalable ingestion under high throughput.

C.10.12 Security Bound

The probability that an adversary successfully forges a valid aggregated signature is bounded by:

$$\Pr[\text{Forge}] \leq \epsilon_{\text{sig}} + \epsilon_{\text{coll}} \tag{C.36}$$

where ϵ_{sig} is the forgery probability of the signature scheme and ϵ_{coll} accounts for any hash collisions.

C.10.13 Conclusion

The threshold cryptographic framework provides a robust mechanism for distributing trust across multiple validators while maintaining a unified verification outcome. By requiring a quorum of validators to produce a valid proof, the system eliminates single points of failure and ensures resilience against Byzantine adversaries. When integrated with the Proof of Reflexion and lineage construction, this mechanism establishes a zero-trust ingestion pipeline in which telemetry is admitted only upon satisfying a collectively enforced integrity condition.

C.11 Ingestion Consistency, Ordering, and Deterministic Acceptance

The correctness of the zero-trust ingestion framework depends not only on the validity of individual telemetry elements but also on the preservation of a globally consistent ordering and deterministic acceptance behavior across all participating nodes. In distributed systems, inconsistencies in ordering or acceptance decisions can lead to divergence in system state, undermining the integrity guarantees established by the cryptographic core.

This section formalizes the constraints and mechanisms required to ensure that ingestion remains deterministic, ordered, and globally consistent under concurrent and adversarial conditions.

C.11.1 Temporal Ordering Model

Let $\{D_t\}_{t\in\mathbb{N}}$ denote a sequence of telemetry elements indexed by logical time t. The ingestion process must enforce a strict total order over accepted telemetry such that:

$$t_1 < t_2 \implies D_{t_1} \prec D_{t_2} \tag{C.37}$$

where $\prec$ denotes the ingestion order. This ordering is not derived from wall-clock time but from the lineage structure, which imposes a deterministic sequence.

Let L_t denote the lineage state after accepting D_t. Then:

$$L_t = \mathcal{H}(H_t \parallel L_{t-1}) \tag{C.38}$$

This recursive dependency enforces that D_t cannot be accepted unless L_{t-1} is known and verified, thereby establishing a strict sequential dependency.

C.11.2 Sequential Consistency Invariant

The ingestion process satisfies sequential consistency if all nodes observe the same order of accepted telemetry. Formally, let $\mathcal{N}$ denote the set of nodes, and let $\prec_i$ denote the ordering observed by node i. Then:

$$\forall i, j \in \mathcal{N}, \quad \prec_i = \prec_j \tag{C.39}$$

This invariant ensures that the lineage state is consistent across all nodes.

C.11.3 Deterministic Acceptance Function

Define the acceptance function:

$$\mathcal{A}(D_t, L_{t-1}) = \begin{cases} 1, & \text{if } \mathcal{I}(D_t, L_{t-1}) = \text{accept} \\ 0, & \text{otherwise} \end{cases} \tag{C.40}$$

The ingestion framework requires that $\mathcal{A}$ be deterministic, i.e., for any two nodes i and j:

$$\mathcal{A}_i(D_t, L_{t-1}) = \mathcal{A}_j(D_t, L_{t-1}) \tag{C.41}$$

This property follows from the determinism of serialization, hashing, lineage construction, and proof verification.

C.11.4 Concurrency Model

In practical systems, telemetry elements may be generated concurrently. Let $\{D_t^{(1)}, D_t^{(2)}, \dots\}$ denote concurrent telemetry candidates for the same logical position. The ingestion system must resolve such conflicts deterministically.

Define a selection function:

$$D_t^* = \arg\min_{D \in \mathcal{C}_t} \mathcal{O}(D) \tag{C.42}$$

where $\mathcal{C}_t$ is the set of candidate telemetry elements and $\mathcal{O}$ is a deterministic ordering function defined over canonical encodings:

$$\mathcal{O}(D) = \mathcal{S}(D) \tag{C.43}$$

interpreted lexicographically. This ensures that all nodes select the same D_t^*.

C.11.5 Deterministic Conflict Resolution

Theorem C.3 (Deterministic Conflict Resolution)**.** *Given a set of candidate telemetry elements $\mathcal{C}_t$, all honest nodes select the same element D_t^*.*

Proof. Since $\mathcal{S}$ is canonical, all nodes compute identical encodings for each $D \in \mathcal{C}_t$. The lexicographic ordering over $\mathcal{S}(D)$ is deterministic. Therefore, the minimization yields the same D_t^* across all nodes. □

C.11.6 Resistance to Reordering Attacks

An adversary may attempt to reorder telemetry elements to influence system behavior. Let π be a permutation such that:

$$D'_t = D_{\pi(t)} \tag{C.44}$$

The lineage construction enforces:

$$L'_t = \mathcal{H}(H'_t \parallel L'_{t-1}) \tag{C.45}$$

Since $H'_t \neq H_t$ for non-trivial permutations, it follows that:

$$L'_t \neq L_t \tag{C.46}$$

with overwhelming probability. Therefore, reordered sequences are rejected.

C.11.7 Gap Detection and Continuity Enforcement

The ingestion process enforces continuity of the sequence. Let t be the expected index. A telemetry element D_t is accepted only if:

$$L_{t-1} \text{ is known and valid} \tag{C.47}$$

If a gap occurs, i.e., D_t is received before D_{t-1}, then:

$$\mathcal{I}(D_t, L_{t-1}) = \text{reject} \tag{C.48}$$

This prevents omission attacks.

C.11.8 Replay Attack Prevention

Replay attacks involve reusing previously accepted telemetry. Let D_k be a previously accepted element. For $k < t$, attempting to replay D_k yields:

$$H_k = \mathcal{H}(\mathcal{S}(D_k)) \tag{C.49}$$

but the lineage context differs:

$$L_{t-1} \neq L_{k-1} \tag{C.50}$$

Therefore:

$$\mathcal{H}(H_k \parallel L_{t-1}) \neq L_k \tag{C.51}$$

and the replayed element is rejected.

C.11.9 Global Determinism Theorem

Theorem C.4 (Global Ingestion Determinism)**.** *Assuming all nodes receive the same set of telemetry inputs and implement the RCO ingestion protocol, they will produce identical lineage sequences* $\{L_t\}$*.*

Proof. The ingestion process consists of deterministic operations: canonical serialization, hashing, lineage construction, and threshold verification. Conflict resolution is also deterministic. Therefore, all nodes accept the same sequence of telemetry elements in the same order, resulting in identical lineage sequences. □

C.11.10 Consistency Under Network Delay

Let Δ denote network delay. Nodes may receive telemetry in different orders due to latency. However, since acceptance depends on lineage continuity, nodes buffer out-of-order inputs until L_{t-1} is available.

Define a buffer $\mathcal{B}$ such that:

$$\mathcal{B} = \{D_t \mid L_{t-1} \text{ not yet available}\} \tag{C.52}$$

Once L_{t-1} is established, buffered elements are processed deterministically.

C.11.11 Convergence Property

Theorem C.5 (Eventual Convergence)**.** *Under reliable communication, all honest nodes eventually converge to the same lineage state* L_t*.*

Proof. All valid telemetry is eventually received by all nodes. Deterministic processing ensures identical acceptance decisions. Therefore, all nodes compute the same lineage sequence. □

C.11.12 Integration with Threshold Verification

The acceptance condition incorporates threshold verification:

$$\mathcal{A}(D_t, L_{t-1}) = 1 \iff \text{Verify}(M_t, \Sigma_t, PK) = \text{true} \tag{C.53}$$

This ensures that only telemetry approved by a quorum of validators is considered for ordering.

C.11.13 Final Ordering Invariant

The ingestion process enforces the invariant:

$$\forall t, \quad L_t = \mathcal{H}(H_t \parallel L_{t-1}) \wedge \mathcal{A}(D_t, L_{t-1}) = 1 \tag{C.54}$$

which ensures that the sequence is ordered, continuous, and valid.

C.11.14 Conclusion

The ingestion consistency framework ensures that telemetry is admitted into the system in a deterministic and globally consistent manner. By enforcing strict ordering, deterministic conflict resolution, and continuity constraints, the RCO protocol eliminates ambiguity in sequence construction and ensures that all nodes maintain an identical view of system history. These properties are essential for preserving the integrity guarantees established by the cryptographic core and for enabling reproducible system behavior in distributed environments.

C.12 Unified Ingestion Integrity and End-to-End Security Composition

The preceding sections have established the structural, cryptographic, and distributed properties of the zero-trust ingestion framework. Deterministic serialization ensures canonical representation, cryptographic hashing provides collision-resistant compression, Merkle-lineage enforces temporal integrity, Proof of Reflexion binds telemetry to system state, and threshold cryptography distributes trust across validators. The ingestion layer integrates these components into a unified acceptance mechanism that determines whether telemetry is admitted into the system.

Let the ingestion pipeline be defined as:

$$\mathcal{I}_{\text{RCO}} = \mathcal{V}_{\text{threshold}} \circ \mathcal{V}_{\text{PoR}} \circ \mathcal{C} \circ \mathcal{H} \circ \mathcal{S} \tag{C.55}$$

This pipeline defines a mapping:

$$\mathcal{I}_{\text{RCO}} : \mathcal{D} \times \mathcal{L} \to \{\text{accept}, \text{reject}\} \tag{C.56}$$

such that a telemetry element D_t is accepted if and only if it satisfies all integrity conditions.

C.12.1 Unified Acceptance Condition

The ingestion operator accepts D_t if and only if:

$$\mathcal{I}_{\text{RCO}}(D_t, L_{t-1}) = \text{accept} \iff \begin{cases} \mathcal{S}(D_t) \text{ is canonical} \\ H_t = \mathcal{H}(\mathcal{S}(D_t)) \\ L_t = \mathcal{H}(H_t \parallel L_{t-1}) \\ M_t = \mathcal{H}_S(S_t) \parallel L_t \\ \Sigma_t = \text{Aggregate}(\{\sigma_i\}_{i=1}^m) \\ \text{Verify}(M_t, \Sigma_t, PK) = \text{true} \end{cases} \tag{C.57}$$

This condition ensures that the telemetry is deterministically represented, structurally consistent, temporally valid, state-bound, and collectively verified.

C.12.2 End-to-End Integrity Theorem

Theorem C.6 (End-to-End Ingestion Integrity)**.** *Assuming that:*

1. *The serialization function $\mathcal{S}$ is canonical,*
2. *The hash functions $\mathcal{H}$ and $\mathcal{H}_S$ are collision-resistant,*
3. *The signature scheme is EUF-CMA secure,*
4. *Fewer than t validators are Byzantine,*

then for any telemetry element D'_t such that $D'_t \not\equiv D_t$, it holds that:

$$\Pr[\mathcal{I}_{RCO}(D'_t, L_{t-1}) = accept] \leq \epsilon$$

where ϵ is negligible.

Proof. Let D'_t be an invalid telemetry element. For $\mathcal{I}_{\text{RCO}}(D'_t, L_{t-1})$ to return accept, all conditions in the unified acceptance condition must be satisfied.

First, if $\mathcal{S}$ is canonical, then $D'_t \not\equiv D_t$ implies:

$$\mathcal{S}(D'_t) \neq \mathcal{S}(D_t)$$

which implies:

$$\mathcal{H}(\mathcal{S}(D'_t)) \neq \mathcal{H}(\mathcal{S}(D_t))$$

except with probability $\epsilon_{\mathcal{H}}$.

Second, the lineage condition requires:

$$L'_t = \mathcal{H}(H'_t \parallel L_{t-1})$$

If $H'_t \neq H_t$, then $L'_t \neq L_t$ except with negligible probability due to collision resistance.

Third, the Proof of Reflexion requires a valid signature:

$$\Sigma'_t = \text{Aggregate}(\{\sigma'_i\})$$

for the message $M'_t = \mathcal{H}_S(S'_t) \parallel L'_t$. Since fewer than t validators are Byzantine, the adversary cannot produce t valid partial signatures without either forging signatures or inducing collisions in $\mathcal{H}_S$.

Thus, the adversary must succeed in at least one of the following:

$$\text{(i)} \quad \text{Break canonical determinism} \tag{C.58}$$

$$\text{(ii)} \quad \text{Find a collision in } \mathcal{H} \tag{C.59}$$

$$\text{(iii)} \quad \text{Find a collision in } \mathcal{H}_S \tag{C.60}$$

$$\text{(iv)} \quad \text{Forge } t \text{ valid partial signatures} \tag{C.61}$$

Each of these events has negligible probability. Therefore, the probability that $\mathcal{I}_{\text{RCO}}(D'_t, L_{t-1}) =$ accept is negligible. □

C.12.3 Soundness and Completeness

The ingestion pipeline satisfies both soundness and completeness:

$$\text{Soundness: } \mathcal{I}_{\text{RCO}}(D'_t, L_{t-1}) = \text{accept} \implies D'_t \equiv D_t \tag{C.62}$$

$$\text{Completeness: } D_t \text{ valid} \implies \mathcal{I}_{\text{RCO}}(D_t, L_{t-1}) = \text{accept} \tag{C.63}$$

Soundness ensures that invalid telemetry is never accepted, while completeness ensures that valid telemetry is not rejected.

C.12.4 Elimination of Ingestion-Level Perturbations

Let δ_t^{ingest} denote perturbations introduced at the ingestion layer. Under the RCO protocol:

$$\delta_t^{\text{ingest}} = 0 \quad \forall t \tag{C.64}$$

as any deviation results in rejection. Substituting into the system dynamics:

$$\Delta_{t+1} \leq L_s \Delta_t \tag{C.65}$$

ensures that ingestion does not introduce instability.

C.12.5 Compositional Security

The ingestion layer composes with the cryptographic core to yield end-to-end security. Let $\mathcal{P}_{\text{core}}$ denote the cryptographic core pipeline. Then:

$$\mathcal{P}_{\text{system}} = \mathcal{P}_{\text{core}} \circ \mathcal{I}_{\text{RCO}} \tag{C.66}$$

satisfies:

$$\forall D_t, \quad \mathcal{P}_{\text{system}}(D_t) = \text{true} \implies \text{Valid}(D_t) \tag{C.67}$$

This establishes that the entire system preserves integrity from ingestion to storage.

C.12.6 Global Ingestion Invariant

The ingestion process enforces the invariant:

$$\forall t, \quad \mathcal{I}_{\text{RCO}}(D_t, L_{t-1}) = \text{accept} \implies \begin{cases} D_t \text{ is authentic} \\ D_t \text{ is state-consistent} \\ D_t \text{ is temporally valid} \\ D_t \text{ is uniquely ordered} \end{cases} \tag{C.68}$$

C.12.7 Final Security Bound

The overall probability of system compromise is bounded by:

$$\Pr[\text{Compromise}] \leq \epsilon_{\mathcal{S}} + \epsilon_{\mathcal{H}} + \epsilon_{\mathcal{H}_S} + \epsilon_{\text{sig}} + \epsilon_{\text{threshold}} \tag{C.69}$$

where each term is negligible.

C.12.8 Conclusion

The zero-trust ingestion framework completes the integrity architecture of the RCO protocol by ensuring that only valid telemetry is admitted into the system. Through the integration of deterministic encoding, cryptographic lineage, Proof of Reflexion, and threshold verification, the ingestion process enforces a comprehensive set of invariants that eliminate ambiguity, prevent adversarial manipulation, and ensure global consistency.

This establishes a formally verified boundary condition for the system, guaranteeing that all accepted telemetry satisfies the integrity requirements necessary for stable and reproducible computation.

D. Formal Security Calculus

D.1 Overview

The purpose of this section is to establish a rigorous security framework for the Feedback-Coupled Cryptographic Observation (RCO) protocol by formalizing the adversarial capabilities, defining security games, and proving that the protocol satisfies strong integrity and unforgeability guarantees under standard cryptographic assumptions. Unlike conventional systems in which telemetry integrity is treated as an auxiliary concern, the RCO protocol treats telemetry as a causative variable within system dynamics, necessitating a security model that accounts for both structural and semantic integrity.

Let $\mathcal{D}$ denote the space of telemetry inputs, $\mathcal{S}$ the space of system states, and $\mathcal{L}$ the lineage space. The system operates over sequences:

$$\{(D_t, S_t, L_t)\}_{t=0}^{T} \tag{D.1}$$

The objective of the adversary is to produce a sequence $\{D'_t\}$ such that:

$$\mathcal{I}_{\text{RCO}}(D'_t, L_{t-1}) = \text{accept} \quad \text{and} \quad D'_t \not\equiv D_t \tag{D.2}$$

This constitutes a successful integrity violation.

D.2 Adversarial Model

We define an adversary $\mathcal{A}$ as a probabilistic polynomial-time (PPT) algorithm with access to the following capabilities:

$$\mathcal{A} = (\mathcal{A}_{\text{oracle}}, \mathcal{A}_{\text{network}}, \mathcal{A}_{\text{corrupt}}) \tag{D.3}$$

The oracle capability allows $\mathcal{A}$ to query the system for valid telemetry and corresponding proofs. The network capability allows interception, modification, and replay of messages. The corruption capability allows control over a subset of validator nodes.

Let $\mathcal{C} \subseteq \mathcal{V}$ denote the set of corrupted validators with $|\mathcal{C}| = f$.

D.3 Security Experiment

We define a security experiment $\mathrm{Exp}_{\mathcal{A}}^{\mathrm{RCO}}(\lambda)$ parameterized by a security parameter λ.

D.3.1 Setup Phase

The challenger performs the following:

1. Generate key shares $(K_1, \ldots, K_n)$ and public key PK.
2. Initialize lineage state L_0.
3. Initialize system state S_0.

D.3.2 Query Phase

The adversary may issue the following queries:

Telemetry Query

$$\mathcal{A} \to D_t \tag{D.4}$$

The challenger returns:

$$(H_t, L_t, \Sigma_t) \tag{D.5}$$

where:

$$H_t = \mathcal{H}(\mathcal{S}(D_t)), \quad L_t = \mathcal{H}(H_t \parallel L_{t-1}) \tag{D.6}$$

and Σ_t is the aggregated PoR proof.

Corruption Query The adversary may corrupt up to f validators:

$$\mathcal{A} \to V_i \tag{D.7}$$

and obtains K_i.

Replay Query The adversary may request previously generated telemetry tuples.

D.3.3 Forgery Phase

The adversary outputs $(D'_t, L_{t-1}, \Sigma'_t)$.

D.3.4 Winning Condition

The adversary wins if:

$$\text{Verify}(M'_t, \Sigma'_t, PK) = \text{true} \quad \text{and} \quad D'_t \not\equiv D_t \tag{D.8}$$

where:

$$M'_t = \mathcal{H}_S(S'_t) \parallel L'_t \tag{D.9}$$

D.4 Security Definition

The RCO protocol is secure if for all PPT adversaries $\mathcal{A}$:

$$\Pr[\text{Exp}^{\text{RCO}}_{\mathcal{A}}(\lambda) = 1] \leq \text{negl}(\lambda) \tag{D.10}$$

D.5 Attack Taxonomy

We classify attacks into the following categories:

D.5.1 Injection Attacks

The adversary attempts to introduce arbitrary telemetry:

$$D'_t \in \mathcal{D} \tag{D.11}$$

without corresponding valid state or lineage.

D.5.2 Modification Attacks

The adversary modifies valid telemetry:

$$D'_t = D_t + \delta_t \tag{D.12}$$

D.5.3 Replay Attacks

The adversary reuses previously valid telemetry:

$$D'_t = D_k, \quad k < t \tag{D.13}$$

D.5.4 Reordering Attacks

The adversary permutes the sequence:

$$D'_t = D_{\pi(t)} \tag{D.14}$$

D.5.5 State Substitution Attacks

The adversary attempts to bind telemetry to an incorrect state:

$$S'_t \neq S_t \tag{D.15}$$

D.6 Security Goals

The protocol aims to achieve the following properties:

D.6.1 Integrity

$$\forall D'_t, \quad \text{accept}(D'_t) \implies D'_t \equiv D_t \tag{D.16}$$

D.6.2 Authenticity

$$\text{accept}(D'_t) \implies D'_t \text{ originates from a valid system state} \tag{D.17}$$

D.6.3 Non-Repudiation

$$\exists \Sigma_t \text{ such that } \text{Verify}(M_t, \Sigma_t, PK) = \text{true} \tag{D.18}$$

D.6.4 Forward Integrity

$$D'_k \neq D_k \implies L'_t \neq L_t \tag{D.19}$$

D.7 Reduction Framework

The security of the RCO protocol is reduced to the security of its underlying primitives. Let:

$$\epsilon_{\mathcal{H}}, \epsilon_{\mathcal{H}_S}, \epsilon_{\text{sig}} \tag{D.20}$$

denote the probabilities of breaking the hash function, state hash, and signature scheme, respectively.

Then:

$$\Pr[\text{Break RCO}] \leq \epsilon_{\mathcal{H}} + \epsilon_{\mathcal{H}_S} + \epsilon_{\text{sig}} + \epsilon_{\text{threshold}} \tag{D.21}$$

D.8 Game-Based Security Interpretation

The security experiment may be interpreted as a game between the challenger and the adversary. The adversary's advantage is defined as:

$$\text{Adv}_{\mathcal{A}}^{\text{RCO}}(\lambda) = \Pr[\text{Exp}_{\mathcal{A}}^{\text{RCO}}(\lambda) = 1] \tag{D.22}$$

The protocol is secure if:

$$\text{Adv}_{\mathcal{A}}^{\text{RCO}}(\lambda) \leq \text{negl}(\lambda) \tag{D.23}$$

D.9 Implications

This formalization establishes that any successful attack against the RCO protocol implies a break in at least one underlying cryptographic primitive or a violation of deterministic serialization. Therefore, the security of the system is tightly coupled to well-established hardness assumptions, providing strong assurance of its robustness.

D.10 Conclusion

The adversarial framework defined in this section provides a rigorous foundation for analyzing the security of the RCO protocol. By modeling the capabilities of the adversary and defining precise security goals, we establish a basis for proving that the protocol achieves strong integrity and authenticity guarantees under realistic threat models.

D.11 Replay, Reordering, and Injection Resistance

The security of the RCO protocol against replay, reordering, and injection attacks follows from the combined properties of deterministic serialization, cryptographic hashing, lineage construction, and threshold-verified Proof of Reflexion. In this section, these resistance properties are formalized through adversarial games and probability bounds, demonstrating that any successful attack implies a break in underlying cryptographic assumptions.

D.11.1 Replay Attack Resistance

A replay attack is defined as an adversarial attempt to reuse previously valid telemetry D_k at a later time $t > k$ in order to induce incorrect system behavior.

Replay Game Definition

Define the replay experiment $\mathrm{Exp}_{\mathcal{A}}^{\mathrm{replay}}$:

1. The adversary obtains a valid tuple (D_k, L_k, Σ_k).
2. The adversary constructs a replay attempt:

$$D'_t = D_k \tag{D.24}$$

3. The adversary submits $(D'_t, L_{t-1}, \Sigma_k)$.

The adversary succeeds if:

$$\mathcal{I}_{\mathrm{RCO}}(D'_t, L_{t-1}) = \mathrm{accept} \tag{D.25}$$

Replay Resistance Theorem

Theorem D.1 (Replay Resistance)**.** *Under the RCO protocol, the probability that a replayed telemetry element is accepted is negligible:*

$$\Pr[Exp_{\mathcal{A}}^{replay} = 1] \leq \epsilon_{\mathcal{H}} + \epsilon_{sig}$$

Proof. For acceptance, the adversary must satisfy:

$$L'_t = \mathcal{H}(H_k \parallel L_{t-1}) \tag{D.26}$$

However, the original lineage value satisfies:

$$L_k = \mathcal{H}(H_k \parallel L_{k-1}) \tag{D.27}$$

Since $L_{t-1} \neq L_{k-1}$ for $t > k$, it follows that:

$$\mathcal{H}(H_k \parallel L_{t-1}) \neq \mathcal{H}(H_k \parallel L_{k-1}) \tag{D.28}$$

except with probability $\epsilon_{\mathcal{H}}$ due to collision resistance.

Additionally, the proof Σ_k is bound to the message:

$$M_k = \mathcal{H}_S(S_k) \parallel L_k \tag{D.29}$$

For replay, the required message is:

$$M'_t = \mathcal{H}_S(S'_t) \parallel L'_t \tag{D.30}$$

Since $L'_t \neq L_k$, Σ_k is invalid for M'_t. Therefore, the adversary must forge a new signature, which succeeds with probability at most ϵ_{sig}.

Thus:

$$\Pr[\text{Replay Success}] \leq \epsilon_{\mathcal{H}} + \epsilon_{\text{sig}}$$

□

D.11.2 Reordering Attack Resistance

A reordering attack attempts to permute the sequence of telemetry elements to alter system behavior.

Reordering Game Definition

Let π be a permutation over indices $\{0, \ldots, t\}$. The adversary constructs:

$$D'_t = D_{\pi(t)} \tag{D.31}$$

The adversary wins if:

$$\mathcal{I}_{\text{RCO}}(D'_t, L_{t-1}) = \text{accept} \tag{D.32}$$

Reordering Resistance Theorem

Theorem D.2 (Reordering Resistance)**.** *Any non-trivial permutation π results in rejection with overwhelming probability:*

$$\Pr[Exp_{\mathcal{A}}^{reorder} = 1] \leq \epsilon_{\mathcal{H}} + \epsilon_{sig}$$

Proof. Let $H_t = \mathcal{H}(\mathcal{S}(D_t))$ and $H'_t = \mathcal{H}(\mathcal{S}(D_{\pi(t)}))$.

If $\pi(t) \neq t$, then:

$$H'_t \neq H_t \tag{D.33}$$

with probability $1 - \epsilon_{\mathcal{H}}$.

The lineage requires:

$$L_t = \mathcal{H}(H_t \parallel L_{t-1}) \tag{D.34}$$

but the adversary computes:

$$L'_t = \mathcal{H}(H'_t \parallel L_{t-1}) \tag{D.35}$$

Thus:

$$L'_t \neq L_t \tag{D.36}$$

with overwhelming probability.

Additionally, any attempt to generate a valid Σ'_t for L'_t requires forging a signature or obtaining threshold approval, which is infeasible under assumptions.

Therefore, the adversary fails except with negligible probability. □

D.11.3 Injection Attack Resistance

Injection attacks involve generating arbitrary telemetry D'_t without corresponding legitimate state or lineage.

Injection Game Definition

The adversary outputs $(D'_t, L_{t-1}, \Sigma'_t)$ without querying the oracle for D'_t.

The adversary succeeds if:

$$\mathcal{I}_{\text{RCO}}(D'_t, L_{t-1}) = \text{accept} \tag{D.37}$$

Injection Resistance Theorem

Theorem D.3 (Injection Resistance)**.** *The probability that an adversary successfully injects invalid telemetry is bounded by:*

$$\Pr[Exp_{\mathcal{A}}^{inject} = 1] \leq \epsilon_{\mathcal{H}} + \epsilon_{\mathcal{H}_S} + \epsilon_{sig} + \epsilon_{threshold}$$

Proof. For D'_t to be accepted, all of the following must hold:

(i) Canonical Consistency If $D'_t \not\equiv D_t$, then:

$$\mathcal{S}(D'_t) \neq \mathcal{S}(D_t) \tag{D.38}$$

implying:

$$\mathcal{H}(\mathcal{S}(D'_t)) \neq \mathcal{H}(\mathcal{S}(D_t)) \tag{D.39}$$

except with probability $\epsilon_{\mathcal{H}}$.

(ii) Lineage Consistency The adversary must produce:

$$L'_t = \mathcal{H}(H'_t \parallel L_{t-1}) \tag{D.40}$$

This is feasible only if H'_t is correctly computed, but this does not ensure validity.

(iii) State Binding The adversary must produce a valid state S'_t such that:

$$M'_t = \mathcal{H}_S(S'_t) \parallel L'_t \tag{D.41}$$

matches the signed message. Any mismatch implies either a collision in $\mathcal{H}_S$ or inconsistency.

(iv) Threshold Signature The adversary must produce Σ'_t such that:

$$\text{Verify}(M'_t, \Sigma'_t, PK) = \text{true} \tag{D.42}$$

This requires either:

$$\text{(a)} \quad \text{Forging } t \text{ partial signatures} \tag{D.43}$$

or:

$$\text{(b)} \quad \text{Compromising } t \text{ validators} \tag{D.44}$$

Both events occur with probability at most ϵ_{sig} and $\epsilon_{\text{threshold}}$, respectively.

Combining all cases:

$$\Pr[\text{Injection Success}] \leq \epsilon_{\mathcal{H}} + \epsilon_{\mathcal{H}_S} + \epsilon_{\text{sig}} + \epsilon_{\text{threshold}}$$

□

D.11.4 Combined Attack Bound

For a composite adversary attempting replay, reordering, and injection simultaneously, the success probability is bounded by:

$$\Pr[\text{Attack}] \leq 2\epsilon_{\mathcal{H}} + \epsilon_{\mathcal{H}_S} + 2\epsilon_{\text{sig}} + \epsilon_{\text{threshold}} \tag{D.45}$$

which remains negligible.

D.11.5 Multi-Step Adversarial Strategy Analysis

Consider an adaptive adversary that performs a sequence of operations:

$$\mathcal{A} = \mathcal{A}_k \circ \cdots \circ \mathcal{A}_1 \tag{D.46}$$

Each step attempts to manipulate telemetry. The success probability over k steps satisfies:

$$\Pr[\text{Success}] \leq k \cdot (\epsilon_{\mathcal{H}} + \epsilon_{\mathcal{H}_S} + \epsilon_{\text{sig}} + \epsilon_{\text{threshold}}) \tag{D.47}$$

which remains negligible for polynomial k.

D.11.6 Conclusion

The RCO protocol provides strong resistance against replay, reordering, and injection attacks by combining deterministic representation, cryptographic lineage, state binding, and threshold verification. Each class of attack is reduced to breaking well-established cryptographic assumptions, ensuring that the probability of successful attack remains negligible under standard security models.

D.12 State Substitution, Adaptive Adversaries, and Chosen-Message Attacks

The integrity guarantees of the RCO protocol must hold not only against static adversaries but also against adaptive adversaries capable of interacting with the system, observing outputs, and dynamically adjusting their strategies. In particular, the Proof of Reflexion (PoR) must remain secure under chosen-message attacks and state substitution attempts, wherein the adversary attempts to bind valid lineage to an incorrect or fabricated system state.

This section formalizes these adversarial capabilities and proves that the RCO protocol remains secure under such conditions.

D.12.1 State Substitution Attack Model

A state substitution attack is defined as an attempt by the adversary to produce a valid telemetry proof for a falsified system state. Formally, the adversary attempts to construct:

$$(S'_t, L_t, \Sigma'_t) \tag{D.48}$$

such that:

$$\text{Verify}(\mathcal{H}_S(S'_t) \parallel L_t, \Sigma'_t, PK) = \text{true} \tag{D.49}$$

while:

$$S'_t \neq S_t \tag{D.50}$$

This attack targets the semantic integrity enforced by the PoR mechanism.

D.12.2 PoR Security Game

We define the PoR security experiment $\text{Exp}_{\mathcal{A}}^{\text{PoR}}$ as follows:

1. The challenger generates keys $(K_1, \ldots, K_n)$ and public key PK.
2. The adversary is given access to a signing oracle:

$$\mathcal{O}_{\text{sign}}(S, L) \to \Sigma = \text{Aggregate}(\{\sigma_i\}) \tag{D.51}$$

3. The adversary adaptively queries the oracle with pairs (S_i, L_i).
4. The adversary outputs (S'_t, L_t, Σ'_t).

The adversary wins if:

$$\text{Verify}(\mathcal{H}_S(S'_t) \parallel L_t, \Sigma'_t, PK) = \text{true} \tag{D.52}$$

and (S'_t, L_t) was not queried to the oracle.

D.12.3 PoR Unforgeability Under Chosen-Message Attack

Theorem D.4 (PoR EUF-CMA Security)**.** *Assuming the underlying signature scheme is EUF-CMA secure and $\mathcal{H}_S$ is collision-resistant, the adversary's advantage in $Exp_{\mathcal{A}}^{PoR}$ is negligible:*

$$Adv_{\mathcal{A}}^{PoR}(\lambda) \leq \epsilon_{sig} + \epsilon_{\mathcal{H}_S}$$

Proof. Suppose $\mathcal{A}$ produces (S'_t, L_t, Σ'_t) such that:

$$\text{Verify}(\mathcal{H}_S(S'_t) \parallel L_t, \Sigma'_t, PK) = \text{true}$$

and (S'_t, L_t) was not queried.

Then Σ'_t is a valid signature on a new message:

$$M'_t = \mathcal{H}_S(S'_t) \parallel L_t$$

This implies that $\mathcal{A}$ has either:

$$\text{(i)} \quad \text{forged a signature under EUF-CMA} \tag{D.53}$$

or:

$$\text{(ii)} \quad \text{found } S_t \neq S'_t \text{ such that } \mathcal{H}_S(S_t) = \mathcal{H}_S(S'_t) \tag{D.54}$$

The first case occurs with probability ϵ_{sig}, and the second with probability $\epsilon_{\mathcal{H}_S}$.

Thus:

$$\text{Adv}_{\mathcal{A}}^{\text{PoR}} \leq \epsilon_{\text{sig}} + \epsilon_{\mathcal{H}_S}$$

□

D.12.4 Adaptive Adversary Model

An adaptive adversary $\mathcal{A}$ can:

1. Observe prior telemetry $(D_1, \ldots, D_{t-1})$,
2. Query signing oracles on chosen states,
3. Corrupt validators dynamically,
4. Select attack targets based on observed outputs.

Formally, the adversary operates as:

$$\mathcal{A} = (\mathcal{A}_1 \to \mathcal{A}_2 \to \cdots \to \mathcal{A}_k) \tag{D.55}$$

where each stage depends on previous outputs.

D.12.5 Adaptive Corruption Model

Let $\mathcal{C}_t$ denote the set of corrupted validators at time t. The adversary may expand $\mathcal{C}_t$ over time:

$$\mathcal{C}_1 \subseteq \mathcal{C}_2 \subseteq \cdots \subseteq \mathcal{C}_t \tag{D.56}$$

subject to:

$$|\mathcal{C}_t| < t \tag{D.57}$$

to remain below the threshold.

D.12.6 Security Under Adaptive Corruption

Theorem D.5 (Adaptive Threshold Security)**.** *If at all times $|\mathcal{C}_t| < t$, then the adversary cannot produce a valid aggregated signature for any new message.*

Proof. At time t, the adversary controls $f < t$ validators. To produce a valid aggregated signature, the adversary must obtain at least t valid partial signatures.

Since honest validators sign only valid messages, the adversary must either:

$$\text{(i)} \quad \text{convince honest validators to sign invalid messages} \tag{D.58}$$

or:

$$\text{(ii)} \quad \text{forge signatures for missing shares} \tag{D.59}$$

The first case is prevented by deterministic validation rules. The second contradicts EUF-CMA security.

Thus, the adversary fails. □

D.12.7 Chosen-Message Attack on Lineage

An adversary may attempt to construct a lineage collision under chosen inputs. Let the adversary select D'_t such that:

$$\mathcal{H}(\mathcal{S}(D'_t)) = \mathcal{H}(\mathcal{S}(D_t)) \tag{D.60}$$

This is equivalent to finding a collision in $\mathcal{H}$, which succeeds with probability $\epsilon_{\mathcal{H}}$.

D.12.8 Hybrid Argument for Full Security

We now construct a hybrid argument to prove full system security.

Hybrid 0: Real System The adversary interacts with the full RCO system.

Hybrid 1: Replace $\mathcal{H}$ with Random Oracle The adversary's advantage changes by at most $\epsilon_{\mathcal{H}}$.

Hybrid 2: Replace $\mathcal{H}_S$ with Random Oracle Advantage changes by at most $\epsilon_{\mathcal{H}_S}$.

Hybrid 3: Replace Signature Scheme with Ideal Function Advantage changes by at most ϵ_{sig}.

Conclusion In the ideal system, the adversary has no advantage. Therefore:

$$\text{Adv}_{\mathcal{A}}^{\text{RCO}} \leq \epsilon_{\mathcal{H}} + \epsilon_{\mathcal{H}_S} + \epsilon_{\text{sig}} + \epsilon_{\text{threshold}} \tag{D.61}$$

D.12.9 Multi-Round Adaptive Attack Bound

For an adversary performing q adaptive queries:

$$\text{Adv}_{\mathcal{A}}^{(q)} \leq q \cdot (\epsilon_{\mathcal{H}} + \epsilon_{\mathcal{H}_S} + \epsilon_{\text{sig}} + \epsilon_{\text{threshold}}) \tag{D.62}$$

which remains negligible for polynomial q.

D.12.10 State Consistency Guarantee

The PoR mechanism ensures:

$$\text{accept}(D'_t) \implies \exists S_t \text{ such that } D'_t = \mathcal{G}(S_t) \tag{D.63}$$

Thus, state substitution is prevented.

D.12.11 Conclusion

The RCO protocol remains secure under strong adversarial models including adaptive corruption, chosen-message attacks, and state substitution attempts. The Proof of Reflexion, combined with threshold cryptography and deterministic lineage construction, ensures that any successful attack implies a break in underlying cryptographic primitives, thereby providing strong assurance of system integrity.

D.13 System-Level Security: Composability, Liveness, and Failure Modes

The RCO protocol operates as a composition of multiple cryptographic and distributed components, each of which enforces a specific integrity invariant. While previous sections establish the security of individual components, system-level security requires that these guarantees are preserved under composition, concurrent execution, and partial system

failures. This section formalizes the composability of the protocol, analyzes the trade-offs between safety and liveness, and defines the boundaries under which the system maintains its security guarantees.

D.13.1 Composability Model

Let $\mathcal{P}_{\text{core}}$ denote the cryptographic core pipeline and $\mathcal{I}_{\text{RCO}}$ the ingestion pipeline. The full system is defined as:

$$\mathcal{P}_{\text{system}} = \mathcal{P}_{\text{core}} \circ \mathcal{I}_{\text{RCO}} \tag{D.64}$$

Each component enforces a local invariant:

$$\mathcal{P}_{\text{core}} : \text{Valid}_{\text{core}}(D_t) \tag{D.65}$$

$$\mathcal{I}_{\text{RCO}} : \text{Valid}_{\text{ingest}}(D_t) \tag{D.66}$$

The composed system enforces:

$$\text{Valid}_{\text{system}}(D_t) = \text{Valid}_{\text{core}}(D_t) \wedge \text{Valid}_{\text{ingest}}(D_t) \tag{D.67}$$

D.13.2 Universal Composability Property

Theorem D.6 (Composability of RCO)**.** *If each component of the RCO protocol satisfies its respective security definition under standard cryptographic assumptions, then the composed system $\mathcal{P}_{system}$ is secure.*

Proof. Assume an adversary $\mathcal{A}$ breaks $\mathcal{P}_{\text{system}}$. Then $\mathcal{A}$ must violate either:

$$\text{Valid}_{\text{core}}(D_t) \quad \text{or} \quad \text{Valid}_{\text{ingest}}(D_t) \tag{D.68}$$

If $\text{Valid}_{\text{core}}(D_t)$ is violated, then $\mathcal{A}$ breaks one of the cryptographic primitives: $\mathcal{S}$, $\mathcal{H}$, $\mathcal{H}_S$, or the signature scheme.

If $\text{Valid}_{\text{ingest}}(D_t)$ is violated, then $\mathcal{A}$ breaks threshold security or deterministic verification.

Thus, any successful attack reduces to breaking a component, which occurs with negligible probability. Therefore, the composed system is secure. □

D.13.3 Sequential Composition Under Concurrency

Let multiple ingestion pipelines operate concurrently:

$$\mathcal{I}_1, \mathcal{I}_2, \ldots, \mathcal{I}_m \tag{D.69}$$

Each produces candidate telemetry sequences. The global system enforces a deterministic merge:

$$D_t = \min_{\prec}\{D_t^{(1)}, \ldots, D_t^{(m)}\} \tag{D.70}$$

Since $\prec$ is derived from canonical serialization, all nodes agree on the selected element. Therefore, concurrency does not introduce divergence.

D.13.4 Safety and Liveness Definitions

We define two fundamental system properties:

Safety

$$\text{No invalid telemetry is ever accepted} \tag{D.71}$$

Formally:

$$\forall t, \quad \mathcal{I}_{\text{RCO}}(D_t, L_{t-1}) = \text{accept} \implies D_t \text{ is valid} \tag{D.72}$$

Liveness

$$\text{All valid telemetry is eventually accepted} \tag{D.73}$$

Formally:

$$\forall D_t \text{ valid}, \quad \exists t' \geq t \text{ such that } \mathcal{I}_{\text{RCO}}(D_t, L_{t'-1}) = \text{accept} \tag{D.74}$$

D.13.5 Safety-Liveness Tradeoff

The RCO protocol prioritizes safety over liveness. In particular:

$$\text{If verification is inconclusive, then } \mathcal{I}_{\text{RCO}} = \text{reject} \tag{D.75}$$

This ensures that:

$$\text{Safety is absolute} \quad \text{and} \quad \text{Liveness is conditional} \tag{D.76}$$

D.13.6 Liveness Under Honest Majority

Theorem D.7 (Liveness Guarantee)**.** *If at least t validators are honest and the network is eventually synchronous, then all valid telemetry is eventually accepted.*

Proof. Honest validators produce valid partial signatures for valid telemetry. Once t such signatures are available, aggregation yields a valid Σ_t. Eventual synchrony ensures that these signatures are propagated to all nodes. Therefore, $\mathcal{I}_{\text{RCO}}$ returns accept. □

D.13.7 Failure Modes

We now characterize the conditions under which the system may fail.

Threshold Failure

If:

$$|\mathcal{C}| \geq t \tag{D.77}$$

then adversaries can produce valid signatures for arbitrary messages, violating integrity.

Cryptographic Break

If any primitive is broken:

$$\epsilon_{\mathcal{H}} \approx 1 \quad \text{or} \quad \epsilon_{\text{sig}} \approx 1 \tag{D.78}$$

then security collapses.

Serialization Failure

If $\mathcal{S}$ is not canonical:

$$D_1 \equiv D_2 \not\Longrightarrow \mathcal{S}(D_1) = \mathcal{S}(D_2) \tag{D.79}$$

then deterministic guarantees fail.

Network Partition

In a partitioned network, different subsets of validators may produce conflicting partial signatures. However, without reaching threshold t, no conflicting telemetry is accepted, preserving safety at the cost of liveness.

D.13.8 Byzantine Resilience Boundary

The system tolerates up to:

$$f < t \tag{D.80}$$

Byzantine validators. Beyond this threshold, the adversary can control acceptance.

D.13.9 Graceful Degradation

Under partial failure, the system degrades gracefully:

$$\text{Failure} \implies \text{reduced liveness, not reduced safety} \tag{D.81}$$

D.13.10 End-to-End System Guarantee

Theorem D.8 (System Integrity)**.** *Under the assumptions:*

1. $f < t$,
2. $\mathcal{H}, \mathcal{H}_S$ *are collision-resistant,*
3. *the signature scheme is EUF-CMA secure,*
4. $\mathcal{S}$ *is canonical,*

the RCO system satisfies:

$$\forall t, \quad \mathit{accept}(D_t) \implies D_t \text{ is valid}$$

Proof. Follows from composability of core and ingestion guarantees, and threshold security. □

D.13.11 Compositional Stability

Let the system operate over T steps. The cumulative failure probability satisfies:

$$\Pr[\text{Failure over } T] \leq T \cdot (\epsilon_{\mathcal{H}} + \epsilon_{\mathcal{H}_S} + \epsilon_{\text{sig}} + \epsilon_{\text{threshold}}) \tag{D.82}$$

which remains negligible for polynomial T.

D.13.12 Conclusion

The RCO protocol achieves strong system-level security through the composition of deterministic serialization, cryptographic lineage, Proof of Reflexion, and threshold verification. The protocol guarantees safety under all conditions and liveness under standard network assumptions, while clearly defining the boundaries under which these guarantees hold. This establishes the protocol as a robust and formally verified framework for secure telemetry ingestion in adversarial environments.

E. RCO-Lite Architecture

E.1 Overview

The RCO protocol, as defined in the preceding sections, establishes a mathematically rigorous framework for ensuring deterministic, verifiable, and state-consistent telemetry in feedback-coupled systems. However, practical deployment requires the translation of these formal constructs into a concrete system architecture composed of interacting components, communication channels, and execution pipelines.

This section presents a comprehensive architectural model that implements the RCO protocol in a distributed environment. Each mathematical operator introduced in the cryptographic core is mapped to a corresponding system component, and the ingestion pipeline is realized as a sequence of deterministic processing stages. The resulting architecture ensures that the theoretical guarantees of the protocol are preserved under real-world conditions.

E.2 Architectural Abstraction

Let the system be defined as a tuple:

$$\mathcal{A}_{\text{sys}} = (\mathcal{N}, \mathcal{C}, \mathcal{P}, \mathcal{T}) \tag{E.1}$$

where:

- $\mathcal{N}$ is the set of computational nodes,
- $\mathcal{C}$ is the set of communication channels,
- $\mathcal{P}$ is the set of processing pipelines,
- $\mathcal{T}$ is the set of telemetry streams.

Each node $N_i \in \mathcal{N}$ executes a deterministic processing function:

$$\mathcal{F}_i : (S_t, D_t, L_{t-1}) \to (S_{t+1}, L_t) \tag{E.2}$$

subject to the constraints defined by the RCO protocol.

E.3 Component Decomposition

The architecture is decomposed into the following primary components:

E.3.1 Telemetry Producers

Telemetry producers are entities that generate raw telemetry data from system processes, sensors, or computational outputs. Formally, a producer implements:

$$D_t = \mathcal{G}(S_t) \tag{E.3}$$

where $\mathcal{G}$ is the observation function.

Producers are untrusted by default and must not be assumed to generate valid or consistent data.

E.3.2 Canonicalization Engine

The canonicalization engine implements the deterministic serialization function $\mathcal{S}$. Given input telemetry D_t, it produces:

$$B_t = \mathcal{S}(D_t) \tag{E.4}$$

This component enforces:

$$D_1 \equiv D_2 \implies B_1 = B_2 \tag{E.5}$$

ensuring representation consistency across nodes.

E.3.3 Hashing Module

The hashing module computes the content hash:

$$H_t = \mathcal{H}(B_t) \tag{E.6}$$

This module must use a cryptographically secure hash function with collision resistance properties.

E.3.4 Lineage Engine

The lineage engine maintains the recursive hash chain:

$$L_t = \mathcal{H}(H_t \parallel L_{t-1}) \tag{E.7}$$

It is responsible for:

- maintaining temporal ordering,
- detecting tampering,
- ensuring continuity of the telemetry sequence.

E.3.5 State Binding Module

The state binding module computes:

$$M_t = \mathcal{H}_S(S_t) \parallel L_t \tag{E.8}$$

This module ensures that telemetry is cryptographically tied to the system state.

E.3.6 Validator Network

The validator network consists of nodes $\mathcal{V} = \{V_1, \ldots, V_n\}$ that independently verify telemetry and generate partial signatures.

Each validator implements:

$$\sigma_i = \mathrm{Sign}_i(M_t, K_i) \tag{E.9}$$

Validators operate under a zero-trust model and must independently verify all conditions.

E.3.7 Aggregation Layer

The aggregation layer collects partial signatures and constructs:

$$\Sigma_t = \mathrm{Aggregate}(\sigma_{i_1}, \ldots, \sigma_{i_m}) \tag{E.10}$$

for $m \geq t$.

E.3.8 Verification Engine

The verification engine evaluates:

$$\text{Verify}(M_t, \Sigma_t, PK) \tag{E.11}$$

and determines acceptance or rejection of telemetry.

E.4 Pipeline Architecture

The system implements a deterministic pipeline:

$$D_t \to \mathcal{S} \to \mathcal{H} \to \mathcal{C} \to \mathcal{H}_S \to \text{Sign} \to \text{Aggregate} \to \text{Verify} \tag{E.12}$$

Each stage is stateless with respect to external influence and deterministic given its inputs.

E.5 Data Flow Model

The flow of data through the system is defined as:

$$(D_t, S_t, L_{t-1}) \xrightarrow{\text{pipeline}} (H_t, L_t, \Sigma_t) \tag{E.13}$$

This transformation ensures that each telemetry element is:

- deterministically encoded,
- cryptographically hashed,
- chained into lineage,
- bound to system state,
- verified by multiple validators.

E.6 Distributed Deployment Model

The architecture supports distributed deployment across multiple nodes. Let $\mathcal{N}$ be partitioned into:

$$\mathcal{N} = \mathcal{N}_{\text{producers}} \cup \mathcal{N}_{\text{validators}} \cup \mathcal{N}_{\text{consumers}} \tag{E.14}$$

Each node type performs distinct roles:

- Producers generate telemetry,

- Validators verify and sign,
- Consumers use validated telemetry.

E.7 Consistency Across Nodes

All nodes must maintain identical lineage state:

$$\forall i, j, \quad L_t^{(i)} = L_t^{(j)} \tag{E.15}$$

This is ensured through deterministic processing and consensus on accepted telemetry.

E.8 Fault Tolerance

The system tolerates up to $f < t$ faulty validators. If fewer than t signatures are available, telemetry is rejected, preserving safety.

E.9 Scalability Considerations

The architecture supports horizontal scaling:

$$\text{Throughput} \propto |\mathcal{V}| \tag{E.16}$$

as signature generation and verification can be parallelized.

E.10 Conclusion

The system architecture presented in this section provides a direct mapping from the formal RCO model to a deployable infrastructure. By decomposing the protocol into well-defined components and enforcing deterministic processing at each stage, the architecture preserves the integrity guarantees established in the cryptographic and security analyses while enabling practical implementation in distributed environments.

E.11 Pipeline Execution and Sequence Semantics

The RCO architecture is realized as a deterministic execution pipeline in which telemetry data flows through a sequence of transformation and verification stages. Each stage enforces a specific invariant and produces an output that serves as the input to the next

stage. The correctness of the system depends on the strict ordering and deterministic execution of these stages across all participating nodes.

E.11.1 Execution Model

Let a telemetry event be represented as a tuple:

$$E_t = (D_t, S_t, L_{t-1}) \tag{E.17}$$

The pipeline execution is defined as a sequence of transformations:

$$E_t \xrightarrow{\mathcal{S}} B_t \xrightarrow{\mathcal{H}} H_t \xrightarrow{\mathcal{C}} L_t \xrightarrow{\mathcal{H}_S} M_t \xrightarrow{\text{Sign}} \{\sigma_i\} \xrightarrow{\text{Aggregate}} \Sigma_t \xrightarrow{\text{Verify}} \text{Decision} \tag{E.18}$$

Each transformation is deterministic and side-effect-free with respect to external state.

E.11.2 Stage-by-Stage Execution Semantics

Stage 1: Canonicalization

$$B_t = \mathcal{S}(D_t) \tag{E.19}$$

The system enforces:

$$\forall i, j, \quad \mathcal{S}_i(D_t) = \mathcal{S}_j(D_t) \tag{E.20}$$

ensuring cross-node consistency.

Stage 2: Hashing

$$H_t = \mathcal{H}(B_t) \tag{E.21}$$

This stage compresses the telemetry into a fixed-length representation.

Stage 3: Lineage Update

$$L_t = \mathcal{H}(H_t \parallel L_{t-1}) \tag{E.22}$$

This enforces temporal ordering and immutability.

Stage 4: State Binding

$$M_t = \mathcal{H}_S(S_t) \parallel L_t \tag{E.23}$$

This binds telemetry to system state.

Stage 5: Distributed Signing Each validator computes:

$$\sigma_i = \mathrm{Sign}_i(M_t, K_i) \tag{E.24}$$

Stage 6: Aggregation

$$\Sigma_t = \mathrm{Aggregate}(\sigma_{i_1}, \ldots, \sigma_{i_m}) \tag{E.25}$$

Stage 7: Verification

$$\mathrm{Decision} = \begin{cases} \mathrm{accept}, & \text{if } \mathrm{Verify}(M_t, \Sigma_t, PK) \\ \mathrm{reject}, & \text{otherwise} \end{cases} \tag{E.26}$$

E.11.3 End-to-End Execution Invariant

The pipeline enforces:

$$\mathrm{accept}(E_t) \implies \mathrm{Valid}(D_t) \tag{E.27}$$

ensuring that only valid telemetry progresses.

E.12 Sequence Diagram Representation

The interaction between components may be represented as a sequence diagram. The following LaTeX-compatible representation describes the interaction flow.

E.12.1 Logical Sequence Description

$$\text{Producer} \to \text{Canonicalizer} \to \text{Hasher} \to \text{Lineage Engine} \to \text{Validators} \to \text{Aggregator} \to \text{Verifier} \tag{E.28}$$

E.12.2 Formal Interaction Steps

1. Producer emits D_t
2. Canonicalizer computes $B_t = \mathcal{S}(D_t)$
3. Hasher computes $H_t = \mathcal{H}(B_t)$
4. Lineage engine computes L_t
5. State binding computes M_t

6. Validators receive (M_t) and produce σ_i
7. Aggregator collects σ_i and computes Σ_t
8. Verifier evaluates Σ_t and returns decision

E.12.3 TikZ Sequence Diagram Template

The following structure can be used to render diagrams:

```
\begin{tikzpicture}
\node (producer) {Producer};
\node (canon) [right of=producer] {Canonicalizer};
\node (hash) [right of=canon] {Hasher};
\node (lineage) [right of=hash] {Lineage};
\node (validators) [right of=lineage] {Validators};
\node (agg) [right of=validators] {Aggregator};
\node (verify) [right of=agg] {Verifier};

\draw[->] (producer) -- (canon);
\draw[->] (canon) -- (hash);
\draw[->] (hash) -- (lineage);
\draw[->] (lineage) -- (validators);
\draw[->] (validators) -- (agg);
\draw[->] (agg) -- (verify);
\end{tikzpicture}
```

E.13 Real-World Deployment Topologies

The RCO architecture is designed to operate across multiple deployment environments, including centralized cloud systems, decentralized edge networks, and hybrid configurations.

E.13.1 Cloud Deployment Model

In a cloud deployment, all components are hosted within a centralized infrastructure:

$$\mathcal{N} = \mathcal{N}_{\text{cloud}} \tag{E.29}$$

Characteristics:

- Low latency between components
- High computational resources
- Centralized management

Validators may be deployed as independent services within the cloud to preserve zero-trust properties.

E.13.2 Edge Deployment Model

In an edge deployment, telemetry producers and validators are distributed across geographically dispersed nodes:

$$\mathcal{N} = \mathcal{N}_{\text{edge}} \tag{E.30}$$

Characteristics:

- Reduced latency to data sources
- Increased network variability
- Partial connectivity

The ingestion pipeline must handle intermittent connectivity and delayed aggregation.

E.13.3 Hybrid Deployment Model

A hybrid model combines cloud and edge components:

$$\mathcal{N} = \mathcal{N}_{\text{edge}} \cup \mathcal{N}_{\text{cloud}} \tag{E.31}$$

In this model:

- Canonicalization and hashing occur at the edge,
- Aggregation and verification occur in the cloud,
- Lineage is synchronized across both layers.

E.13.4 Topology Selection Criteria

The choice of deployment topology depends on:

$$\text{Topology} = f(\text{latency}, \text{throughput}, \text{trust}, \text{connectivity}) \tag{E.32}$$

E.13.5 Distributed Validator Placement

Validators should be distributed such that:

$$\forall i, j, \quad \text{NetworkDistance}(V_i, V_j) \text{ is maximized} \tag{E.33}$$

to reduce correlated failures and collusion risk.

E.13.6 Scalability Model

Let n be the number of validators. The system throughput is:

$$\text{Throughput} = O(n) \tag{E.34}$$

while verification cost remains constant due to aggregation.

E.13.7 Fault Isolation

Failures in one component do not propagate:

$$\text{Failure}(C_i) \not\Longrightarrow \text{Failure}(C_j) \tag{E.35}$$

for $i \neq j$, ensuring modular resilience.

E.13.8 Deployment Invariant

Regardless of topology, the system must enforce:

$$\forall t, \quad L_t^{(i)} = L_t^{(j)} \tag{E.36}$$

across all nodes.

E.14 Conclusion

The pipeline execution model and deployment topologies described in this section translate the formal RCO protocol into a concrete, implementable system. By defining deterministic execution semantics, explicit interaction flows, and flexible deployment configurations, the architecture ensures that the theoretical guarantees of the protocol are preserved in practical environments. This establishes a foundation for building scalable,

secure, and verifiable telemetry systems across diverse infrastructure settings.

E.15 API Design, Data Structures, Storage Model, and Performance Analysis

The practical deployment of the RCO protocol requires a well-defined interface layer through which telemetry producers, validators, and consumers interact with the system. This section specifies the application programming interfaces (APIs), internal data structures, storage mechanisms, and performance characteristics necessary to implement the protocol in production environments.

E.16 API Design

The API layer provides a deterministic interface for submitting telemetry, retrieving lineage states, and verifying proofs.

E.16.1 Telemetry Submission API

The primary entry point into the system is the telemetry submission endpoint:

$$\texttt{SubmitTelemetry}(D_t, S_t) \to (L_t, \Sigma_t, \text{status}) \tag{E.37}$$

The function executes the full ingestion pipeline:

$$(D_t, S_t, L_{t-1}) \to (H_t, L_t, \Sigma_t) \tag{E.38}$$

The response includes:

- L_t: updated lineage state,
- Σ_t: aggregated proof,
- status: accept or reject.

E.16.2 Verification API

External entities may verify telemetry independently:

$$\texttt{VerifyTelemetry}(D_t, S_t, L_{t-1}, \Sigma_t) \to \{\text{true}, \text{false}\} \tag{E.39}$$

This API recomputes:

$$B_t = \mathcal{S}(D_t), \quad H_t = \mathcal{H}(B_t), \quad L_t = \mathcal{H}(H_t \parallel L_{t-1}) \tag{E.40}$$

and verifies:

$$\text{Verify}(\mathcal{H}_S(S_t) \parallel L_t, \Sigma_t, PK) \tag{E.41}$$

E.16.3 Lineage Query API

$$\texttt{GetLineage}(t) \to L_t \tag{E.42}$$

This allows retrieval of lineage states for auditing.

E.16.4 Batch Processing API

For high-throughput environments:

$$\texttt{SubmitBatch}(\{D_i, S_i\}_{i=1}^k) \to \{(L_i, \Sigma_i)\}_{i=1}^k \tag{E.43}$$

Batch processing preserves ordering:

$$L_i = \mathcal{H}(H_i \parallel L_{i-1}) \tag{E.44}$$

E.17 Data Structures

The system maintains several core data structures.

E.17.1 Telemetry Record

$$\mathcal{R}_t = (D_t, B_t, H_t, L_t, \Sigma_t) \tag{E.45}$$

This structure encapsulates all information required for verification.

E.17.2 Lineage Chain

The lineage is stored as a sequence:

$$\mathcal{L} = \{L_0, L_1, \ldots, L_t\} \tag{E.46}$$

Each element depends on its predecessor.

E.17.3 Validator State

Each validator maintains:

$$\mathcal{V}_i = (K_i, PK, \text{local cache}) \tag{E.47}$$

E.17.4 Proof Object

$$\Sigma_t = (\sigma_{i_1}, \ldots, \sigma_{i_m}) \tag{E.48}$$

or its aggregated form.

E.18 Storage Model

The storage layer must ensure persistence, immutability, and efficient retrieval.

E.18.1 Append-Only Log

Telemetry records are stored in an append-only log:

$$\mathcal{S}_{\text{log}} = [\mathcal{R}_0, \mathcal{R}_1, \ldots, \mathcal{R}_t] \tag{E.49}$$

This ensures immutability:

$$\forall i < j, \quad \mathcal{R}_i \text{ cannot be modified after } \mathcal{R}_j \text{ is written} \tag{E.50}$$

E.18.2 Indexing

Efficient lookup is enabled via indexing:

$$\text{Index} : t \mapsto \mathcal{R}_t \tag{E.51}$$

E.18.3 Distributed Storage

In distributed environments:

$$\mathcal{S}_{\text{dist}} = \bigcup_{i=1}^{n} \mathcal{S}_i \tag{E.52}$$

Each node stores a replica of the log.

E.18.4 Consistency Model

The system enforces strong consistency:

$$\forall i, j, \quad \mathcal{S}_i = \mathcal{S}_j \tag{E.53}$$

subject to eventual synchronization.

E.18.5 Checkpointing

To reduce storage overhead, checkpoints may be introduced:

$$C_k = L_k \tag{E.54}$$

allowing reconstruction from checkpoints.

E.19 Memory Model

The system uses:

- In-memory buffers for pending telemetry,
- Persistent storage for finalized records,
- Caches for frequently accessed lineage states.

E.20 Performance Characteristics

E.20.1 Time Complexity

For each telemetry element:

$$T_{\text{total}} = T_{\mathcal{S}} + T_{\mathcal{H}} + T_{\mathcal{C}} + T_{\text{sign}} + T_{\text{agg}} + T_{\text{verify}} \tag{E.55}$$

Where:

- $T_{\mathcal{S}} = O(n \log n)$ for sorting keys,
- $T_{\mathcal{H}} = O(|B_t|)$,

- $T_{\mathcal{C}} = O(1)$,
- $T_{\text{sign}} = O(1)$ per validator,
- $T_{\text{agg}} = O(t)$,
- $T_{\text{verify}} = O(1)$.

Thus:

$$T_{\text{total}} = O(n \log n + |B_t| + t) \tag{E.56}$$

E.20.2 Space Complexity

$$S_{\text{total}} = O(t \cdot k) \tag{E.57}$$

where k is hash size.

E.20.3 Network Complexity

Each telemetry requires:

$$O(n) \text{ messages for signature distribution} \tag{E.58}$$

E.20.4 Latency Model

$$T_{\text{latency}} = \max_i T_{\text{sign}}^{(i)} + T_{\text{network}} + T_{\text{agg}} \tag{E.59}$$

E.20.5 Throughput

$$\text{Throughput} \propto \frac{1}{T_{\text{latency}}} \tag{E.60}$$

E.21 Optimization Strategies

E.21.1 Parallel Signing

Validators operate in parallel:

$$T_{\text{sign}} = \max_i T_{\text{sign}}^{(i)} \tag{E.61}$$

E.21.2 Batch Aggregation

$$\Sigma = \text{Aggregate}(\{\sigma_i^{(j)}\}) \tag{E.62}$$

for multiple j.

E.21.3 Caching

$$\text{Cache}(L_t) \to O(1) \text{ retrieval} \tag{E.63}$$

E.22 Scalability Analysis

Let n be validators and t threshold.

$$\text{Scalability} = O(n) \quad \text{(parallelizable)} \tag{E.64}$$

Verification remains constant due to aggregation.

E.23 Fault Tolerance

The system tolerates:

$$f < t \tag{E.65}$$

faulty validators.

E.24 Deployment Constraints

$$\text{Consistency} > \text{Availability} \tag{E.66}$$

ensuring safety-first design.

E.25 End-to-End Performance Bound

$$\Pr[\text{Failure}] \leq \epsilon_{\mathcal{H}} + \epsilon_{\mathcal{H}_S} + \epsilon_{\text{sig}} + \epsilon_{\text{threshold}} \tag{E.67}$$

E.26 Conclusion

This section defines a complete production-ready specification for implementing the RCO protocol. By formalizing APIs, data structures, storage models, and performance characteristics, it bridges the gap between theoretical design and practical deployment. The architecture ensures that the integrity guarantees established in earlier sections are preserved while enabling scalable and efficient operation in real-world environments.

F. API & SDK Specification

F.1 Use Cases and Applications

F.2 Overview

The Feedback-Coupled Cryptographic Observation (RCO) protocol establishes a universal integrity framework applicable to any computational system in which telemetry participates in feedback-driven decision-making processes. Unlike traditional integrity mechanisms that operate at the level of storage or transmission, the RCO protocol operates at the level of system evolution, ensuring that all telemetry influencing system state transitions is deterministic, verifiable, and causally consistent.

Let a general system be defined as:

$$S_{t+1} = \mathcal{F}(S_t, A_t, D_t) \tag{F.1}$$

where D_t represents telemetry inputs. The applicability of the RCO protocol arises from the condition:

$$\frac{\partial S_{t+1}}{\partial D_t} \neq 0 \tag{F.2}$$

which implies that telemetry directly influences system behavior. Any system satisfying this condition is susceptible to telemetry-induced perturbations and therefore benefits from the enforcement of RCO invariants.

F.3 Application in Artificial Intelligence Systems

Artificial intelligence systems, particularly those involving iterative training and inference, rely heavily on telemetry data such as gradients, rewards, and environmental observations. These systems are inherently feedback-driven and thus highly sensitive to inconsistencies in telemetry.

F.3.1 Training Pipeline Integrity

Let θ_t denote model parameters and D_t the training data or gradient signals. The update rule is:

$$\theta_{t+1} = \theta_t + \alpha \cdot \nabla J(\theta_t, D_t) \tag{F.3}$$

Any perturbation δ_t in D_t leads to:

$$\theta'_{t+1} = \theta_t + \alpha \cdot \nabla J(\theta_t, D_t + \delta_t) \tag{F.4}$$

The divergence is:

$$\|\theta'_{t+1} - \theta_{t+1}\| \leq \alpha \cdot L\|\delta_t\| \tag{F.5}$$

where L is the Lipschitz constant of the gradient.

The RCO protocol enforces:

$$\delta_t = 0 \tag{F.6}$$

thereby ensuring deterministic training trajectories.

F.3.2 Federated Learning

In federated learning, multiple clients contribute updates:

$$\theta_{t+1} = \sum_{i=1}^{n} w_i \cdot \theta_t^{(i)} \tag{F.7}$$

Without RCO, malicious clients can inject poisoned updates. With RCO:

$$\text{Valid}(\theta_t^{(i)}) \implies \Sigma_t^{(i)} \text{ verified} \tag{F.8}$$

ensuring only authenticated updates are aggregated.

F.3.3 Inference Integrity

Inference systems rely on telemetry such as input features and intermediate activations. The RCO protocol ensures:

$$\text{Inference}(x) = f(x, D_t) \tag{F.9}$$

is reproducible and verifiable.

F.4 Application in Financial Systems

Financial systems are highly sensitive to telemetry such as market data, transaction logs, and risk indicators.

F.4.1 Algorithmic Trading

Let S_t represent system state and D_t market data:

$$A_t = \pi(S_t, D_t) \tag{F.10}$$

Small discrepancies in D_t lead to divergent trading decisions. The RCO protocol ensures:

$$D_t^{(i)} = D_t^{(j)} \quad \forall i, j \tag{F.11}$$

across all nodes.

F.4.2 Auditability

Financial systems require audit trails:

$$\mathcal{A}(\{D_t\}) \to \text{history} \tag{F.12}$$

RCO ensures:

$$\mathcal{A}(\{L_t, \Sigma_t\}) = \text{verifiable history} \tag{F.13}$$

F.4.3 Fraud Detection

Telemetry inconsistencies are often indicators of fraud. RCO enforces:

$$\delta_t \neq 0 \implies \text{reject} \tag{F.14}$$

thus preventing manipulated inputs.

F.5 Application in Cybersecurity Systems

Cybersecurity systems rely on logs, alerts, and telemetry from distributed sources.

F.5.1 Intrusion Detection Systems

Let D_t represent network telemetry. Detection function:

$$\text{Alert}_t = \mathcal{F}(D_t) \tag{F.15}$$

Adversarial manipulation of D_t leads to evasion. RCO ensures:

$$\text{Valid}(D_t) \implies \text{trusted detection} \tag{F.16}$$

F.5.2 Forensic Analysis

Forensic reconstruction requires:

$$\{D_t\} \to \text{event timeline} \tag{F.17}$$

RCO guarantees:

$$L_t \text{ uniquely identifies sequence} \tag{F.18}$$

preventing tampering.

F.6 Application in Distributed Infrastructure

Distributed systems depend on telemetry for coordination.

F.6.1 Consensus Systems

Let nodes exchange telemetry $D_t^{(i)}$. Consensus requires:

$$D_t^{(i)} = D_t^{(j)} \tag{F.19}$$

RCO enforces this via deterministic encoding and verification.

F.6.2 Microservices Observability

Telemetry includes logs, metrics, traces:

$$D_t = (\text{logs}, \text{metrics}, \text{traces}) \tag{F.20}$$

RCO ensures consistency across services.

F.7 Cross-Domain Applicability Condition

The RCO protocol applies to any system satisfying:

$$\exists \mathcal{F} \text{ such that } \frac{\partial S}{\partial D} \neq 0 \tag{F.21}$$

This includes virtually all modern computational systems.

F.8 Economic and Operational Impact

The enforcement of deterministic telemetry results in:

$$\text{Error Reduction} \propto \|\delta_t\| \tag{F.22}$$

and since RCO enforces $\delta_t = 0$:

$$\text{Error} \to 0 \tag{F.23}$$

leading to improved reliability, reduced risk, and enhanced auditability.

F.9 Conclusion

The RCO protocol provides a universal solution to telemetry integrity across diverse domains including artificial intelligence, finance, cybersecurity, and distributed systems. By enforcing deterministic representation, cryptographic lineage, and state-bound verification, the protocol eliminates a fundamental class of errors and vulnerabilities inherent in modern computational infrastructures. This establishes the protocol as a foundational technology for next-generation high-assurance systems.

F.10 Advanced High-Impact Use Cases and Strategic Positioning

The applicability of the RCO protocol extends beyond conventional computational systems into domains characterized by high-stakes decision-making, safety-critical operations, and strict regulatory requirements. In such environments, the integrity of telemetry is not merely desirable but essential for ensuring correctness, accountability, and safety. This section explores advanced use cases in autonomous systems, critical infrastructure, and regulatory frameworks, and provides a comparative analysis with existing paradigms such as blockchain-based systems.

F.11 Autonomous Systems

Autonomous systems, including self-driving vehicles, robotic platforms, and unmanned aerial systems, operate in dynamic environments where decisions are continuously influenced by sensor telemetry. These systems are inherently feedback-driven and must maintain strict consistency between perceived state and executed actions.

F.11.1 Control Loop Integrity

Let S_t denote the internal state of an autonomous system and D_t the sensor telemetry. The control law is defined as:

$$A_t = \mathcal{K}(S_t, D_t) \tag{F.24}$$

and the system evolves as:

$$S_{t+1} = \mathcal{F}(S_t, A_t) \tag{F.25}$$

Any perturbation δ_t in D_t results in:

$$A'_t = \mathcal{K}(S_t, D_t + \delta_t) \tag{F.26}$$

which may lead to unsafe behavior. The RCO protocol enforces:

$$\delta_t = 0 \tag{F.27}$$

ensuring that control decisions are based on authentic and consistent telemetry.

F.11.2 Sensor Fusion Consistency

Modern autonomous systems rely on multiple sensors:

$$D_t = \{D_t^{(1)}, D_t^{(2)}, \ldots, D_t^{(m)}\} \tag{F.28}$$

Fusion algorithms compute:

$$\hat{D}_t = \mathcal{F}_{\text{fusion}}(D_t) \tag{F.29}$$

RCO ensures that each component $D_t^{(i)}$ is verifiable and consistent across nodes, preventing adversarial manipulation of individual sensors.

F.11.3 Safety Guarantees

The safety of autonomous systems depends on bounded error:

$$\|S_t' - S_t\| \leq \epsilon \tag{F.30}$$

RCO enforces deterministic telemetry, reducing uncertainty and improving safety margins.

F.12 Critical Infrastructure Systems

Critical infrastructure systems, including power grids, transportation networks, and industrial control systems, rely on telemetry for monitoring and control.

F.12.1 Supervisory Control Systems

Let D_t represent measurements from distributed sensors. Control decisions are based on:

$$A_t = \mathcal{F}(D_t) \tag{F.31}$$

Manipulated telemetry can lead to incorrect control actions. RCO ensures:

$$\text{Valid}(D_t) \implies \text{trusted control} \tag{F.32}$$

F.12.2 Distributed Grid Stability

In power systems, stability depends on synchronized telemetry:

$$D_t^{(i)} = D_t^{(j)} \quad \forall i, j \tag{F.33}$$

RCO enforces this condition, preventing instability due to inconsistent measurements.

F.12.3 Failure Propagation Prevention

Telemetry errors can propagate:

$$\Delta_{t+1} \leq L\Delta_t + \|\delta_t\| \tag{F.34}$$

RCO eliminates δ_t, ensuring:

$$\Delta_{t+1} \leq L\Delta_t \tag{F.35}$$

thus preventing cascading failures.

F.13 Comparison with Blockchain-Based Systems

Blockchain systems provide immutability and distributed consensus, but they are fundamentally different from the RCO protocol.

F.13.1 Structural Differences

Blockchain defines:

$$B_t = \mathcal{H}(D_t \parallel B_{t-1}) \tag{F.36}$$

which ensures immutability but does not enforce:

$$D_t = \mathcal{G}(S_t) \tag{F.37}$$

In contrast, RCO enforces both:

$$\text{Integrity} + \text{State Consistency} \tag{F.38}$$

F.13.2 Lack of Deterministic Serialization

Blockchain systems do not enforce canonical serialization, leading to:

$$D_1 \equiv D_2 \not\Longrightarrow \mathcal{H}(D_1) = \mathcal{H}(D_2) \tag{F.39}$$

RCO eliminates this ambiguity through deterministic encoding.

F.13.3 Absence of State Binding

Blockchain signatures are applied to transactions:

$$\Sigma_t = \mathrm{Sign}(D_t) \tag{F.40}$$

but do not bind to system state. RCO introduces:

$$\Sigma_t = \mathrm{Sign}(\mathcal{H}_S(S_t) \parallel L_t) \tag{F.41}$$

which enforces semantic integrity.

F.13.4 Latency Constraints

Blockchain consensus introduces latency:

$$T_{\text{blockchain}} \gg T_{\text{RCO}} \tag{F.42}$$

RCO enables near real-time validation.

F.13.5 Novelty Positioning

The RCO protocol introduces a new paradigm:

$$\text{Telemetry as a Causative Variable} \tag{F.43}$$

with enforced invariants, which is not addressed by blockchain systems.

F.14 Regulatory and Compliance Applications

Regulatory environments require auditability, traceability, and non-repudiation.

F.14.1 Audit Trail Verification

Regulators require:

$$\mathcal{A}(\{D_t\}) = \text{complete history} \tag{F.44}$$

RCO ensures:

$$\mathcal{A}(\{L_t, \Sigma_t\}) = \text{cryptographically verifiable history} \tag{F.45}$$

F.14.2 Compliance Constraints

Systems must satisfy:

$$\text{Compliance} \implies \text{Data Integrity} \tag{F.46}$$

RCO provides:

$$\text{Integrity} \Rightarrow \text{Compliance} \tag{F.47}$$

F.14.3 Non-Repudiation

Entities cannot deny actions:

$$\Sigma_t \Rightarrow \text{proof of origin} \tag{F.48}$$

F.14.4 Forensic Reconstruction

In case of disputes:

$$\{L_t, \Sigma_t\} \rightarrow \text{verifiable reconstruction} \tag{F.49}$$

F.15 Cross-Domain Economic Impact

The adoption of RCO results in:

$$\text{Risk Reduction} \propto \|\delta_t\| \tag{F.50}$$

Since:

$$\delta_t = 0 \tag{F.51}$$

we obtain:

$$\text{Risk} \to \text{minimum} \tag{F.52}$$

F.16 Strategic Positioning

The RCO protocol occupies a unique position:

$$\text{RCO} \notin \{\text{Blockchain}, \text{Logging}, \text{Monitoring}\} \tag{F.53}$$

but instead defines a new class:

$$\text{Feedback-Coupled Integrity Systems} \tag{F.54}$$

F.17 Conclusion

The advanced use cases presented in this section demonstrate that the RCO protocol addresses critical challenges in high-impact domains where telemetry integrity directly affects system behavior. By providing deterministic, verifiable, and state-consistent telemetry, the protocol enables safer autonomous systems, more resilient infrastructure, and stronger regulatory compliance. Furthermore, its distinction from existing paradigms such as blockchain underscores its novelty and establishes it as a foundational innovation in secure computational systems.

F.18 Industry-Specific Deep Dives and Future Applications

The RCO protocol is not limited to current computational paradigms but extends into emerging and highly regulated domains where telemetry integrity directly impacts safety, legality, and long-term system reliability. This section provides detailed analysis of its application in healthcare systems, defense and mission-critical operations, artificial intelligence governance frameworks, and digital twin infrastructures. These domains represent both present and future environments in which deterministic and verifiable telemetry

becomes a fundamental requirement.

F.19 Healthcare Systems

Healthcare systems increasingly rely on continuous telemetry derived from medical devices, patient monitoring systems, and diagnostic platforms. In such environments, telemetry directly influences clinical decision-making, and any inconsistency can result in severe consequences.

F.19.1 Clinical Decision Support Systems

Let S_t represent patient state and D_t represent clinical telemetry such as vital signs, laboratory results, and imaging data. Decision support systems compute:

$$A_t = \mathcal{F}(S_t, D_t) \tag{F.55}$$

where A_t may correspond to treatment recommendations.

Any perturbation δ_t in D_t yields:

$$A'_t = \mathcal{F}(S_t, D_t + \delta_t) \tag{F.56}$$

The divergence:

$$\|A'_t - A_t\| \leq L\|\delta_t\| \tag{F.57}$$

can lead to incorrect treatment decisions. The RCO protocol enforces:

$$\delta_t = 0 \tag{F.58}$$

thereby ensuring that clinical decisions are based on authentic and consistent data.

F.19.2 Medical Device Telemetry

Modern medical devices generate continuous telemetry streams:

$$D_t = \{d_t^{(1)}, d_t^{(2)}, \ldots, d_t^{(n)}\} \tag{F.59}$$

RCO ensures that each telemetry element is verifiable and immutable, enabling reliable

monitoring and preventing tampering.

F.19.3 Regulatory Compliance in Healthcare

Healthcare systems must comply with strict regulatory frameworks requiring:

$$\text{Traceability} \wedge \text{Integrity} \wedge \text{Auditability} \tag{F.60}$$

RCO provides:

$$\{L_t, \Sigma_t\} \Rightarrow \text{cryptographically verifiable medical history} \tag{F.61}$$

F.20 Defense and Mission-Critical Systems

Defense systems operate in adversarial environments where telemetry integrity is a primary attack vector.

F.20.1 Command and Control Systems

Let D_t represent battlefield telemetry and S_t system state. Commands are generated as:

$$A_t = \mathcal{C}(S_t, D_t) \tag{F.62}$$

Adversarial manipulation of D_t leads to:

$$A'_t = \mathcal{C}(S_t, D_t + \delta_t) \tag{F.63}$$

which may result in incorrect or dangerous actions. RCO ensures that:

$$\text{Valid}(D_t) \implies \text{trusted command generation} \tag{F.64}$$

F.20.2 Secure Sensor Networks

Distributed sensors produce telemetry:

$$D_t^{(i)}, \quad i = 1, \ldots, n \tag{F.65}$$

RCO ensures:

$$D_t^{(i)} = D_t^{(j)} \quad \text{(after canonicalization and validation)} \tag{F.66}$$

preventing inconsistencies.

F.20.3 Resilience to Adversarial Manipulation

The adversarial model in defense systems includes:

$$\mathcal{A} = (\text{spoofing}, \text{jamming}, \text{injection}) \tag{F.67}$$

RCO reduces all such attacks to cryptographic failures, ensuring robustness.

F.21 Artificial Intelligence Governance

As AI systems become integral to decision-making processes, governance frameworks must ensure transparency, accountability, and reproducibility.

F.21.1 Model Accountability

Let θ_t represent model parameters influenced by telemetry D_t:

$$\theta_{t+1} = \mathcal{F}(\theta_t, D_t) \tag{F.68}$$

RCO ensures that:

$$\theta_{t+1} \text{ is reproducible given } \{D_t\} \tag{F.69}$$

enabling auditability.

F.21.2 Bias and Manipulation Detection

If telemetry is manipulated:

$$D'_t = D_t + \delta_t \tag{F.70}$$

then model outputs may exhibit bias. RCO eliminates such manipulation by enforcing:

$$\delta_t = 0 \tag{F.71}$$

F.21.3 Regulatory Framework Integration

AI governance requires:

$$\text{Explainability} \wedge \text{Traceability} \tag{F.72}$$

RCO provides:

$$\{L_t, \Sigma_t\} \rightarrow \text{verifiable audit trail} \tag{F.73}$$

F.22 Digital Twin Systems

Digital twins are virtual representations of physical systems that rely on continuous telemetry for synchronization.

F.22.1 State Synchronization

Let S_t^{physical} and S_t^{digital} denote physical and digital states. Synchronization requires:

$$S_t^{\text{digital}} = \mathcal{F}(S_t^{\text{physical}}, D_t) \tag{F.74}$$

Any inconsistency in D_t leads to divergence:

$$\|S_t^{\text{digital}} - S_t^{\text{physical}}\| \propto \|\delta_t\| \tag{F.75}$$

RCO enforces $\delta_t = 0$, ensuring accurate synchronization.

F.22.2 Predictive Modeling

Digital twins use telemetry for prediction:

$$S_{t+1}^{\text{pred}} = \mathcal{F}(S_t^{\text{digital}}, D_t) \tag{F.76}$$

RCO ensures that predictions are based on reliable data.

F.22.3 Lifecycle Integrity

Over long durations:

$$\sum_{t=0}^{T} \|\delta_t\| \to \text{drift} \tag{F.77}$$

RCO eliminates drift by enforcing deterministic telemetry.

F.23 Future Applicability

The RCO protocol is applicable to emerging domains including:

$$\{\text{smart cities}, \text{autonomous economies}, \text{cyber-physical systems}\} \tag{F.78}$$

In all such systems:

$$\frac{\partial S}{\partial D} \neq 0 \tag{F.79}$$

ensuring relevance.

F.24 Global Impact Perspective

The enforcement of deterministic telemetry leads to:

$$\text{System Reliability} \to \text{maximum} \tag{F.80}$$

$$\text{Risk} \to \text{minimum} \tag{F.81}$$

$$\text{Auditability} \to \text{complete} \tag{F.82}$$

F.25 Strategic Outlook

The RCO protocol represents a shift from:

$$\text{Post-hoc verification} \to \text{pre-validated execution} \tag{F.83}$$

and from:

$$\text{Trust-based systems} \rightarrow \text{verification-based systems} \tag{F.84}$$

F.26 Conclusion

The industry-specific applications presented in this section demonstrate that the RCO protocol addresses fundamental challenges across healthcare, defense, artificial intelligence governance, and digital twin systems. By ensuring deterministic, verifiable, and state-consistent telemetry, the protocol provides a foundation for building high-assurance systems in environments where correctness, safety, and accountability are paramount. Its applicability to emerging domains further establishes it as a forward-looking technology with broad and lasting impact.

G. Minimal Working Example

G.1 Minimal Working Example

G.2 Overview

This section provides a concrete instantiation of the RCO protocol in a minimal configuration sufficient to demonstrate its correctness, implementability, and end-to-end execution. The objective is to construct the smallest system that satisfies all protocol invariants while remaining simple enough for direct implementation and verification.

The minimal system consists of:

$$\mathcal{N} = \{\text{Producer}, V_1, V_2, V_3, \text{Verifier}\} \tag{G.1}$$

with a threshold:

$$t = 2 \tag{G.2}$$

such that at least two validators are required to produce a valid proof.

G.3 System Initialization

G.3.1 Key Generation

A distributed key generation process produces:

$$(K_1, K_2, K_3), \quad PK \tag{G.3}$$

where each K_i is a private key share and PK is the public key.

G.3.2 Initial State

The system initializes:

$$L_0 = \mathcal{H}(0) \tag{G.4}$$

$$S_0 = \text{initial system state} \tag{G.5}$$

G.4 Telemetry Instance

Consider a concrete telemetry input:

$$D_1 = \{\text{temperature} : 25.0, \text{pressure} : 1.01\} \tag{G.6}$$

G.4.1 Canonical Serialization

Applying $\mathcal{S}$:

$$B_1 = \texttt{"pressure:1.01|temperature:25.0"} \tag{G.7}$$

G.5 Hash Computation

$$H_1 = \mathcal{H}(B_1) \tag{G.8}$$

G.6 Lineage Update

$$L_1 = \mathcal{H}(H_1 \parallel L_0) \tag{G.9}$$

G.7 State Binding

Assume:

$$S_1 = \text{system state after processing } D_1 \tag{G.10}$$

Compute:

$$M_1 = \mathcal{H}_S(S_1) \parallel L_1 \tag{G.11}$$

G.8 Validator Signing

Each validator computes:

$$\sigma_i = \mathrm{Sign}_i(M_1, K_i) \tag{G.12}$$

Select two validators:

$$\Sigma_1 = \mathrm{Aggregate}(\sigma_1, \sigma_2) \tag{G.13}$$

G.9 Verification

The verifier computes:

$$\mathrm{Verify}(M_1, \Sigma_1, PK) = \mathrm{true} \tag{G.14}$$

Thus:

$$\mathcal{I}_{\mathrm{RCO}}(D_1, L_0) = \mathrm{accept} \tag{G.15}$$

G.10 End-to-End Consistency

The final record is:

$$\mathcal{R}_1 = (D_1, B_1, H_1, L_1, \Sigma_1) \tag{G.16}$$

which satisfies all protocol invariants.

G.11 Minimal Correctness Property

$$\mathrm{accept}(D_1) \implies \mathrm{Valid}(D_1) \tag{G.17}$$

demonstrating correctness of the minimal system.

G.12 Conclusion

This minimal working example demonstrates that the RCO protocol can be instantiated with a small number of components while preserving its full integrity guarantees. The example provides a concrete foundation for implementation and validates the practical feasibility of the protocol.

G.13 Pseudocode and Execution Trace

This section provides a detailed pseudocode specification of the RCO protocol along with a step-by-step execution trace across multiple telemetry events. The objective is to demonstrate that the protocol can be directly implemented in a computational system while preserving its formal guarantees.

G.14 Core Pseudocode Definitions

G.14.1 Canonical Serialization Function

```
function CANONICALIZE(D):
    keys = SORT_KEYS(D)
    buffer = ""
    for k in keys:
        buffer += k + ":" + TO_STRING(D[k]) + "|"
    return buffer.strip("|")
```

G.14.2 Hash Function Wrapper

```
function HASH(data):
    return SHA256(data)
```

G.14.3 Lineage Update Function

```
function UPDATE_LINEAGE(H_t, L_prev):
    return HASH(H_t + L_prev)
```

G.14.4 State Binding Function

```
function BIND_STATE(S_t, L_t):
    return HASH(SERIALIZE_STATE(S_t)) + L_t
```

G.14.5 Validator Signing

```
function SIGN_PARTIAL(M_t, K_i):
    return SIGN(M_t, K_i)
```

G.14.6 Aggregation Function

```
function AGGREGATE(signatures):
    return COMBINE(signatures)
```

G.14.7 Verification Function

```
function VERIFY(M_t, Sigma_t, PK):
    return VERIFY_SIGNATURE(M_t, Sigma_t, PK)
```

G.14.8 Full Ingestion Pipeline

```
function INGEST(D_t, S_t, L_prev, validators, threshold):
    B_t = CANONICALIZE(D_t)
    H_t = HASH(B_t)
    L_t = UPDATE_LINEAGE(H_t, L_prev)
    M_t = BIND_STATE(S_t, L_t)

    partial_sigs = []
    for V in validators:
        if V.verify_locally(D_t, S_t, L_prev):
            partial_sigs.append(SIGN_PARTIAL(M_t, V.K_i))

    if len(partial_sigs) < threshold:
        return REJECT

    Sigma_t = AGGREGATE(partial_sigs)

    if VERIFY(M_t, Sigma_t, PK):
        return (ACCEPT, L_t, Sigma_t)
    else:
        return REJECT
```

G.15 Step-by-Step Execution Trace

We now demonstrate execution over multiple time steps.

G.15.1 Initial Conditions

$$L_0 = \mathcal{H}(0) \tag{G.18}$$

G.15.2 Step 1: First Telemetry Input

$$D_1 = \{\text{temperature} : 25.0, \text{pressure} : 1.01\} \tag{G.19}$$

Canonicalization

$$B_1 = \texttt{"pressure:1.01|temperature:25.0"} \tag{G.20}$$

Hashing

$$H_1 = \mathcal{H}(B_1) \tag{G.21}$$

Lineage

$$L_1 = \mathcal{H}(H_1 \parallel L_0) \tag{G.22}$$

State Binding

$$M_1 = \mathcal{H}_S(S_1) \parallel L_1 \tag{G.23}$$

Signing and Aggregation

$$\Sigma_1 = \text{Aggregate}(\sigma_1, \sigma_2) \tag{G.24}$$

Verification

$$\text{Verify}(M_1, \Sigma_1, PK) = \text{true} \tag{G.25}$$

G.15.3 Step 2: Second Telemetry Input

$$D_2 = \{\text{temperature} : 26.0, \text{pressure} : 1.02\} \tag{G.26}$$

Canonicalization

$$B_2 = \texttt{"pressure:1.02|temperature:26.0"} \tag{G.27}$$

Hashing

$$H_2 = \mathcal{H}(B_2) \tag{G.28}$$

Lineage

$$L_2 = \mathcal{H}(H_2 \parallel L_1) \tag{G.29}$$

State Binding

$$M_2 = \mathcal{H}_S(S_2) \parallel L_2 \tag{G.30}$$

Signing

$$\sigma_i^{(2)} = \mathrm{Sign}_i(M_2, K_i) \tag{G.31}$$

Aggregation

$$\Sigma_2 = \mathrm{Aggregate}(\sigma_1^{(2)}, \sigma_3^{(2)}) \tag{G.32}$$

Verification

$$\mathrm{Verify}(M_2, \Sigma_2, PK) = \mathrm{true} \tag{G.33}$$

G.15.4 State Evolution

The lineage evolves as:

$$L_0 \to L_1 \to L_2 \tag{G.34}$$

Each step satisfies:

$$L_t = \mathcal{H}(H_t \parallel L_{t-1}) \tag{G.35}$$

G.15.5 Determinism Property

For any node i:

$$L_t^{(i)} = L_t \tag{G.36}$$

ensuring consistency.

G.16 Failure Case Demonstration

Consider an adversarial modification:

$$D_2' = \{\text{temperature} : 30.0, \text{pressure} : 1.02\} \tag{G.37}$$

Then:

$$B_2' \neq B_2 \implies H_2' \neq H_2 \implies L_2' \neq L_2 \tag{G.38}$$

Thus:

$$\text{Verify}(M_2', \Sigma_2, PK) = \text{false} \tag{G.39}$$

and the input is rejected.

G.17 Execution Invariant

For all steps:

$$\text{accept}(D_t) \implies \begin{cases} \text{deterministic encoding} \\ \text{valid lineage} \\ \text{state consistency} \\ \text{verified proof} \end{cases} \tag{G.40}$$

G.18 Conclusion

The pseudocode and execution trace demonstrate that the RCO protocol can be implemented using standard computational primitives while preserving its formal guarantees. The step-by-step execution confirms that the system maintains deterministic behavior, rejects invalid inputs, and ensures consistent state evolution across all nodes.

G.19 Reference Implementation, System Constraints, and Deployment Mapping

This section provides a concrete reference implementation outline of the RCO protocol, defines system-level constraints required for correct operation, and maps the protocol to real-world software and infrastructure stacks. The objective is to demonstrate that the protocol can be deployed using existing technologies while preserving its formal guarantees.

G.20 Reference Implementation Architecture

A minimal reference implementation consists of the following modules:

$$\mathcal{M} = \{\text{Ingestion Service}, \text{Validator Nodes}, \text{Aggregation Service}, \text{Storage Engine}, \text{Verification Service}\} \tag{G.41}$$

Each module corresponds directly to components defined in the architectural model.

G.20.1 Ingestion Service

The ingestion service implements the deterministic pipeline:

```
INPUT: D_t, S_t, L_prev
OUTPUT: L_t, Sigma_t, status

1. B_t = CANONICALIZE(D_t)
2. H_t = HASH(B_t)
3. L_t = HASH(H_t + L_prev)
4. M_t = HASH_STATE(S_t) + L_t
5. DISPATCH M_t to validators
```

G.20.2 Validator Node

Each validator node executes:

```
INPUT: M_t
OUTPUT: sigma_i

1. if LOCAL_VERIFY(M_t):
2.     sigma_i = SIGN(M_t, K_i)
3.     return sigma_i
4. else:
5.     return null
```

G.20.3 Aggregation Service

```
INPUT: {sigma_i}
OUTPUT: Sigma_t

1. if count(valid signatures) >= threshold:
2.     Sigma_t = AGGREGATE(signatures)
3.     return Sigma_t
4. else:
5.     return REJECT
```

G.20.4 Verification Service

```
INPUT: M_t, Sigma_t
OUTPUT: decision

1. if VERIFY(M_t, Sigma_t, PK):
2.     return ACCEPT
3. else:
4.     return REJECT
```

G.20.5 Storage Engine

The storage engine maintains:

```
append_only_log = []

function STORE(record):
    append_only_log.append(record)
```

G.21 System Constraints

The correct operation of the RCO protocol requires the following constraints.

G.21.1 Deterministic Serialization Constraint

$$\forall D, \quad \mathcal{S}(D) \text{ must be canonical} \tag{G.42}$$

Any violation leads to divergence across nodes.

G.21.2 Cryptographic Assumptions

The system relies on:

$$\mathcal{H}, \mathcal{H}_S \text{ are collision-resistant} \tag{G.43}$$

$$\text{Signature scheme is EUF-CMA secure} \tag{G.44}$$

G.21.3 Threshold Constraint

$$f < t \leq n \tag{G.45}$$

where f is the number of faulty validators.

G.21.4 Network Constraint

The system assumes eventual message delivery:

$$\forall m, \quad \text{delivered}(m) \text{ within finite time} \tag{G.46}$$

G.21.5 State Consistency Constraint

All nodes must compute:

$$S_t^{(i)} = S_t^{(j)} \tag{G.47}$$

for identical inputs.

G.22 Minimal Deployment Diagram

The system may be deployed as:

$$\text{Producer} \to \text{Ingestion Service} \to \text{Validator Cluster} \to \text{Aggregator} \to \text{Verifier} \to \text{Storage} \tag{G.48}$$

G.22.1 Logical Flow

1. Producer emits telemetry
2. Ingestion service processes and forwards message
3. Validators independently sign
4. Aggregator combines signatures
5. Verifier confirms validity
6. Storage persists record

G.22.2 TikZ Deployment Template

```
\begin{tikzpicture}
\node (prod) {Producer};
\node (ingest) [right of=prod] {Ingestion};
```

```
\node (validators) [right of=ingest] {Validators};
\node (agg) [right of=validators] {Aggregator};
\node (verify) [right of=agg] {Verifier};
\node (store) [right of=verify] {Storage};

\draw[->] (prod) -- (ingest);
\draw[->] (ingest) -- (validators);
\draw[->] (validators) -- (agg);
\draw[->] (agg) -- (verify);
\draw[->] (verify) -- (store);
\end{tikzpicture}
```

G.23 Real-World Technology Mapping

The RCO protocol can be implemented using existing technologies as follows.

G.23.1 Programming Languages

- Go: for concurrency and networking
- Rust: for memory safety and cryptographic operations
- Python: for prototyping and orchestration

G.23.2 Cryptographic Libraries

- OpenSSL or libsodium for hashing and signatures
- BLS signature libraries for aggregation

G.23.3 Networking Layer

- gRPC for inter-node communication
- REST APIs for external interaction

G.23.4 Storage Systems

- Append-only logs using Kafka or Pulsar
- Persistent storage using PostgreSQL or distributed databases

G.23.5 Deployment Infrastructure

- Containerization using Docker
- Orchestration using Kubernetes
- Cloud deployment on distributed regions

G.24 Scalability and Deployment Constraints

G.24.1 Horizontal Scaling

$$\text{Throughput} \propto |\mathcal{V}| \tag{G.49}$$

G.24.2 Latency Bound

$$T_{\text{latency}} = \max(T_{\text{sign}}) + T_{\text{network}} + T_{\text{agg}} \tag{G.50}$$

G.24.3 Fault Isolation

$$\text{Failure}(V_i) \not\Longrightarrow \text{Failure}(V_j) \tag{G.51}$$

G.25 End-to-End Deployment Invariant

$$\forall t, \quad \text{Stored}(R_t) \implies \text{Valid}(D_t) \tag{G.52}$$

G.26 Implementation Feasibility Statement

The RCO protocol requires only standard computational primitives:

$$\{\text{hashing}, \text{signatures}, \text{networking}, \text{storage}\} \tag{G.53}$$

and does not depend on specialized hardware or non-standard assumptions.

G.27 Conclusion

This section demonstrates that the RCO protocol can be implemented using existing technologies and deployed in real-world environments. The reference implementation, system constraints, and deployment mapping collectively establish the practical feasibility of the protocol, confirming that its formal guarantees can be realized in operational

systems. This completes the minimal working example and validates the end-to-end applicability of the RCO framework.

H. Failure Mode Analysis

H.1 Failure Modes and Effects Analysis (FMEA)

H.2 Overview

This section presents a comprehensive Failure Modes and Effects Analysis (FMEA) of the Feedback-Coupled Cryptographic Observation (RCO) protocol. The objective of this analysis is to identify, classify, and formally evaluate all potential failure modes that may arise within the system, quantify their impact on system integrity, and demonstrate that the protocol either prevents such failures or bounds their effects under well-defined assumptions.

Unlike conventional FMEA approaches that focus on mechanical or software component failures in isolation, the RCO protocol requires a multi-layered analysis that integrates cryptographic, distributed, and feedback-coupled system dynamics. In particular, failure must be defined not only in terms of component malfunction but also in terms of violation of system invariants.

Let the system state be defined as:

$$\mathcal{S}_t = (D_t, S_t, L_t, \Sigma_t) \tag{H.1}$$

A failure occurs if:

$$\text{Valid}(D_t) = \text{false} \quad \wedge \quad \text{accept}(D_t) = \text{true} \tag{H.2}$$

or if system invariants are violated:

$$\exists t \text{ such that } L_t \neq \mathcal{H}(H_t \parallel L_{t-1}) \tag{H.3}$$

H.3 Failure Classification Model

Failures are categorized into the following classes:

$$\mathcal{F} = \{\mathcal{F}_{\text{representation}}, \mathcal{F}_{\text{cryptographic}}, \mathcal{F}_{\text{distributed}}, \mathcal{F}_{\text{adversarial}}, \mathcal{F}_{\text{operational}}\} \tag{H.4}$$

Each class corresponds to a distinct layer of the system.

H.4 Failure Severity Model

Define severity as a function:

$$\text{Severity} : \mathcal{F} \to \mathbb{R}^+ \tag{H.5}$$

where:

$$\text{Severity}(f) = \|\Delta S_t\| \tag{H.6}$$

representing the deviation in system state caused by failure f.

H.5 Failure Probability Model

Let:

$$P(f) = \Pr[\text{failure } f \text{ occurs}] \tag{H.7}$$

The overall system failure probability is:

$$P_{\text{system}} = \sum_{f \in \mathcal{F}} P(f) \tag{H.8}$$

H.6 Risk Function

Define risk as:

$$R(f) = \text{Severity}(f) \cdot P(f) \tag{H.9}$$

The protocol aims to minimize:

$$\sum_{f \in \mathcal{F}} R(f) \tag{H.10}$$

H.7 Failure Detection Model

A failure is detectable if:

$$\exists \mathcal{D} : \mathcal{S}_t \to \{\text{true}, \text{false}\} \tag{H.11}$$

such that:

$$\mathcal{D}(\mathcal{S}_t) = \text{true} \implies \text{failure detected} \tag{H.12}$$

In RCO:

$$\mathcal{D} = \mathcal{I}_{\text{RCO}} \tag{H.13}$$

H.8 Failure Containment Principle

The RCO protocol enforces:

$$\text{Failure} \implies \text{Reject} \tag{H.14}$$

ensuring that failures do not propagate into accepted system state.

H.9 Global Failure Invariant

$$\forall t, \quad \text{accept}(D_t) \implies \text{no failure occurred at } t \tag{H.15}$$

H.10 Representation-Level Failures

H.10.1 Non-Canonical Encoding

Failure occurs if:

$$D_1 \equiv D_2 \wedge \mathcal{S}(D_1) \neq \mathcal{S}(D_2) \tag{H.16}$$

This leads to:

$$H_1 \neq H_2 \tag{H.17}$$

H.10.2 Impact

$$\text{Severity} = \|\Delta L_t\| \tag{H.18}$$

H.10.3 Mitigation

RCO enforces deterministic serialization:

$$\forall D, \quad \mathcal{S}(D) \text{ is canonical} \tag{H.19}$$

H.11 Cryptographic Failures

H.11.1 Hash Collision

Failure occurs if:

$$\exists D_1 \neq D_2 \text{ such that } \mathcal{H}(D_1) = \mathcal{H}(D_2) \tag{H.20}$$

H.11.2 Probability

$$P(f) = \epsilon_{\mathcal{H}} \tag{H.21}$$

H.11.3 Impact

$$\text{Severity} \approx \text{maximum} \tag{H.22}$$

H.11.4 Mitigation

Use collision-resistant hash functions.

H.11.5 Signature Forgery

Failure occurs if:

$$\text{Verify}(M, \Sigma, PK) = \text{true for forged } \Sigma \tag{H.23}$$

H.11.6 Probability

$$P(f) = \epsilon_{\text{sig}} \tag{H.24}$$

H.12 Distributed System Failures

H.12.1 Validator Failure

If validator V_i fails:

$$\sigma_i = \emptyset \tag{H.25}$$

System continues if:

$$|\mathcal{V}_{\text{active}}| \geq t \tag{H.26}$$

H.12.2 Network Delay

Messages delayed:

$$T_{\text{delay}} > T_{\text{threshold}} \tag{H.27}$$

H.12.3 Impact

$$\text{Liveness reduced} \tag{H.28}$$

H.13 Adversarial Failures

H.13.1 Injection Attempt

$$D'_t \not\equiv D_t \tag{H.29}$$

H.13.2 Detection

$$\mathcal{I}_{\text{RCO}}(D'_t) = \text{reject} \tag{H.30}$$

H.14 Operational Failures

H.14.1 Storage Failure

Loss of record:

$$\mathcal{R}_t \notin \mathcal{S}_{\text{log}} \tag{H.31}$$

H.14.2 Mitigation

Replication:

$$\mathcal{S}_{\text{dist}} = \bigcup \mathcal{S}_i \tag{H.32}$$

H.15 Quantitative Risk Bound

The total system risk is:

$$R_{\text{system}} \leq \epsilon_{\mathcal{H}} + \epsilon_{\text{sig}} + \epsilon_{\text{threshold}} \tag{H.33}$$

H.16 Conclusion

This section establishes a formal framework for analyzing failures in the RCO protocol. By defining failure modes across multiple layers and quantifying their probabilities and impacts, we demonstrate that the protocol either prevents failures or bounds their effects under standard assumptions. This provides strong assurance of system robustness and reliability.

H.17 Component-Level Failure Analysis and Propagation Modeling

The RCO protocol is composed of multiple deterministic components, each responsible for enforcing a specific invariant. A comprehensive failure analysis requires evaluating each component independently and then analyzing how failures propagate through the system pipeline. This section presents a detailed component-level FMEA and establishes formal containment properties.

H.18 Component Decomposition

Let the system pipeline be defined as:

$$\mathcal{P} = \mathcal{V}_{\text{verify}} \circ \mathcal{A}_{\text{agg}} \circ \mathcal{S}_{\text{sign}} \circ \mathcal{B}_{\text{state}} \circ \mathcal{C}_{\text{lineage}} \circ \mathcal{H}_{\text{hash}} \circ \mathcal{S}_{\text{canon}} \tag{H.34}$$

Each component introduces potential failure modes.

H.19 Canonicalization Component FMEA

H.19.1 Failure Mode C1: Non-Deterministic Key Ordering

$$\exists D \text{ such that } \mathcal{S}_i(D) \neq \mathcal{S}_j(D) \tag{H.35}$$

H.19.2 Effect

$$H_t^{(i)} \neq H_t^{(j)} \implies L_t^{(i)} \neq L_t^{(j)} \tag{H.36}$$

H.19.3 Severity

$$\text{Severity}(C1) = \|\Delta L_t\| \to \text{high} \tag{H.37}$$

H.19.4 Detection

$$\mathcal{I}_{\text{RCO}} \to \text{reject} \tag{H.38}$$

H.19.5 Containment Proof

Since signature aggregation requires identical M_t:

$$M_t^{(i)} \neq M_t^{(j)} \implies \Sigma_t \text{ invalid} \tag{H.39}$$

Thus failure is contained.

H.20 Hashing Component FMEA

H.20.1 Failure Mode H1: Hash Collision

$$\mathcal{H}(B_1) = \mathcal{H}(B_2), \quad B_1 \neq B_2 \tag{H.40}$$

H.20.2 Propagation

$$H_t^{(1)} = H_t^{(2)} \implies L_t^{(1)} = L_t^{(2)} \tag{H.41}$$

H.20.3 Probability

$$P(H1) = \epsilon_{\mathcal{H}} \tag{H.42}$$

H.20.4 Containment

No containment at protocol level; relies on cryptographic assumption.

H.21 Lineage Engine FMEA

H.21.1 Failure Mode L1: Incorrect Lineage Update

$$L'_t \neq \mathcal{H}(H_t \parallel L_{t-1}) \tag{H.43}$$

H.21.2 Propagation

$$L'_t \neq L_t \implies M'_t \neq M_t \tag{H.44}$$

H.21.3 Detection

$$\text{Verify}(M'_t, \Sigma_t, PK) = \text{false} \tag{H.45}$$

H.21.4 Containment Theorem

Theorem H.1. *Any lineage inconsistency results in rejection with probability* $1 - \epsilon_{sig}$.

Proof. Invalid lineage produces mismatched message M'_t. Without valid signatures, verification fails. □

H.22 State Binding FMEA

H.22.1 Failure Mode S1: Incorrect State Hash

$$\mathcal{H}_S(S'_t) \neq \mathcal{H}_S(S_t) \tag{H.46}$$

H.22.2 Propagation

$$M'_t \neq M_t \tag{H.47}$$

H.22.3 Detection

$$\text{Verify}(M'_t, \Sigma_t, PK) = \text{false} \tag{H.48}$$

H.23 Validator Component FMEA

H.23.1 Failure Mode V1: Malicious Validator

$$\sigma_i = \text{Sign}(M'_t, K_i) \tag{H.49}$$

H.23.2 Condition

$$|\mathcal{C}| < t \tag{H.50}$$

H.23.3 Containment

$$\text{Aggregate}(\sigma_i) \text{ invalid} \tag{H.51}$$

H.23.4 Failure Mode V2: Validator Crash

$$\sigma_i = \emptyset \tag{H.52}$$

H.23.5 Impact

$$\text{Liveness degraded} \tag{H.53}$$

H.24 Aggregation Layer FMEA

H.24.1 Failure Mode A1: Insufficient Signatures

$$|\{\sigma_i\}| < t \tag{H.54}$$

H.24.2 Outcome

$$\text{reject} \tag{H.55}$$

H.24.3 Failure Mode A2: Invalid Aggregation

$$\Sigma_t \neq \text{Aggregate}(\sigma_i) \tag{H.56}$$

H.24.4 Detection

$$\text{Verify}(\Sigma_t) = \text{false} \tag{H.57}$$

H.25 Verification Engine FMEA

H.25.1 Failure Mode VFY1: False Acceptance

$$\text{Verify}(M'_t, \Sigma'_t, PK) = \text{true} \tag{H.58}$$

H.25.2 Probability

$$P(VFY1) = \epsilon_{\text{sig}} \tag{H.59}$$

H.26 Failure Propagation Model

Let failure at stage i be f_i. The propagated failure is:

$$f_{i+1} = \mathcal{T}(f_i) \tag{H.60}$$

where $\mathcal{T}$ is the transformation function.

H.26.1 Propagation Bound

$$\|f_{i+1}\| \leq L_i \|f_i\| \tag{H.61}$$

H.26.2 Containment Condition

$$\exists j \text{ such that } \mathcal{I}_{\text{RCO}}(f_j) = \text{reject} \tag{H.62}$$

H.27 Global Containment Theorem

Theorem H.2. *All non-cryptographic failures are detected and contained by the ingestion pipeline.*

Proof. Each failure alters M_t. Without valid threshold signatures, verification fails. Thus, failure cannot propagate into accepted state. □

H.28 Failure Isolation Property

$$f_i \not\Longrightarrow f_j \quad \text{for } i \neq j \tag{H.63}$$

H.29 Cascading Failure Prevention

$$\text{Failure} \rightarrow \text{Reject} \rightarrow \text{No propagation} \tag{H.64}$$

H.30 Quantitative Propagation Bound

$$\Pr[\text{propagated failure}] \leq \epsilon_{\text{sig}} + \epsilon_{\mathcal{H}} \tag{H.65}$$

H.31 Conclusion

This section provides a detailed component-level failure analysis and demonstrates that failures are either cryptographically negligible or deterministically contained. The propagation model and containment proofs establish that the RCO protocol prevents cascading failures and maintains system integrity under a wide range of conditions.

H.32 Extreme Edge Cases, Byzantine Scenarios, and Resilience Modeling

The preceding analysis has established that individual component failures are either prevented or contained by the RCO protocol. This section extends the analysis to extreme conditions, including Byzantine adversaries, simultaneous multi-component failures, and worst-case system stress scenarios. The objective is to characterize the precise boundaries under which the system maintains its guarantees and to formally define the conditions under which degradation or collapse may occur.

H.33 Extreme Edge Case Model

We define an extreme scenario as one in which multiple failure modes occur simultaneously:

$$\mathcal{F}_{\text{extreme}} = \{f_1, f_2, \ldots, f_k\} \tag{H.66}$$

with k maximized under system constraints.

The combined effect is:

$$\Delta S_t^{\text{extreme}} = \sum_{i=1}^{k} \Delta S_t^{(f_i)} \tag{H.67}$$

The protocol must ensure:

$$\text{accept}(\mathcal{F}_{\text{extreme}}) = \text{false} \tag{H.68}$$

unless all invariants hold.

H.34 Byzantine Failure Model

Let $\mathcal{C} \subseteq \mathcal{V}$ denote the set of compromised validators.

$$|\mathcal{C}| = f \tag{H.69}$$

H.34.1 Byzantine Capability

The adversary may:

$$\mathcal{A}_{\text{Byz}} = \{\text{sign invalid messages}, \text{withhold signatures}, \text{collude}\} \tag{H.70}$$

H.34.2 Threshold Condition

The system remains secure if:

$$f < t \tag{H.71}$$

H.34.3 Failure Condition

System compromise occurs if:

$$f \geq t \tag{H.72}$$

H.34.4 Byzantine Collapse Theorem

Theorem H.3. *If $f \geq t$, the adversary can produce valid signatures for arbitrary messages.*

Proof. The adversary controls at least t validators, enabling construction of Σ_t for any M_t. Thus verification succeeds. □

H.35 Multi-Failure Interaction Model

Consider failures across multiple layers:

$$f = (f_{\text{canon}}, f_{\text{hash}}, f_{\text{lineage}}, f_{\text{sig}}) \tag{H.73}$$

H.35.1 Joint Failure Probability

$$P(f) = \prod_i P(f_i) \tag{H.74}$$

H.35.2 Upper Bound

$$P(f) \leq \epsilon_{\mathcal{H}} \cdot \epsilon_{\text{sig}} \cdot \epsilon_{\text{threshold}} \tag{H.75}$$

which is negligible.

H.36 Worst-Case Scenario Analysis

H.36.1 Simultaneous Hash Collision and Signature Forgery

$$\mathcal{H}(D_1) = \mathcal{H}(D_2) \wedge \text{Forge}(\Sigma) \tag{H.76}$$

H.36.2 Probability

$$P = \epsilon_{\mathcal{H}} \cdot \epsilon_{\text{sig}} \tag{H.77}$$

H.36.3 Impact

$$\text{Severity} \to \text{maximum} \tag{H.78}$$

H.37 System Collapse Conditions

The system collapses if any of the following hold:

$$\text{(i)} \quad f \geq t \tag{H.79}$$

$$\text{(ii)} \quad \epsilon_{\mathcal{H}} \approx 1 \tag{H.80}$$

$$\text{(iii)} \quad \epsilon_{\text{sig}} \approx 1 \tag{H.81}$$

$$\text{(iv)} \quad \mathcal{S} \text{ is non-canonical} \tag{H.82}$$

H.38 Graceful Degradation Model

When partial failures occur:

$$\text{Failure} \rightarrow \text{Reject} \rightarrow \text{Retry} \tag{H.83}$$

H.38.1 Liveness Degradation

$$\text{Throughput} \downarrow \quad \text{Latency} \uparrow \tag{H.84}$$

H.38.2 Safety Preservation

$$\text{Invalid data never accepted} \tag{H.85}$$

H.39 Recovery Mechanisms

H.39.1 Validator Recovery

Failed validators rejoin:

$$\mathcal{V}_{\text{active}} \rightarrow \mathcal{V} \tag{H.86}$$

H.39.2 State Reconstruction

Using lineage:

$$L_t \rightarrow \{D_0, \ldots, D_t\} \tag{H.87}$$

H.39.3 Replay Recovery

Buffered inputs are reprocessed:

$$\mathcal{B} \to \mathcal{I}_{\text{RCO}} \tag{H.88}$$

H.40 Resilience Theorem

Theorem H.4. *If $f < t$ and cryptographic assumptions hold, the system maintains safety under arbitrary failure combinations.*

Proof. Failures either alter M_t or prevent threshold formation. In both cases, verification fails or data is rejected. □

H.41 Long-Term Stability

Over time horizon T:

$$\Pr[\text{system failure over } T] \leq T \cdot (\epsilon_{\mathcal{H}} + \epsilon_{\text{sig}} + \epsilon_{\text{threshold}}) \tag{H.89}$$

H.42 Adversarial Saturation Scenario

Adversary attempts continuous attack:

$$\mathcal{A} = \{\mathcal{A}_1, \ldots, \mathcal{A}_T\} \tag{H.90}$$

H.42.1 Bound

$$\Pr[\text{success}] \leq T \cdot \epsilon \tag{H.91}$$

H.43 Global Resilience Invariant

$$\forall t, \quad \text{accept}(D_t) \implies \text{system integrity preserved} \tag{H.92}$$

H.44 Safety vs Collapse Boundary

$$\text{Safe Region} : f < t \tag{H.93}$$

$$\text{Collapse Region} : f \geq t \tag{H.94}$$

H.45 Formal Safety Envelope

$$\mathcal{E}_{\text{safe}} = \{(f, \epsilon_{\mathcal{H}}, \epsilon_{\text{sig}}) \mid f < t, \epsilon \ll 1\} \tag{H.95}$$

H.46 Conclusion

This section establishes the behavior of the RCO protocol under extreme conditions, including Byzantine adversaries, multi-failure scenarios, and worst-case cryptographic events. The analysis demonstrates that the system maintains safety under all conditions except those that violate fundamental cryptographic assumptions or threshold constraints. Even in degraded states, the protocol preserves integrity while sacrificing liveness, ensuring that invalid telemetry never propagates into system state. This confirms the robustness of the RCO protocol as a high-assurance system suitable for safety-critical and adversarial environments.

I. Infrastructure Profiles

J. SRE Runbooks

K. Performance & Cost Analysis

L. Theoretical Foundations

M. Applications

N. Claims and Novelty

O. Appendix A: Mathematic Proofs and Congnizitions

P. Appendix B: Code Listings and Algorithmic Approaches

Q. Appendix C: Protocol Constants